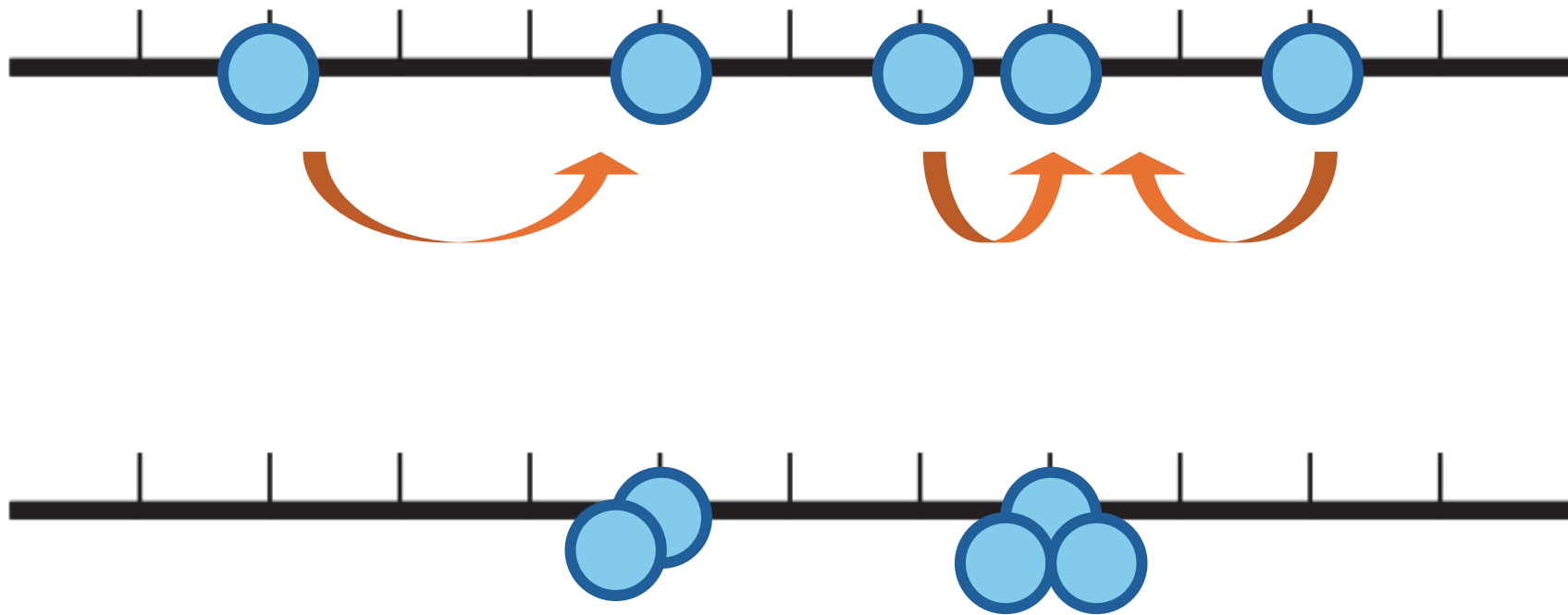


いっしょ (together)

解説担当: mendako (萩原千晴)

問題概要



小課題

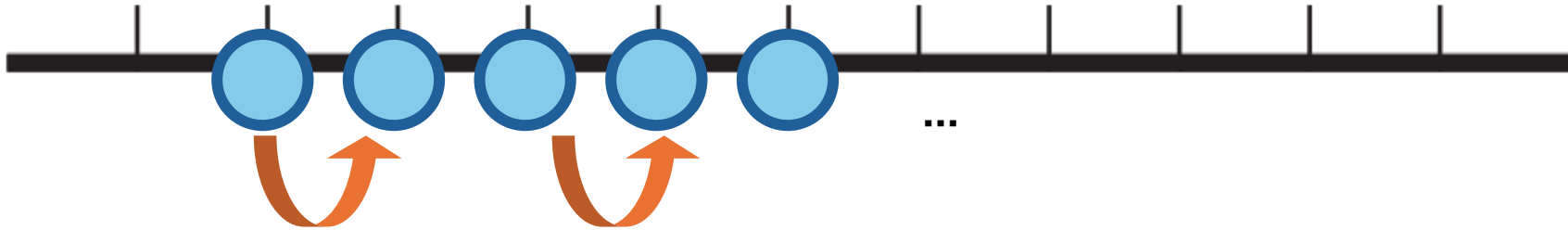
- 小課題1 (17点) $N = 4$
- 小課題2 (20点) $X_i = i$
- 小課題3 (35点) $N \leq 100, X_i \leq 100$
- 小課題4 (28点) 追加制約なし

小課題 1 ($N = 4$)



左の 2 匹、右の 2 匹で一緒になれば良さそう

小課題 2 ($X_i = i$)



隣のビーターと一緒にできれば良さそう

小課題 3 ($N \leq 100, X_i \leq 100$)

- $dp[i]$: i 匹目まで組んだときの移動距離の総和の最小値
- l 匹目から r 匹目をまとめることを考える

$dp[r] =$
 $\min(dp[r], dp[l] + \{\text{ビーバー } l+1, l+2, \dots, r \text{ をまとめるコストの最小値}\})$

どうやって一緒にする？

2 匹 or 3 匹集まることはありそう

4 匹以上集まることは...？

→ 2, 3 匹ずつに分かれて集まった方が良い

左から見て 2, 3 匹ずつ組んでいけば良い

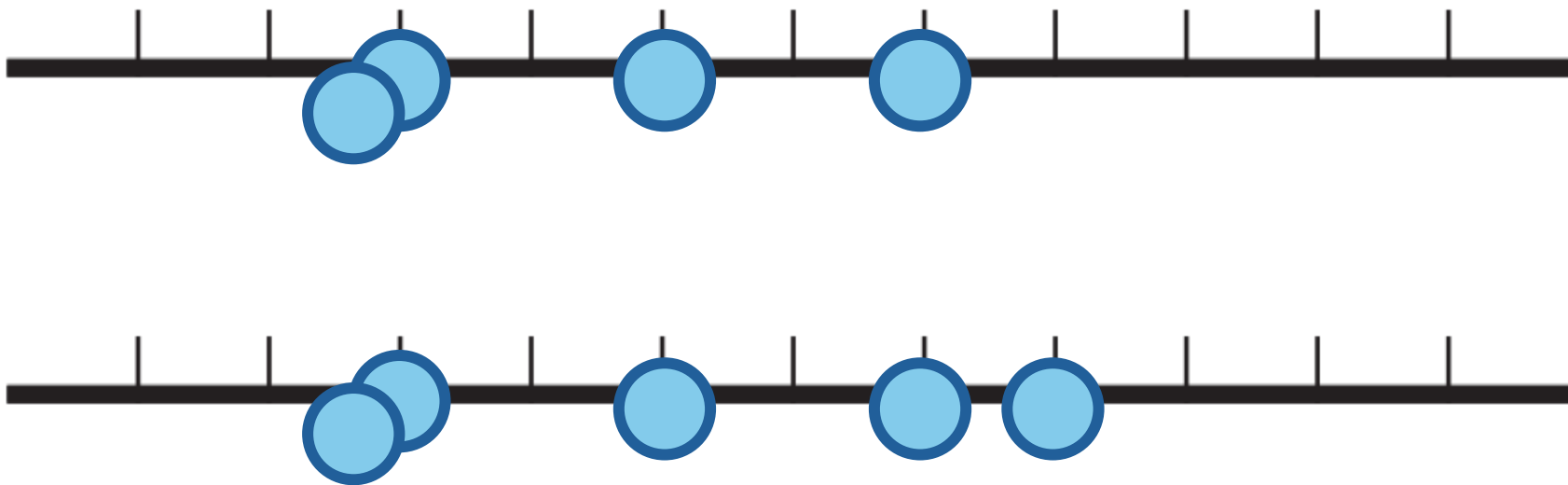
小課題 4 (満点解法)

- $O(N)$ で解きたい
 i 匹目を見ているとする
- 2 匹で一緒に
→ $i - 2$ 匹目まで決まっている状態から $i - 1 \cdot i$ 匹目が一緒になる
- 3 匹で一緒に
→ $i - 3$ 匹目まで決まっている状態から $i - 2 \cdot i - 1 \cdot i$ 匹目が一緒になる

$$dp[i] = \min(dp[i - 2] + X[i] - X[i - 1], dp[i - 3] + X[i] - X[i - 2])$$

小課題 4 (満点解法)

$$\text{dp}[i] = \min(\text{dp}[i - 2] + X[i] - X[i - 1], \text{dp}[i - 3] + X[i] - X[i - 2])$$





バームクーヘン 2 (baumkuchen 2) 解説

解説: ヘファナン色葉 (iro_)

Step 0

問題概要

0

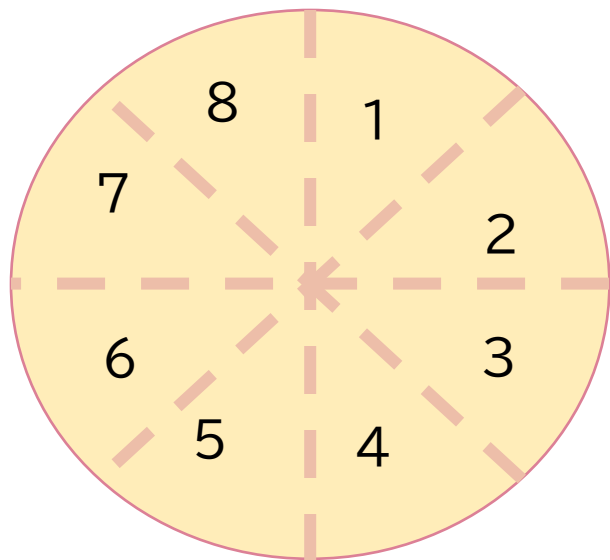
問題概要

- $K \times L$ のパーツからなるケーキがある
- Q 回、パーツ X を人 Y に 割り当てるという要望がくる
- ケーキを長さ L のパーツで切る(ピース)
- 人に割り当てる
- 条件を満たす場合を任意の mod で求める

0

問題概要

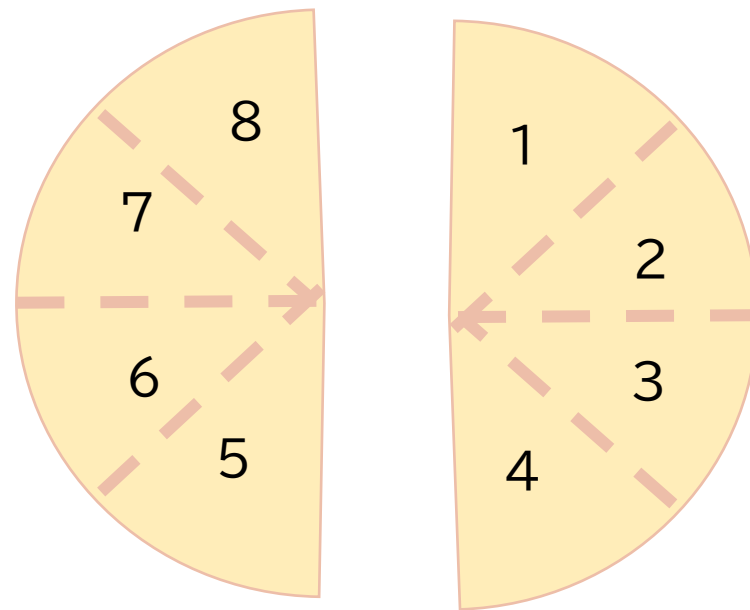
切って



割り当てる

人 1

人 2



0

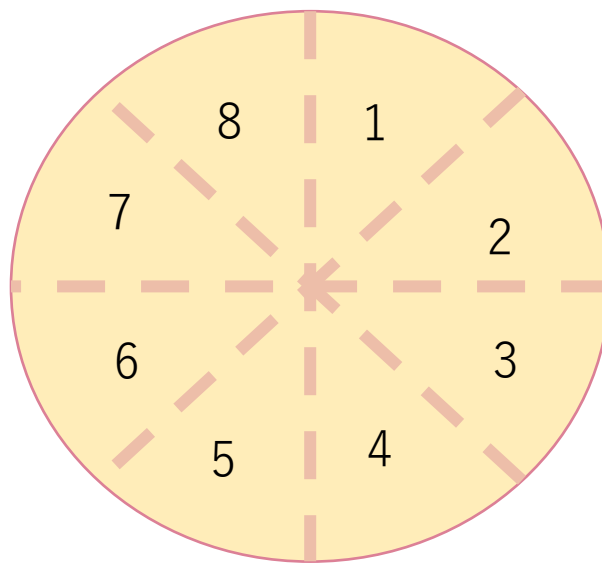
問題概要

要望が来る

要望:

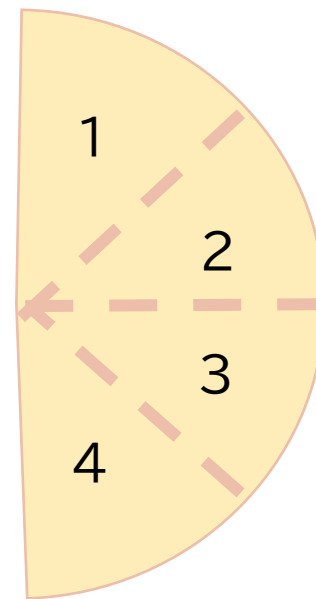
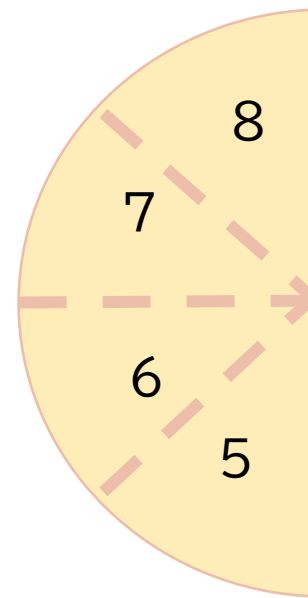
パーツ 1 を人 2 に
割り当てる

切って



人 1

人 2



0

制約

- $K, L, Q \leq 200,000$
- $10^8 \leq P \leq 10^9$
- P は素数
- $1 \leq X_j \leq KL$
- $X_j \neq X_k$
- $1 \leq Y_j \leq K$

0

小課題

- 小課題1 (13点) : $K \leq 8, L \leq 10, Q \leq 10$
- 小課題2 (14点) : $K \leq 100, L \leq 100, Q \leq 100$
- 小課題3 (18点) : $K \leq 400, L \leq 400, Q \leq 400$
- 小課題4 (17点) : $K \leq 2\,500, L \leq 2\,500, Q \leq 2\,500$
- 小課題5 (28点) : $K \leq 10$
- 小課題6 (10点) : 追加の制約はない

Subtask 1

$$K \leq 8, L \leq 10, Q \leq 10$$

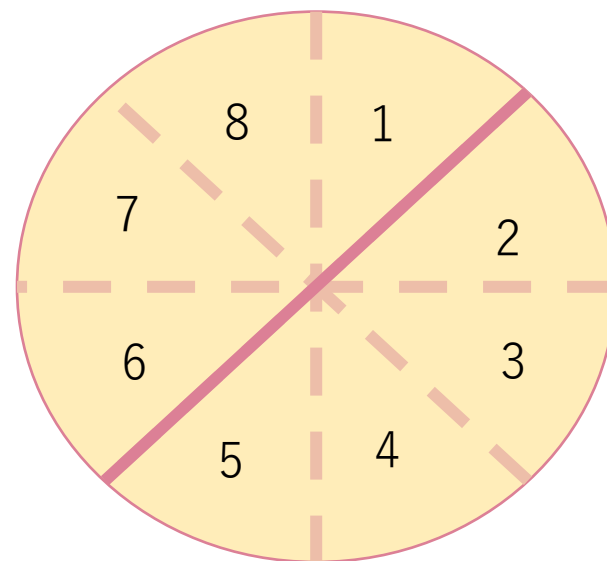
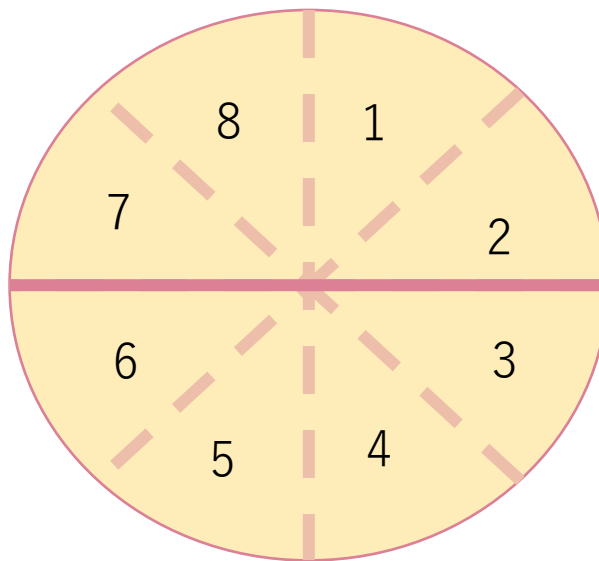
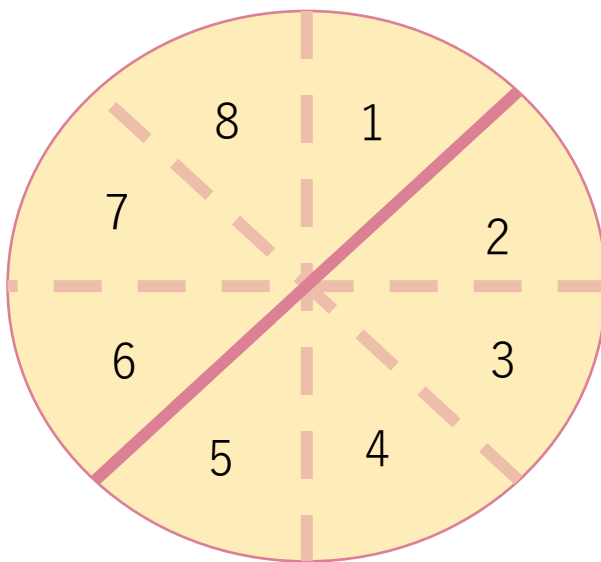
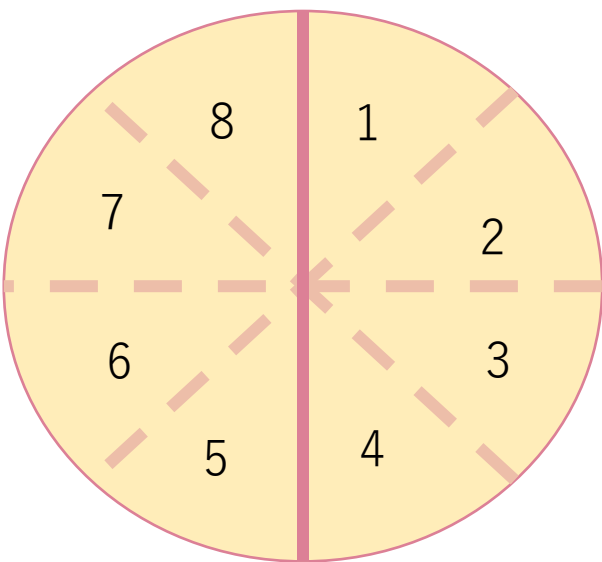
1

はじめに

- P で割った余りを答えましょう

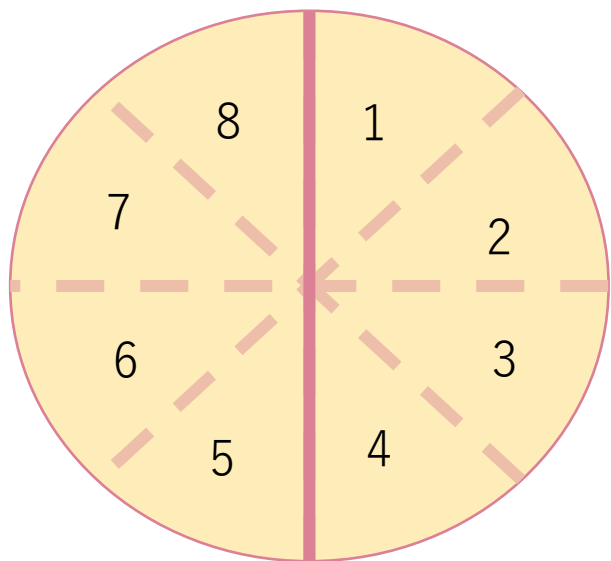
1 まずは切り分け方を考える

$$K = 2 \quad L = 4$$



1

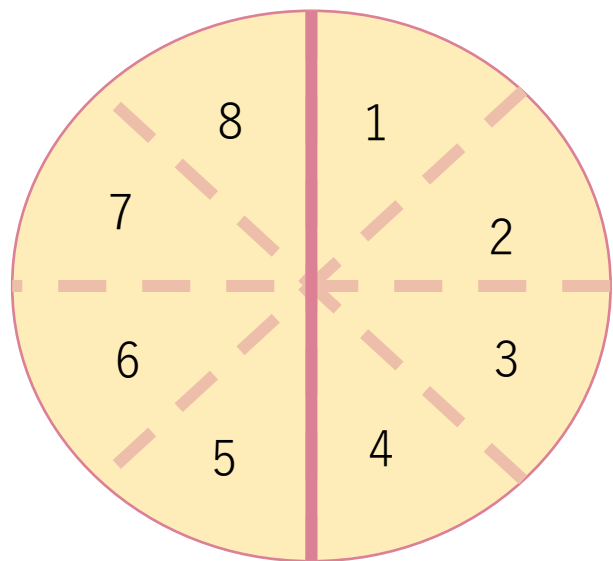
切り分け方



切り込み 8と1から
切り込み 4と5から

切り分け方は L 通りのみ！！

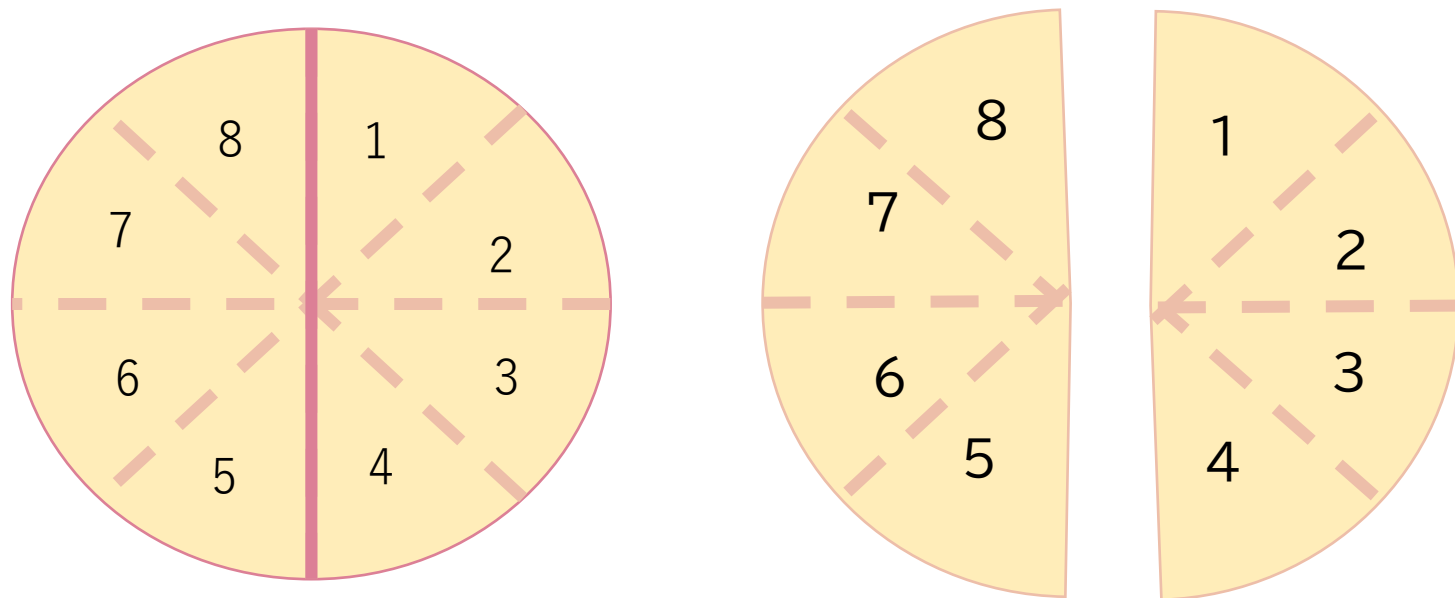
1 切り分け方は L 通りのみ



切り分け方は L 通りのみ！！
L を固定して考える

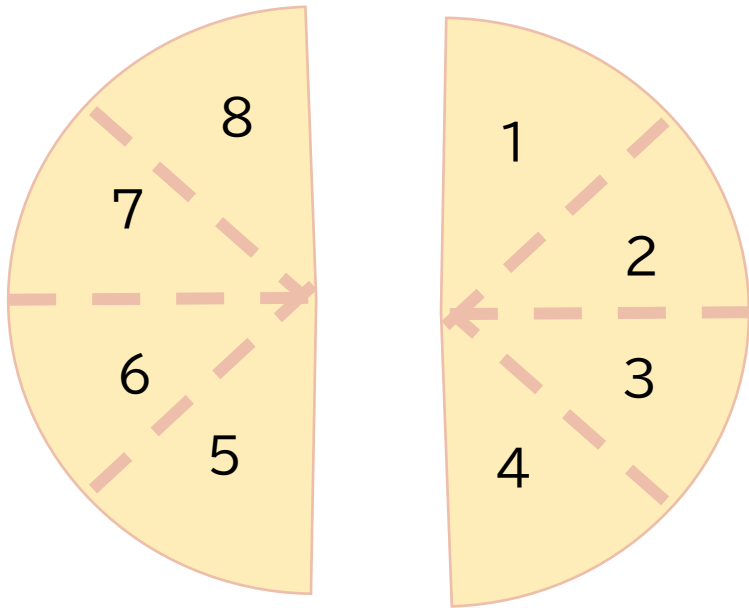
1

切り分け方を固定する



ピースを人に割り当てる
 $K!$ 通り

1 切ったケーキを割り当てる



ピースを人に割り当てる
K! 通り

要望を満たしているか確認

Subtask 2

$K \leq 100, L \leq 100, Q \leq 100$

2 切り分ける場所

- 切り分ける場所は固定して考える

2 切ったケーキを割り当てる

- パーツ $K * L$ 個を順番に見ていく
- 要望がある場合その人に割り当てる
- 要望がない場合任意の人に割り当てることができる
- 2人以上に割り当てることはできないので、場合分けする

Subtask 3

$K \leq 400, L \leq 400, Q \leq 400$

3 切ったケーキを割り当てる

- パーツ $K * L$ 個を順番に見ていく
- 高速化させたい

3 切ったケーキを割り当てる

- パーツ $K * L$ 個を順番に見ていく
- 高速化させたい
- そのピースを食べたいという要望があるか確認する $O(K)$
- あらかじめ要望が来た時に、切り分ける場所決めた時にどのピースが欲しいのか記録する

Subtask 4

$K \leq 2\,500$, $L \leq 2\,500$, $Q \leq 2\,500$

4 切ったケーキを割り当てる

- そのピースを食べたいという要望があるか確認する $O(K)$
- 高速化させたい

4 切ったケーキを割り当てる

- そのピースを食べたいという要望があるか確認する $O(K)$
- 高速化させたい
- 前計算させる
- 要望が来た時にLで切り分ける時にどのピースが欲しいのか
確認

Subtask 5

$$K \leq 10$$

5 L が大きい

- 切り分け方を固定していたら間に合わない

5 L が大きい

- 切り分け方を固定していたら間に合わない
- あらかじめ要望が来た時に、条件を更新する

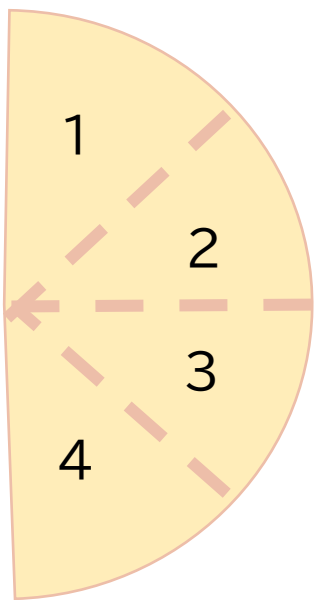
5 L が大きい

- 切り分ける方を固定していたら間に合わない
- あらかじめ要望が来た時に、条件を更新する
- 条件2つ
- 1. 人 i に割り当てられた区間の間で切ってはいけない
- 2. 人 i と人 j の間で切る
- この2つの条件を満たす切り分け方を数える
- データ構造を用いて管理する

5

条件1

1. 人 i に割り当てられた区間の間で切ってはいけない



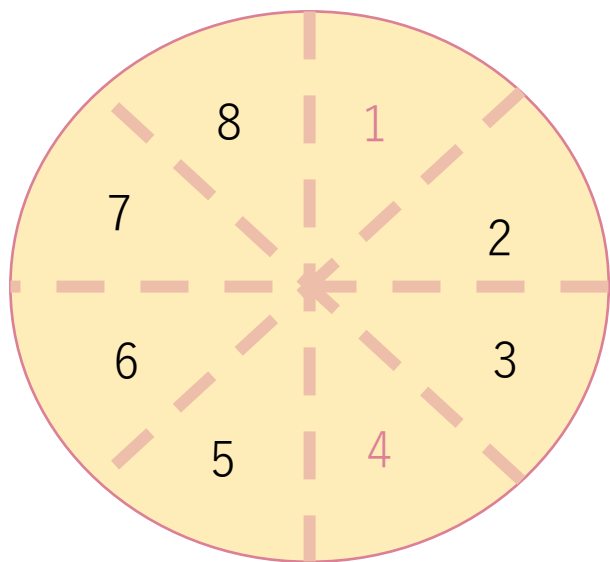
要望 パーツ 1 とパーツ 4 を割り当てる

切れ目 1 2 3 できないことができない

5 条件2

2.人 i と人 j の間で切る

K 人の要望を区間として記録し、区間と区間の間で必ず切る



パーツ 1 を人1に割り当てる

パーツ 4 を人2に割り当てる

切れ目 1 2 3 できらなければならない

Subtask 6

追加の制約はない

6 K が大きい

- 条件 2 を高速化させたい
- 毎回のクエリで要望が一つ増えるだけのため、前後の区間のみ確認すればよい

Step 7

得点分布

7

得点分布

100 点

90 点

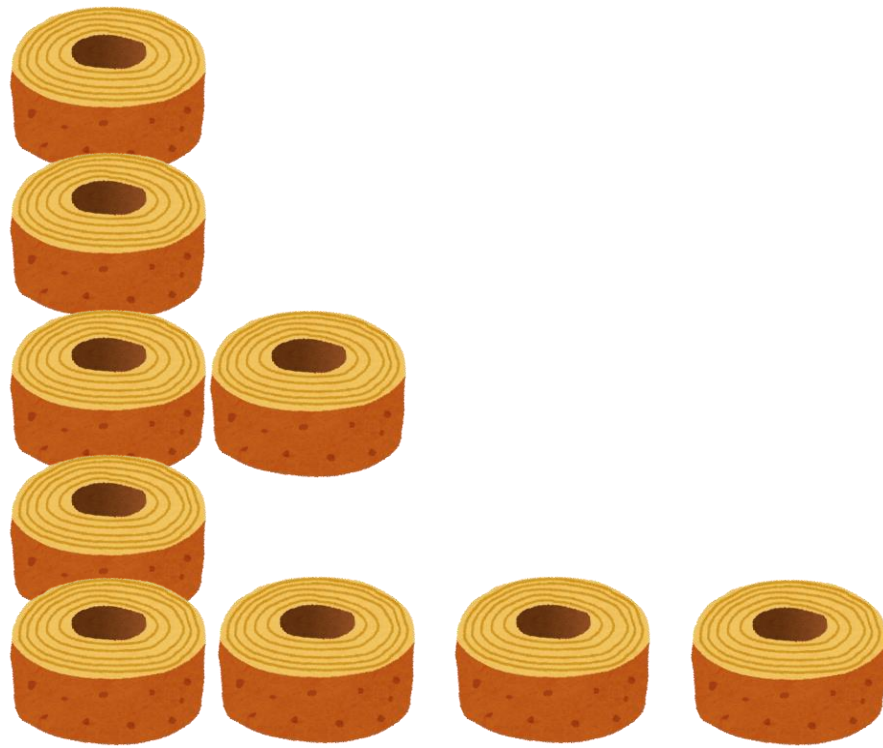
62 点

45 点

27 点

13 点

0 点



ボードゲーム

— 解説担当: 西本将樹 (maspy) —

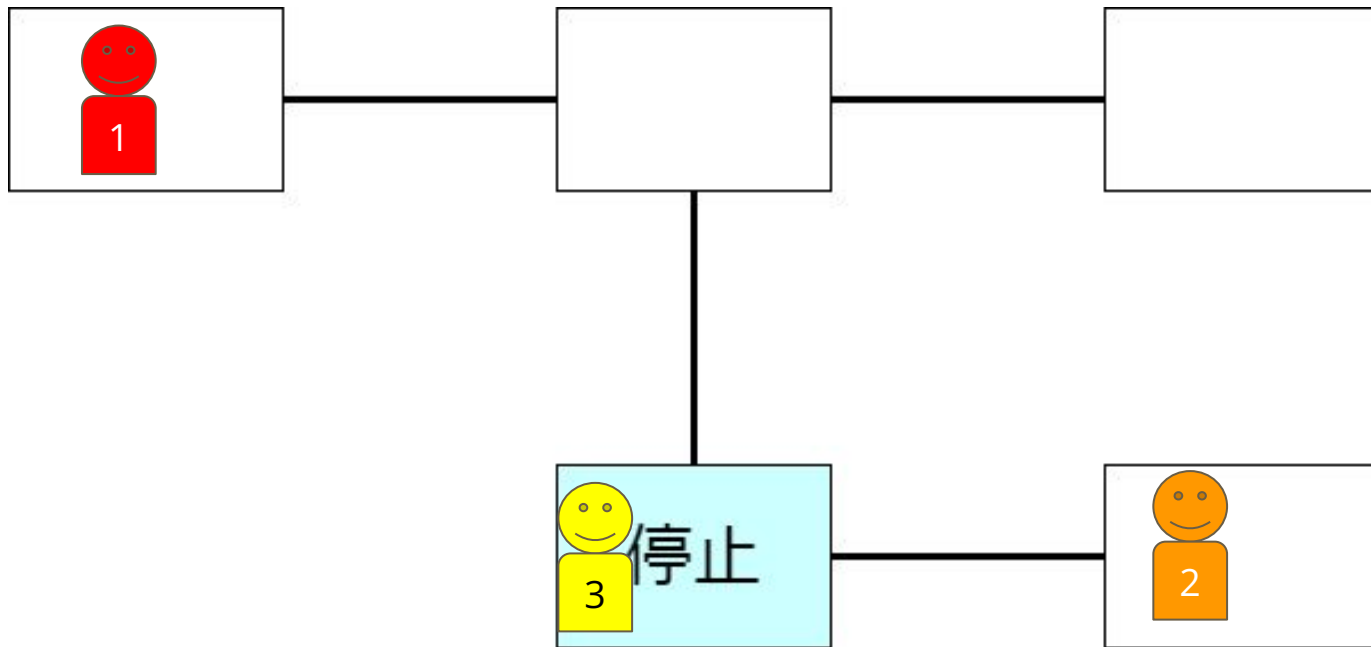
問題概要

- K 個のコマがある
- 動かし方のルールがある(問題文をよく読みましょう)
- 目標を達成するまでの最小操作回数を求めよ

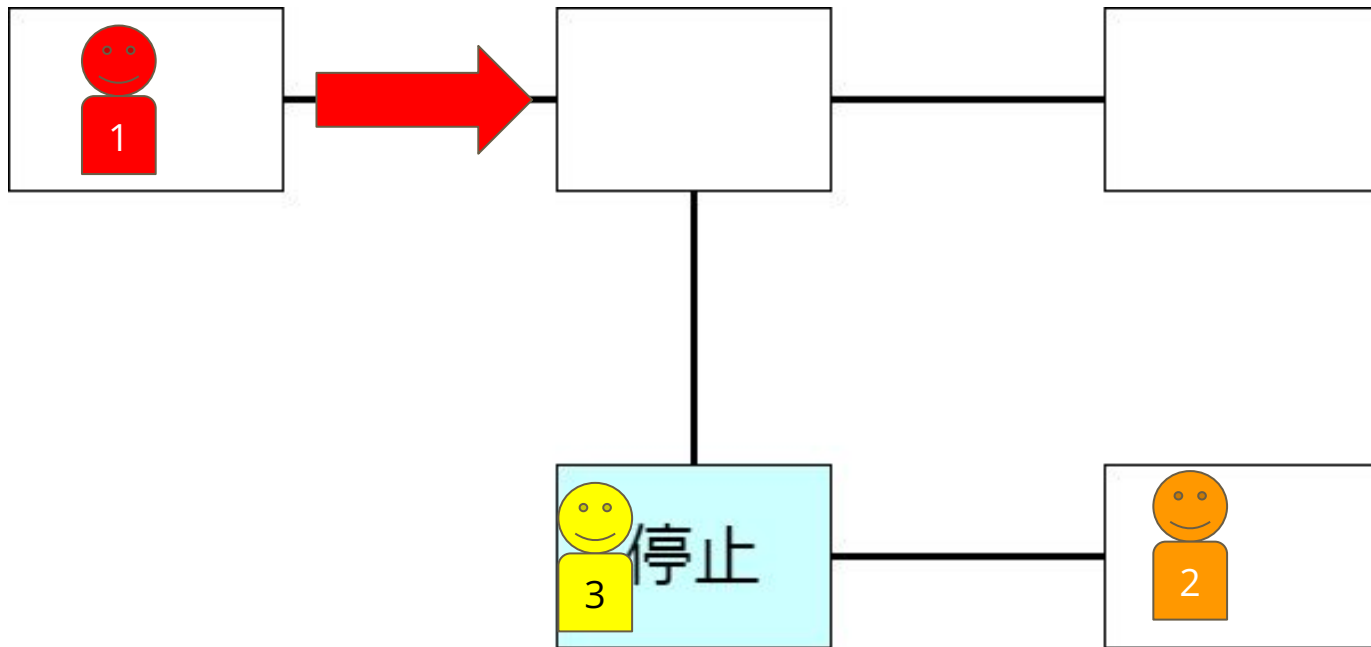
K 人ゲームと言っているが, 勝負要素等はなし.

1 人が動かすと思ってもよい.

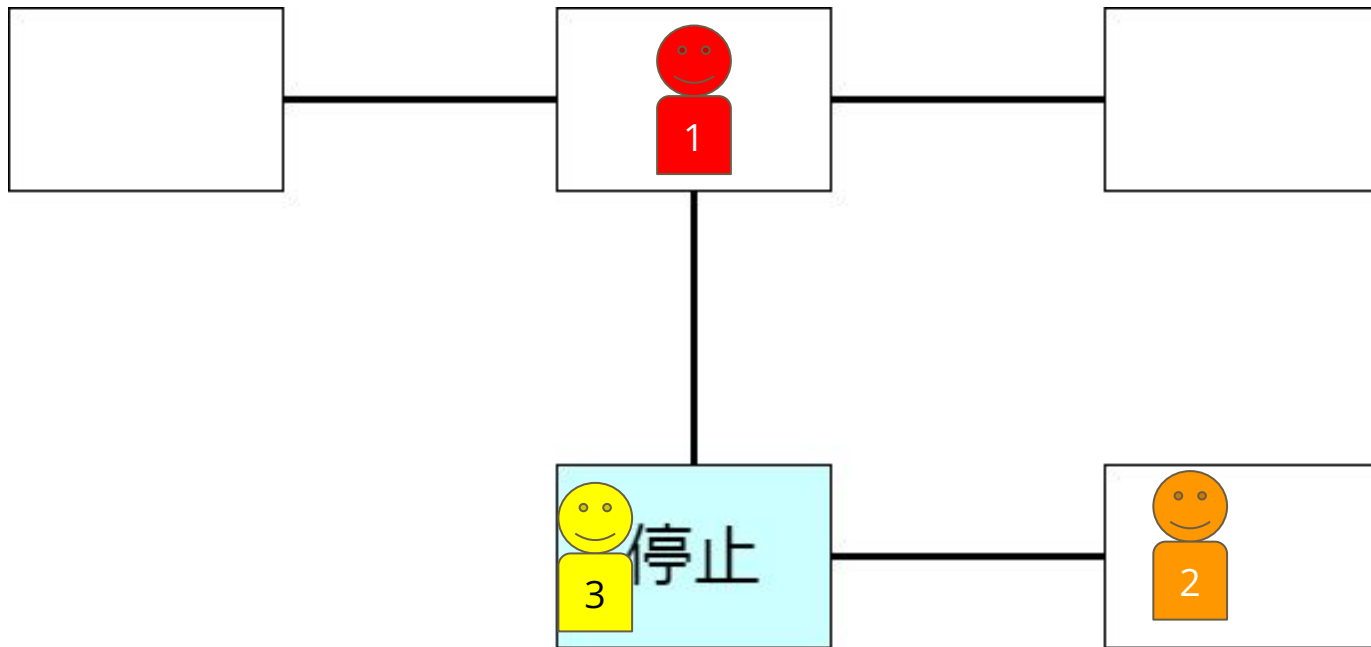
問題概要(動かし方のルール)



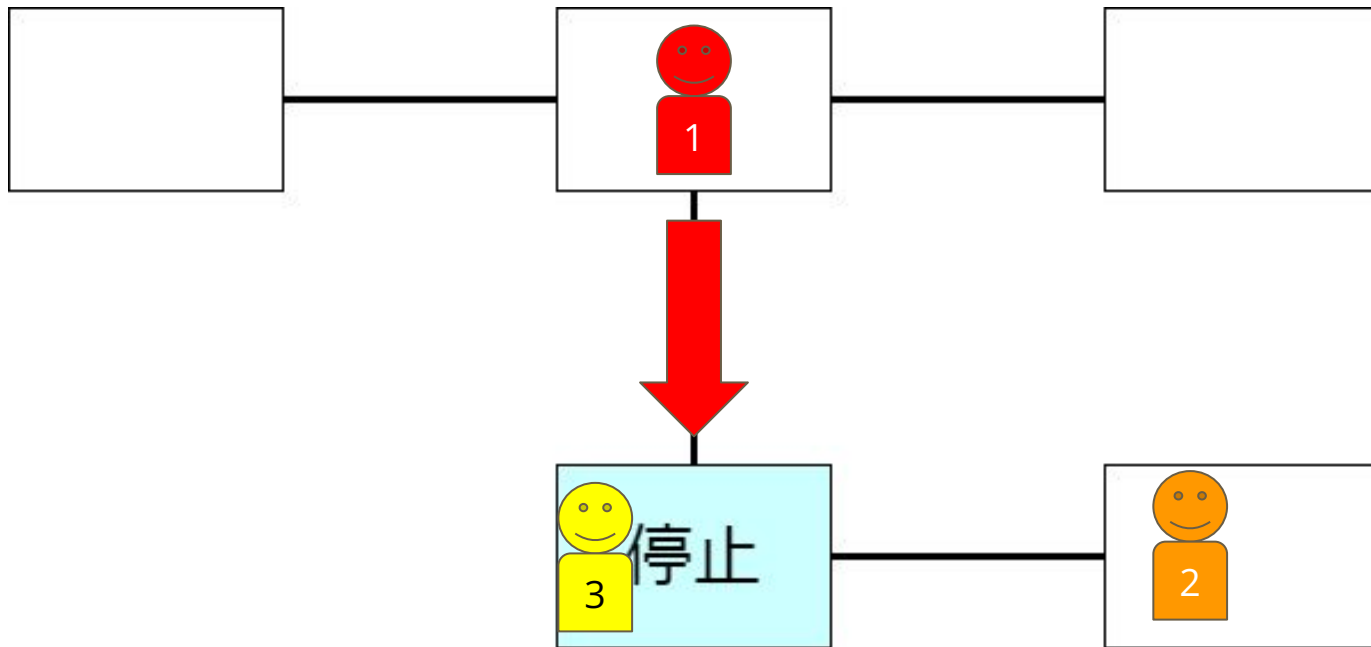
問題概要(動かし方のルール)



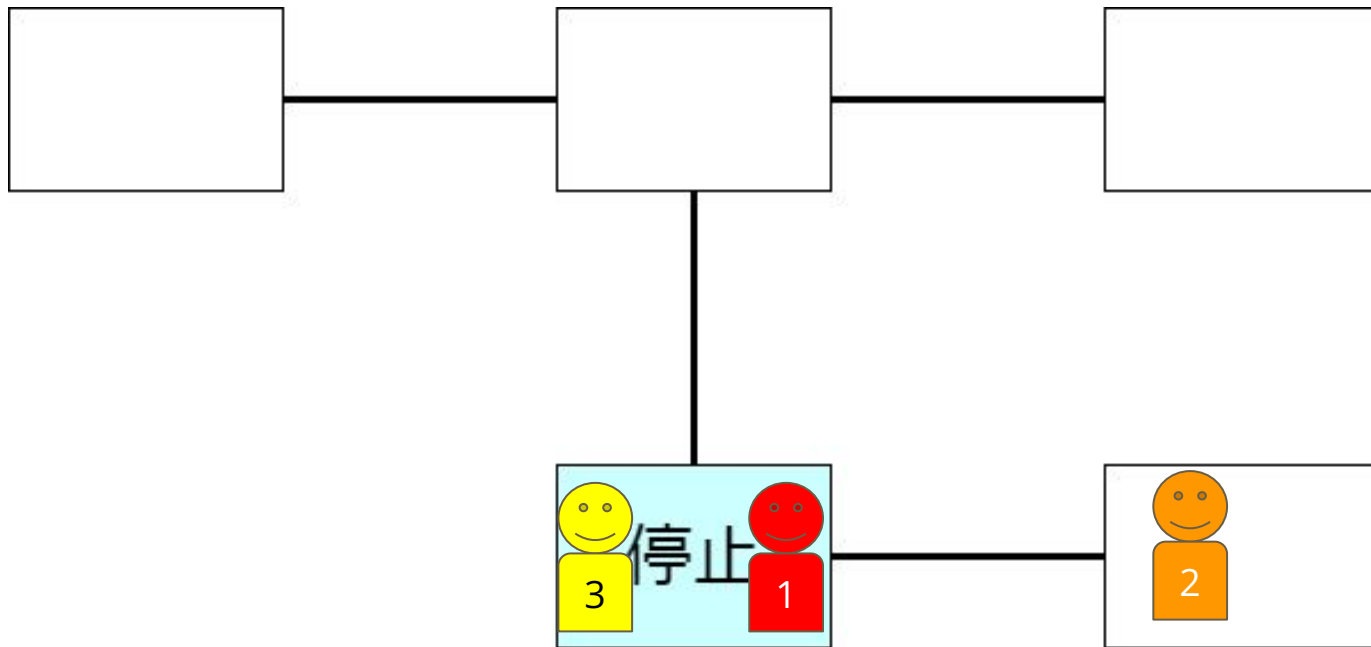
問題概要(動かし方のルール)



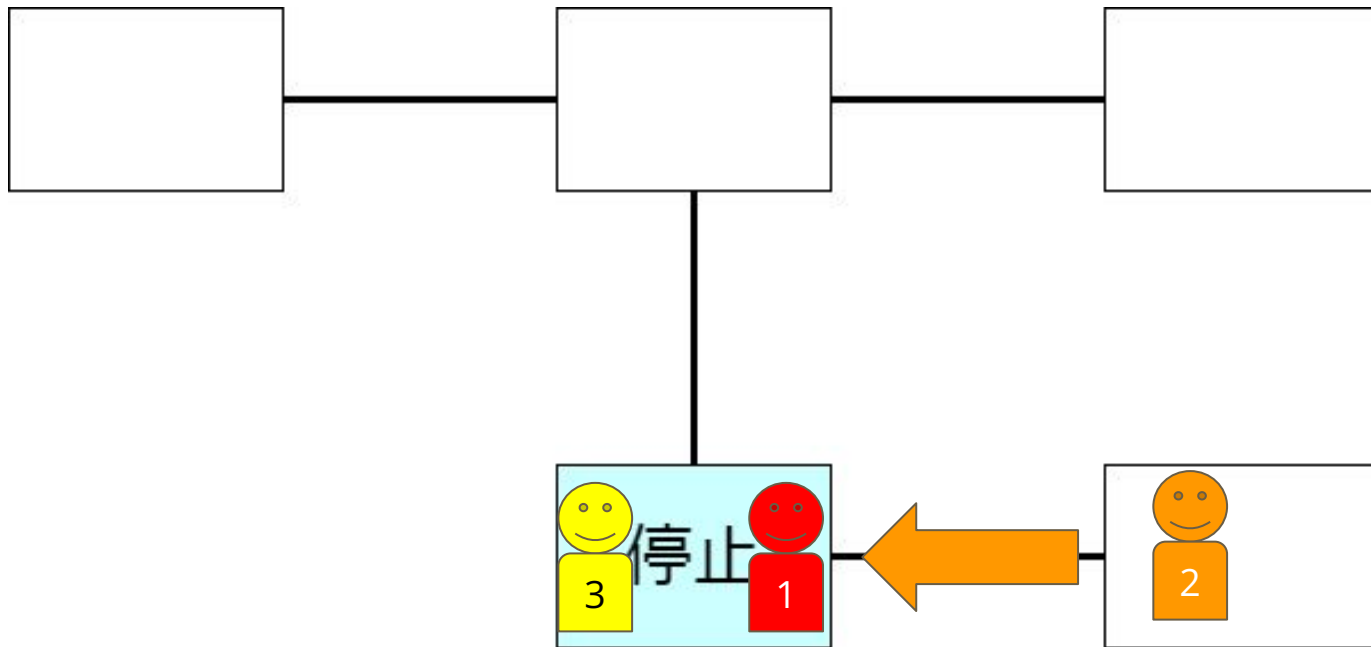
問題概要(動かし方のルール)



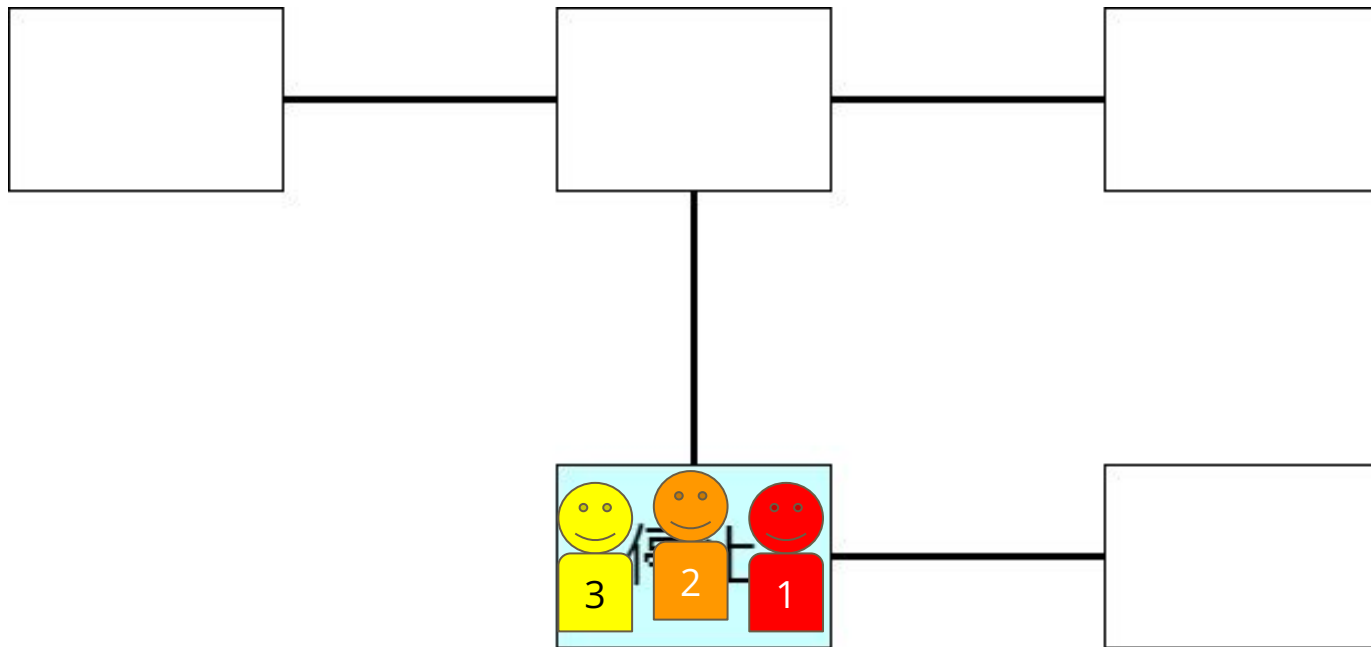
問題概要(動かし方のルール)



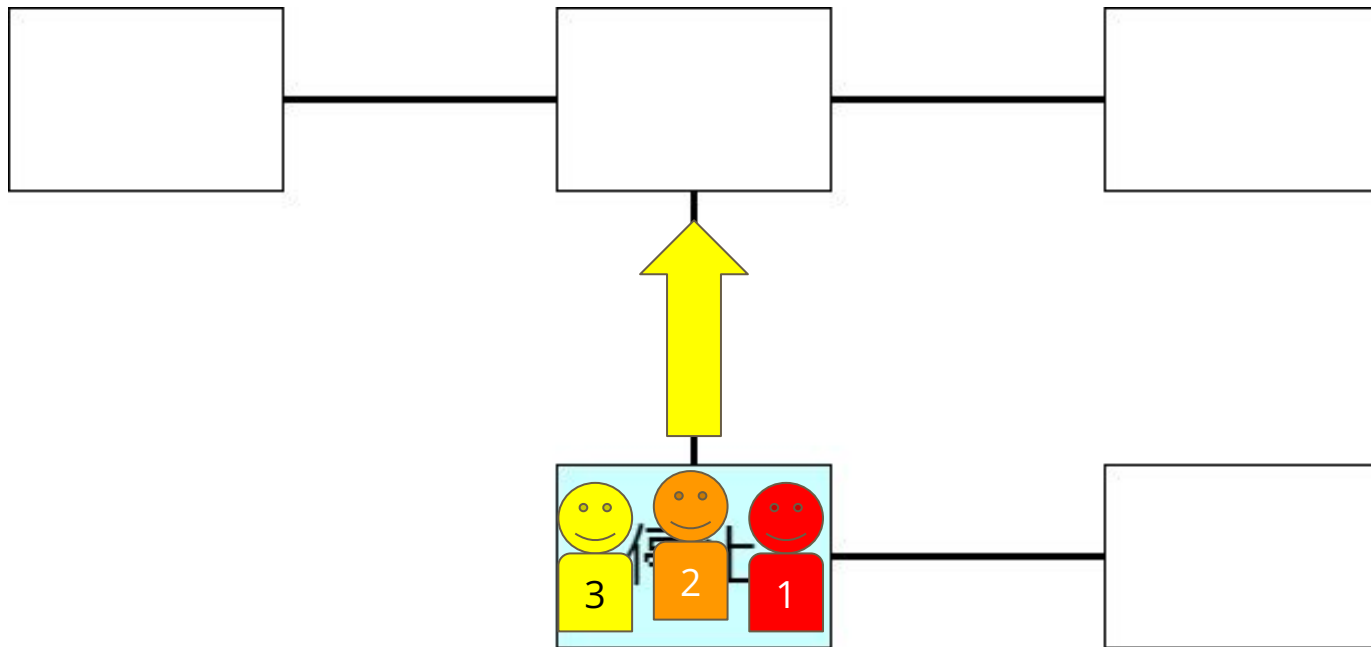
問題概要(動かし方のルール)



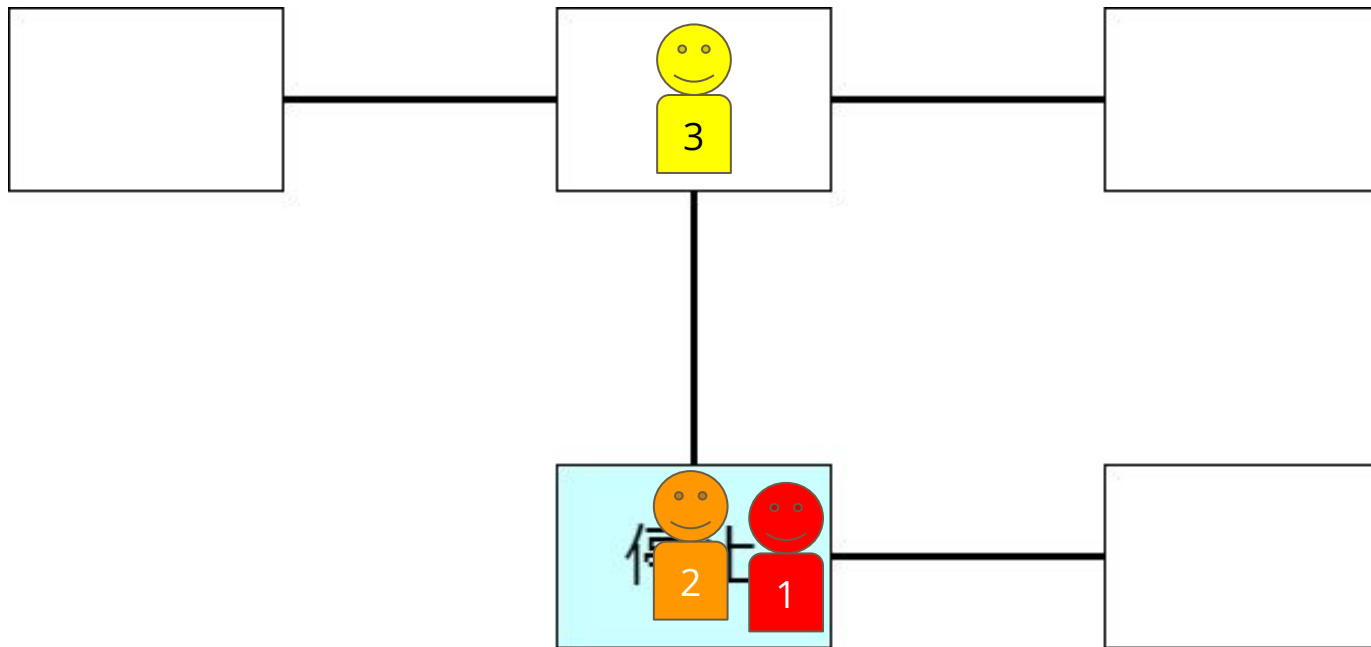
問題概要(動かし方のルール)



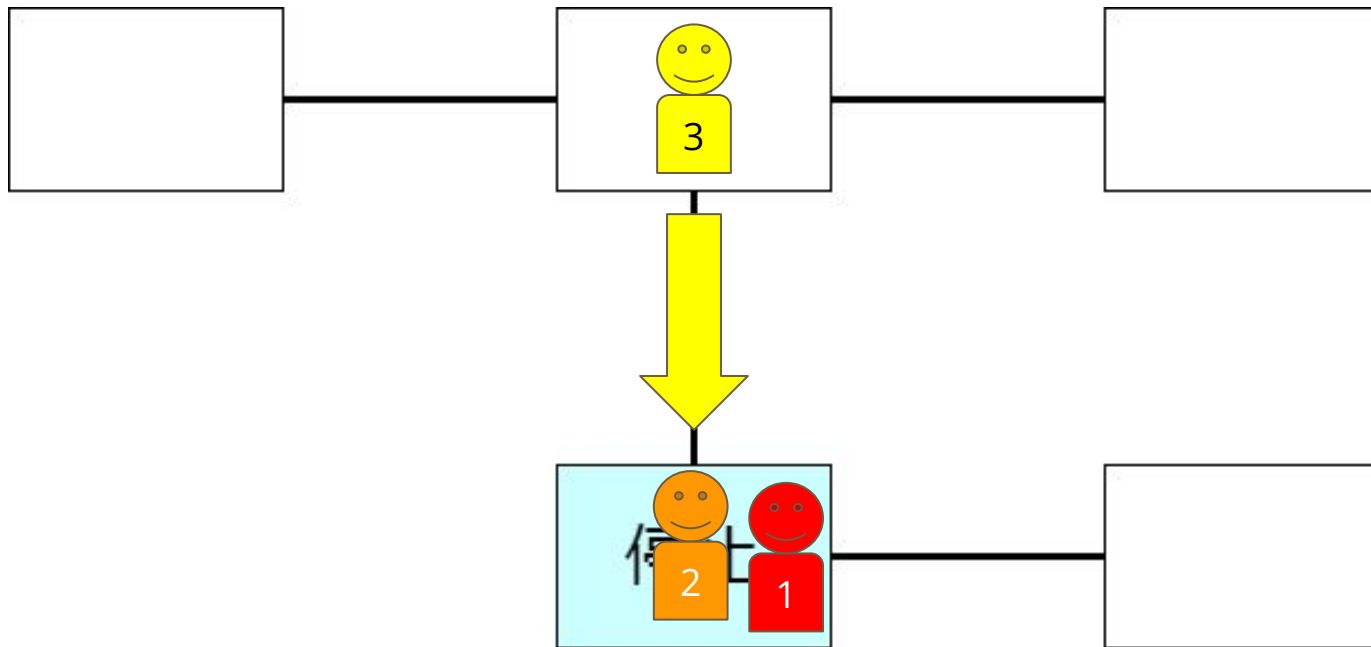
問題概要(動かし方のルール)



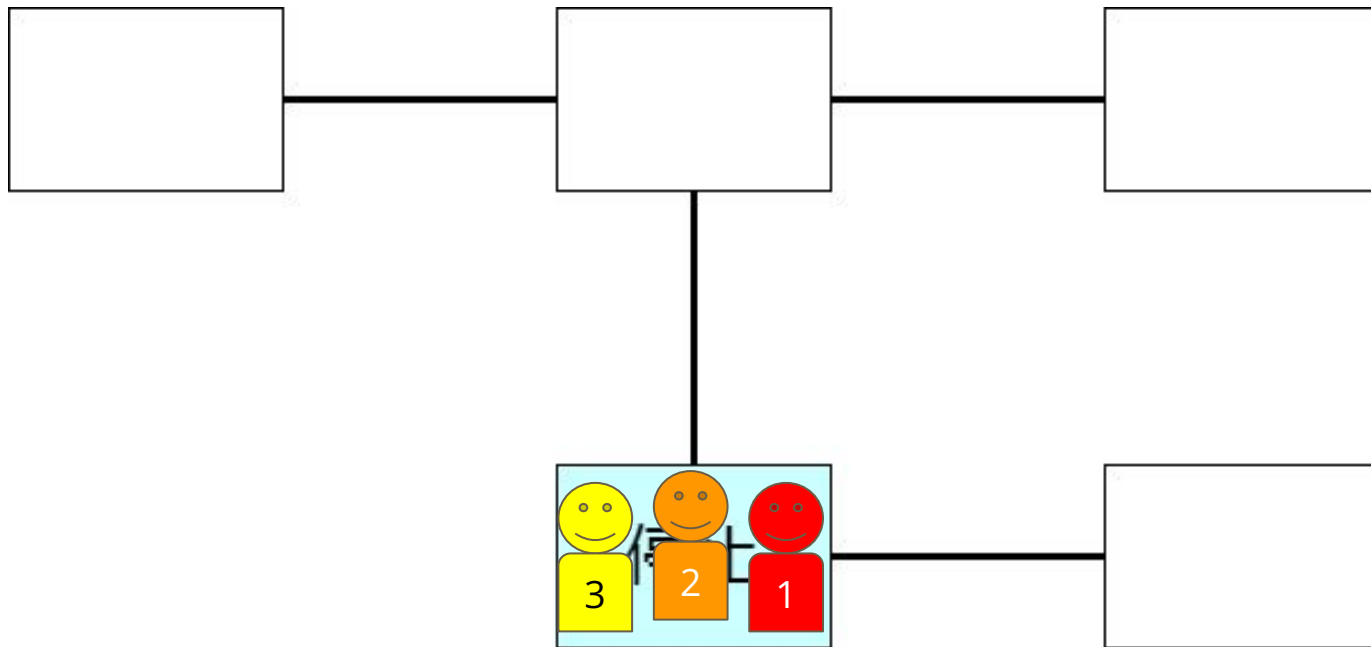
問題概要(動かし方のルール)



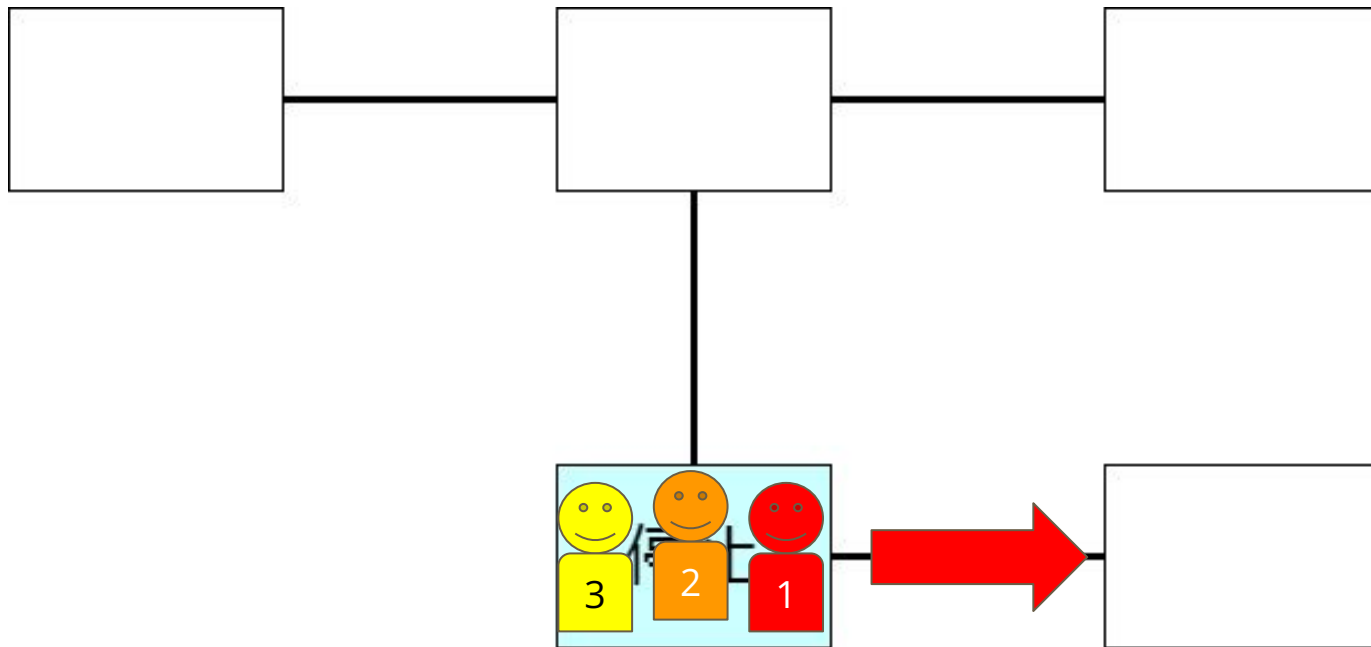
問題概要(動かし方のルール)



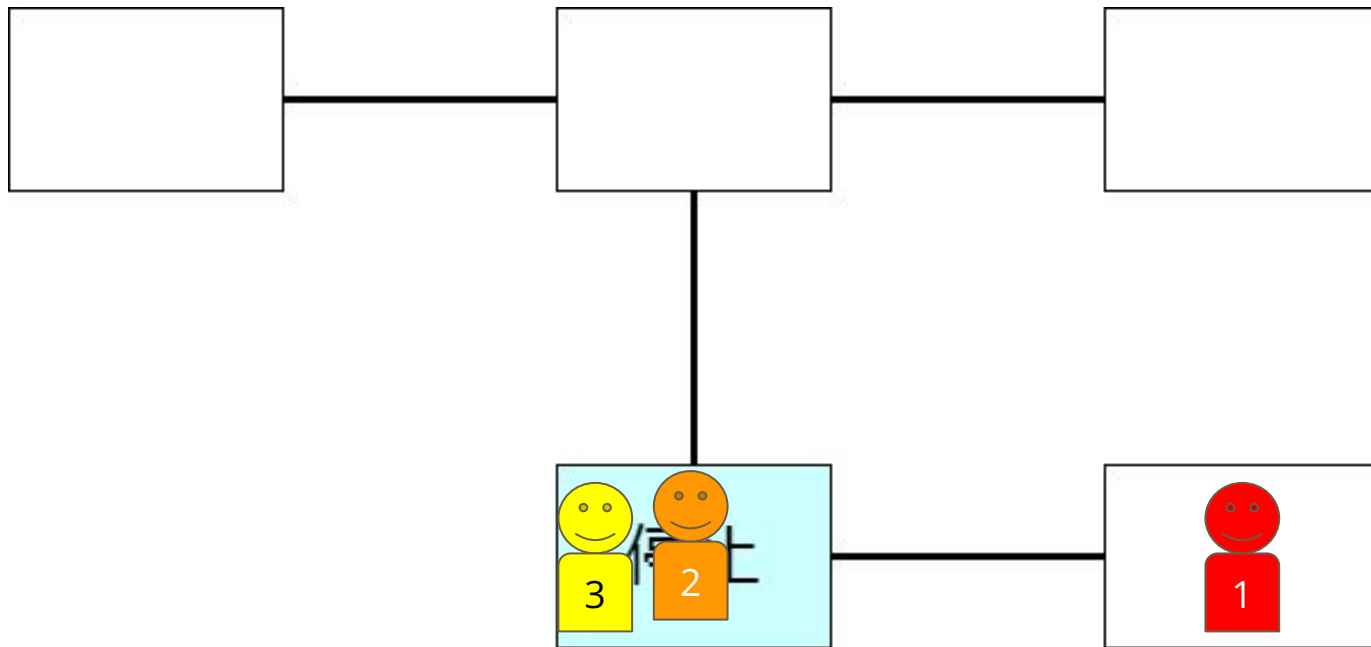
問題概要(動かし方のルール)



問題概要(動かし方のルール)



問題概要(動かし方のルール)



小課題1: 停止マスが0個 (JOI 3pts, JOIG 7pts)

小課題1: 停止マスが0個

- 単にプレイヤー 1 が動き続ける問題です.
- すべての辺重みが 1 であるような, 単一始点最短路問題
- 幅優先探索 (BFS) で解ける
- 計算量は $O(N+M)$

小課題1: 停止マスが0個

最短路を求める手法としては次が有名です

- (01) - BFS
- Dijkstra
- Warshall Floyd
- Bellman Ford, SPFA

計算量は？ 負辺はあってもよい？ 負閉路は検出可能？

小課題2: 停止マスが1個 (JOI 7pts, JOIG 12pts)

小課題2: 停止マスが1個

- プレイヤ 1 は次のどちらか
 - マス T まで停止マスを踏まずに進む
 - マス T まで停止マスを 1 回踏んで進む
- 他のプレイヤ
 - 停止マス(唯一)まで進む

小課題2: 停止マスが1個

- プレイヤ 1 は次の**どちらか** → **最後に考える**
 - マス T まで停止マスを踏まずに進む → **計算可能**
 - マス T まで停止マスを 1 回踏んで進む → **計算可能**
- 他のプレイヤー
 - 停止マス(唯一)まで進む → **計算可能**

小課題2: 停止マスが1個

【解法】

- プレイヤ 1: 停止マスを 0 回・1 回踏んで T に行くときの移動回数を計算する
- 他のプレイヤー: 停止するまでの移動回数を計算する

小課題2: 停止マスが1個

【解法】

- プレイヤ 1: 停止マスを 0 回・1 回踏んで T に行くときの移動回数を計算する

→ 2N 状態の最短路問題

- 他のプレイヤ: 停止するまでの移動回数を計算する

→ 停止マスを始点として最短路

小課題2: 停止マスが1個

【実装の注意】

停止マスの扱いに注意しよう

- プレイヤ 1 の場合: 到達する瞬間は数えない
- 他のプレイヤの場合: 停止マスから距離 0 だとターン終了まで 2 回かかる

小課題3: 停止マスが2個 (JOI 7pts, JOIG 12pts)

小課題3: 停止マスが2個

- プレイヤ 1 は次のどれか
 - マス T まで停止マスを踏まずに進む
 - マス T まで停止マスを 1 回踏んで進む
 - マス T まで停止マスを 2 回踏んで進む
- 他のプレイヤ
 - 停止マスのどちらかまで進む
 - 2 回目のターンは 1 or 2 手で終わらせる

小課題3: 停止マスが2個

- プレイヤ 1 は次の**どれか** → **最後に決める**
 - マス T まで停止マスを踏まずに進む → **計算可能**
 - マス T まで停止マスを 1 回踏んで進む → **計算可能**
 - マス T まで停止マスを 2 回踏んで進む → **計算可能**
- 他のプレイヤー
 - 停止マスのどちらかまで進む → **計算可能**
 - 2 回目のターンは 1 or 2 手で終わらせる → **計算可能**

小課題3: 停止マスが2個

答えを求めるには

- プレイヤ 1: 停止マスを 0 回・1 回・2 回踏んで T に行くときの移動回数を計算する

→ **3N 状態の最短路問題**

- 他のプレイヤ: 停止するまでの移動回数を計算する, 2 回目のターン終了にかかる時間を計算する

→ **最短路問題 2 回 / 停止マスが隣接しているかチェック**

小課題4 : N, M, K は 3000 以下 (JOI 19pts, JOIG 23pts)

小課題4: N, M, K は 3000 以下

これまでの解法の通り, プレイヤ 1 が停止マスを踏む回数 n を固定したときのことを計算することを考えましょう.

- **プレイヤ 1 について計算したいもの**
 - 停止マスを n 回踏んでマス T に到達するときの移動回数
- **他のプレイヤについて計算したいもの**
 - 自分のターンを n 回消化するためにかかる移動回数
- $n = 0, 1, \dots, N$ すべてに対してこれが求まれば解ける

小課題4: N, M, K は 3000 以下

- プレイヤ 1 について計算したいもの
 - 停止マス n 回踏んでマス T に到達するときの移動回数

→ これは **N^2 状態の最短路問題**

$\text{dist}[k][T] :=$ マス T に到達し, 到達時点で停止を k 回行っているときの最小移動回数 (T での停止は含めない)

小課題4: N, M, K は 3000 以下

- **他のプレイヤーについて計算したいもの**
 - 自分のターンを n 回消化するためにかかる移動回数

この部分について, 2 つの解法を紹介します. 「2 つめ」は小課題5のときに扱います.

小課題4: N, M, K は 3000 以下

次の N^2 状態の最短路問題と考えることができます.

- マス v で自分のターンを始めて, k ターン消化を目標としている状態 (v, k)
- v, w が隣接しているとき, (v, k) から (w, k) や $(w, k-1)$ に進める
- (v, k) から $(\text{any}, 0)$ への距離は？

小課題4: N, M, K は 3000 以下

次の N^2 状態の最短路問題と考えることができます.

- マス v で自分のターンを始めて, k ターン消化を目標としている状態 (v, k)
- v, w が隣接しているとき, (v, k) から (w, k) や $(w, k-1)$ に進める
- (v, k) から $(any, 0)$ への距離は？

→ **$(any, 0)$ から逆向きに最短路を求めればよい**

小課題1～4まとめ (JOI 36pts, JOIG 54pts)

ここまで解けていれば、問題に登場するすべての要素を正しく理解して計算に組み込んでいるといってい良いでしょう。

考えるべき状態は基本的なものばかりで、計算も BFS だけと基本的ですが、それでもすべての実装を正確にこなすのは結構大変だと思います。

少しだけ解説者なりに、**実装のためのアドバイス**をします。

実装のためのアドバイス

まず細かい計算式を正確に作るためには、変数の意味を正確に定義して、それに沿って考えることが大切です.

$\text{dist}[k][T] :=$ 停止マス k 回でマス T に居るときの移動回数



$\text{dist}[k][T] :=$ マス T に到達し、到達時点で停止を k 回行っているときの最小移動回数 (T での停止は含めない)

実装のためのアドバイス

自分が定義した $\text{dist}[k][T]$ が、マス T で停止マスを踏むタイミングを考慮しているのか等を明確に答えられることを確認してから実装に臨みましょう. ソースコードにコメントを入れておくことも有効だと思います.

```
// dist[k][v]  
// -- v につくまでにターンエンドを k 回やった  
// -- v でのターンエンドは含まない  
vc<vc<int>> dist(N, vc<int>(N, infty<int>));
```

実装のためのアドバイス

実装ミスを減らすことも大事ですが、実装ミスをしたときのリカバリも超重要です。以下の 2 つを手早く実装できるようにしておくとても便利です。

愚直な解法

- なるべく問題文で言われている通りに実装する
- 計算量は気にしない

ランダムケース生成

- 制約を満たす小さいケースを適当に作る

実装のためのアドバイス

【愚直な解法】例えばコマの場所・次に動かせるコマの番号を状態とした最短路問題を解けばよいです. とにかく全部の状態を考えて最短路問題, というのはかなりよく使います.

```
using T = pair<vc<int>, int>; // コマの場所・次に動かせるコマ
map<T, int> dist;
deque<T> que;
auto upd = [&](vc<int> X, int k, int x) -> void {
    T t = {X, k};
    if (!dist.count(t)) { chmin(ANS[X[0]], x), dist[t] = x, que.push_back(t); }
};
upd(X, 0, 0);
while (!que.empty()) {
```

実装のためのアドバイス

【ランダムケース生成】私は Python をよく使います. 何らかの言語の乱数の扱いに慣れておきましょう. 自分の実装が扱えている通りのケースであれば厳密に問題の制約に合わせる必要也没有せん. 画像のコードでは多重辺も作っています.

```
N = randint(2, 8)
M = randint(N - 1, N + 3)
K = randint(2, 4)
print(N, M, K)

for v in range(2, N + 1):
    print(randint(1, v - 1), v)
for _ in range(M - (N - 1)):
    a = randint(1, N - 1)
    b = randint(a + 1, N)
    print(a, b)

print("".join(random.choice("01") for _ in range(N)))
print(*[randint(1, N) for _ in range(K)])
```

小課題5: $K = 2$ (JOI 24pts, JOIG 26pts)

小課題5以降では小課題4までの解法の計算量を改善することが問われています.

小課題5: $K = 2$

- 小課題4での解法
- プレイヤ 1 について計算したいもの
 - 停止マスから n 回踏んでマス T に到達するときの移動回数

→ **計算したいものが N^2 通りもあるので無理**

n の範囲を絞り込んだりするのも難しい (n が巨大な解と n が小さな解の比較も明らかではない)

→ プレイヤ 1 のコスト・他プレイヤのコストを個別に求めようとするとなぜか難しい

小課題5: $K = 2$

【解法の大筋】

- 他のプレイヤーの移動回数は, まあまあ簡単な n の関数になる.
- →「**総コスト** = プレイヤ 1 の移動回数 + 他のプレイヤーの移動回数」を直接扱える

小課題5: $K = 2$

【他のプレイヤーの移動回数について】

2 回目以降のターンの過ごし方にはどのようなものがあるか？

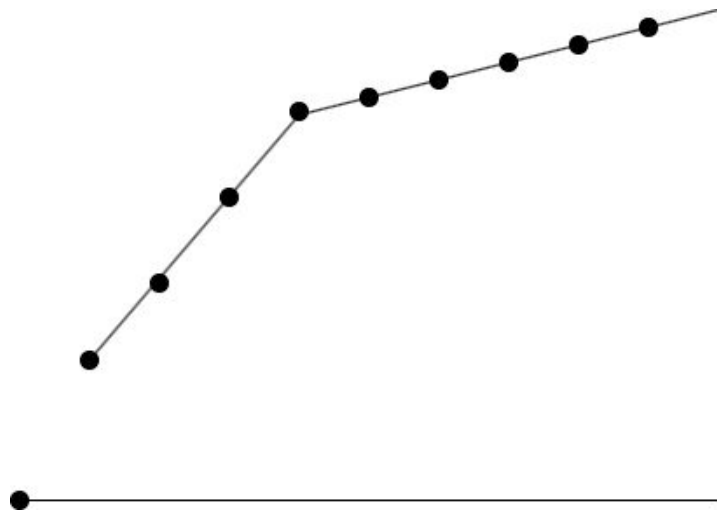
- 1 手かけてターンを終了する
- 2 手かけてターンを終了する
- 将来 1 手かけてターンを終了できるようになることを目指して移動

小課題5: $K = 2$

【他のプレイヤーの移動回数について】

だいたいこんな形の関数になる

- 0 回ときは 0
- 途中まで傾き 2
- 途中から傾き 1



小課題5: $K = 2$

【他のプレイヤーの移動回数について】

- 途中まで傾き 2 → 最短路問題(簡単)
- 途中から傾き 1

→ $dp[v] :=$ 頂点 v からスタートしてどこかでコストが $f(n)=n+c$ にできるような c の最小値

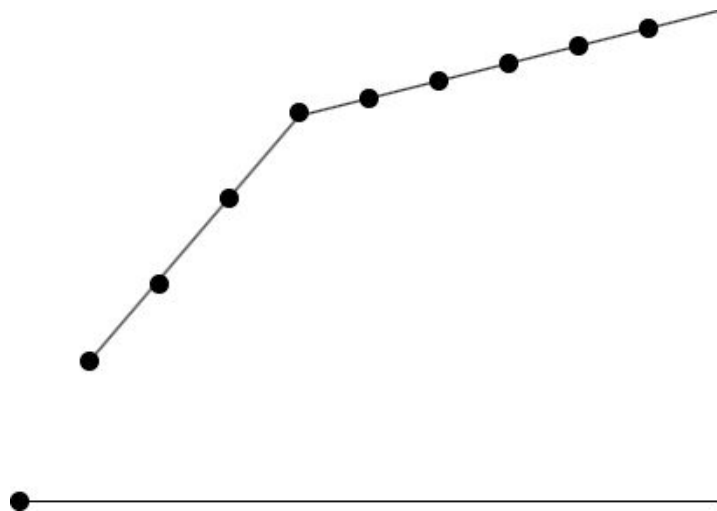
→ これも最短路問題みたいに計算できる

小課題5: $K = 2$

【他のプレイヤーの移動回数について】

左の方と右の方で挙動が違う

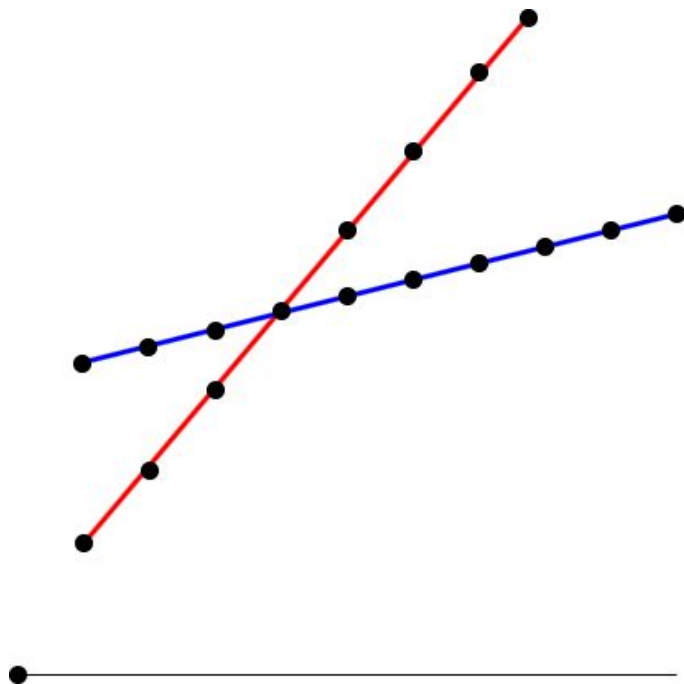
→ 結局プレイヤー 1 の停止回数を
正確に把握する必要がないか？



小課題5: $K = 2$

【他のプレイヤーの移動回数について】

- 最小移動回数は 2 つの min
- min じゃない方も好きなように選べることにしてしまっって解けばよい



小課題5: $K = 2$

【解法】

- プレイヤ 2 の最小移動回数の関数を表す 2 つの 1 次関数を求める.
- それぞれの 1 次関数を使うと決めてしまつて 2 回解く.
- 両プレイヤーの移動回数の総和が, $2N$ 状態の最短路問題で解けるようになる.

小課題5: $K = 2$

【解法】

1 次関数 $an + b$ を使うとき, v から w への移動について,

- 間にプレイヤー 2 のターンがはさまるとき: コスト $1 + a$
- プレイヤ 1 のターンが継続するとき: コスト 1

として最短路を計算すればよい.

($S[v]$ が 1 かつ v が $X[0]$ ではないときコスト a 発生)

小課題5: $K = 2$

【計算量】

- Dijkstra $\rightarrow O((N+M)\log N)$
- コストが 2 種類しかないことを使って $O(N+M)$ 時間で解くこともできる
 - 最後の移動コストが c_1 のものを入れておく que_1
 - 最後の移動コストが c_2 のものを入れておく que_2
 - 2 つの que の先頭のうち小さいものを取り出す

小課題5: $K = 2$

【実装注意点】

- 停止 0 回とその他は区別する必要があります.
- 初期位置が停止マスの隣であるとき, 次のような関数になります.

停止回数	0	1	2	3	4
移動回数	0	1	3	5	7

小課題6 : $K \leq 100$ (JOI 9pts, JOIG 7pts)

小課題6: $K \leq 100$

小課題 5 の解法がそのまま $K \leq 100$ に一般化可能です.

プレイヤー 2, 3, 4, ... の移動回数の総和は,

- 各プレイヤーは傾き 1, 2 の 1 次関数を持っている.
- 各プレイヤーが使う 1 次関数をひとつずつ選んで足す.

として形で考えることができます.

小課題6: $K \leq 100$

プレイヤー 1 以外の戦略は、傾き $K-1, K, \dots, 2(K-1)$ の 1 次関数によって表すことができます。

これらの 1 次関数に対して小課題5と同様にして答を計算できます。

$O(K(N+M))$ 時間で解けます。

小課題7 : $N, M, K \leq 30000$ (JOI 23pts, JOIG 12pts)

小課題8 : $N, M, K \leq 50000$ (JOI 9pts, JOIG 4pts)

最後の 2 つの小課題は制約が類似しており、必要な考察に大きな違いはありません。

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

- 小課題4での解法
- プレイヤ 1 について計算したいもの
 - 停止マスから n 回踏んでマス T に到達するときの移動回数

→ 計算したいものが N^2 通りもあるので無理

n の範囲を絞り込んだりするのも難しい (n が巨大な解と n が小さな解の比較も明らかではない)

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

- 小課題4での解法
- プレイヤ 1 について計算したいもの
 - 停止マスで n 回踏んでマス T に到達するときの移動回数

→ 計算したいものが N^2 通りもあるので無理

n の範囲を絞り込んだりするの難しい (n が巨大な解と n が小さな解の比較も明らかではない)

→ **しかし K が大きければ n の範囲を絞り込める**

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

- 停止マス 1 回あたり, 最低でもコスト $K-1$ がかかる.
- 停止マスを n 回余分に使った場合, プレイヤ 1 の移動回数で $n(K-1)$ 回得しないと損失回復できない.
- 停止マスの回数は最小回数から見て $+N/(K-1)$ 以下であるときに限定できる.

$O(N^2/K)$ 状態の最短路問題 $\rightarrow K$ が大きければ高速.

小課題7 : $N, M, K \leq 30000$

小課題8 : $N, M, K \leq 50000$

【解法まとめ】

- それぞれの他プレイヤーのコスト関数を求める
- それらを足し合わせたコスト関数を求める(累積和などを使う)
- K が小さい(or コスト関数の傾きの種類数が少ない) 場合
→ 小課題6の解法を使う
- K が大きい(or コスト関数の傾きの種類数が多い)
→ 停止マスの回数も状態とした最短路問題
→ 適切に枝狩りをすると $O(N^2/K)$ 状態
- 適切に使い分けると, $O((N+M)^{1.5})$ 時間で解ける.

小課題7 : $N, M, K \leq 30000$

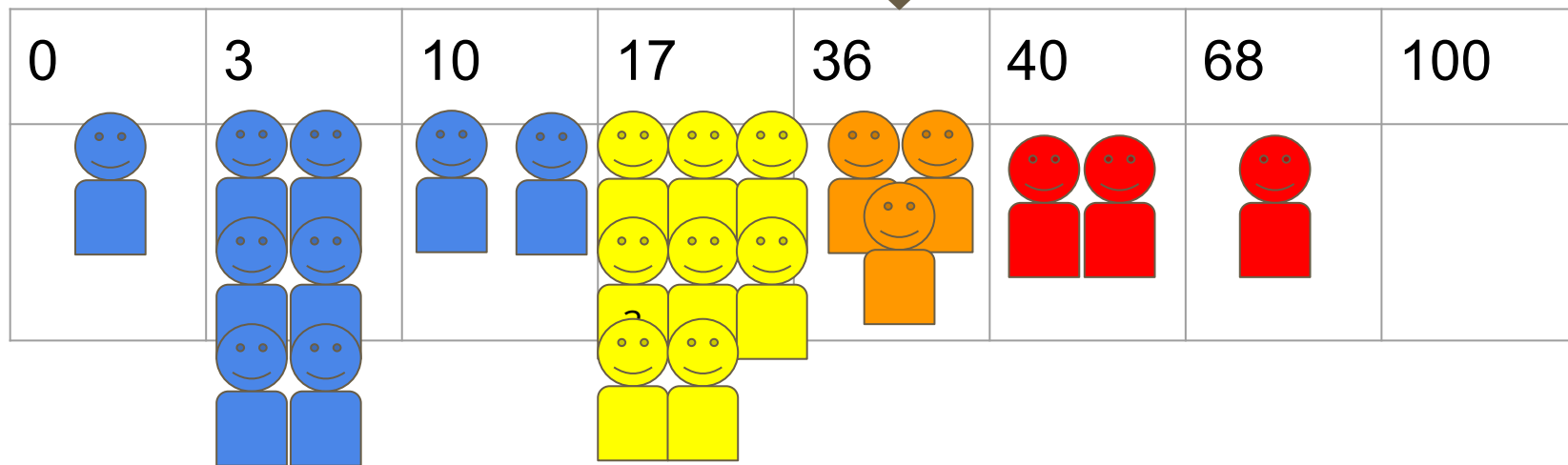
小課題8 : $N, M, K \leq 50000$

ある程度高速な実装をすれば小課題8まで通ります. TLE してしまった場合には以下のような高速化手法をいくつか試すとよいでしょう.

- 2 種の解法のしきい値を調整
- Dijkstra 法を避けて線形時間の最短路アルゴリズムを使う
- 枝狩りの計算式をより丁寧にやる
- 最後の解法を $O(N)$ メモリで実装する
- ...

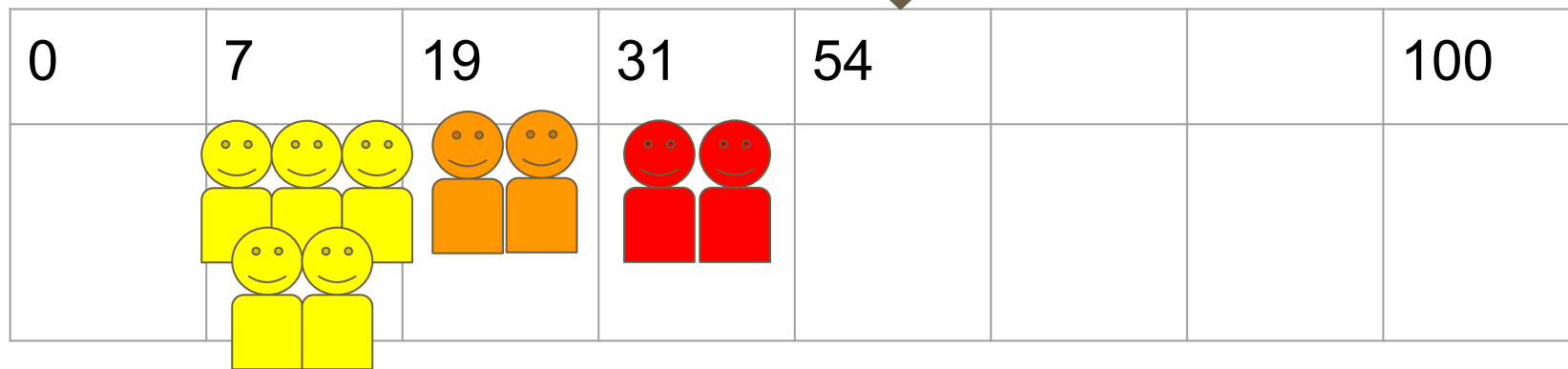
得点分布(JOI, 23人)

小課題1～4



得点分布 (JOIG, 9人)

小課題1 ~ 4



Tricolor Lights 解説

ynymxiaolongbao

問題概要

- AnnaとBrunoの二人が協力して、ディ太郎に勝ちたい
- 長さ N のボードがあり、Annaは各場所をR, G, B のいずれかの色で塗る
- 各場所には一色だけ禁止色 s_i があり、その色では塗れない

R G G B R B B G

問題概要

- Annaが整数 L を指定する
- ディ太郎がボードの長さ L の区間を切り取り、その色の並びをBrunoに伝える
- Brunoはディ太郎が切り取った場所がどこかを答える

RGGBRBBG

→答え3

←→
長さ5

問題概要

- Lは大きくした方が簡単、Nは小さい方が簡単
- 小課題 4 まで：Lの上限は130固定で、Nが段々大きくなる
- 小課題 5 ：Nが大きい値で固定で、Lの上限が段々小さくなる

R G G B R B B G

→ 答え 3

← 長さ 5 →

小課題 1 ($N \leq 131 = L + 1$)

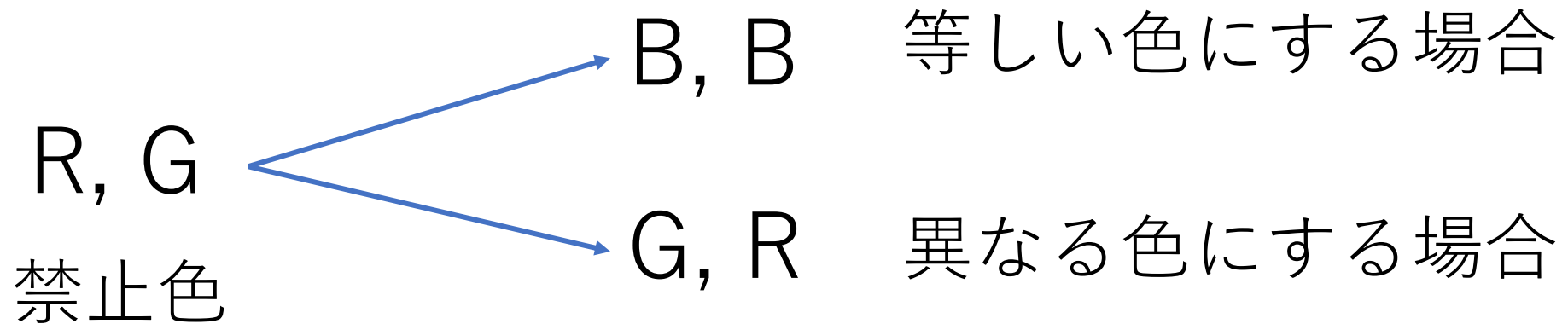
- $N \leq 130$ のとき、 $L = N$ に設定すれば答えはいつも1
- $N = 131$ のとき、 $L = 130$ にすると、1~130なのか2~131なのか判定できれば良い
- 思ったより難しい

小課題 1 ($N \leq 131 = L + 1$)

- 気持ち：一つの桁ではちゃんと情報を伝えられない
- たとえば、「この桁がRだったらこういう意味」と決めておいたとしても、その桁をRにすることが禁止されてると、そうできない
- →二つの桁を使って情報を伝える

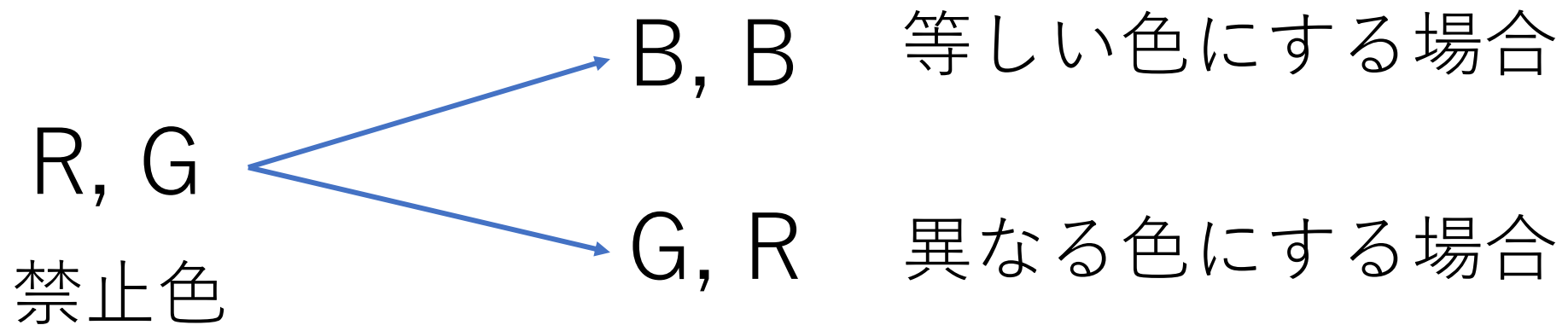
小課題 1 ($N \leq 131 = L + 1$)

- 二つの桁を使って情報を伝える
- 二つの桁が等しいか／異なるか を自由に決められる



小課題 1 ($N \leq 131 = L + 1$)

- 等しくする場合：禁止されている色はR,G,Bのうち高々二種類
→残りの色を選んで両方それで塗る
- 異なるようにする場合：塗り方は $2 \times 2 = 4$ 通りあるが、等しい色にする方法は3通りしかないなので、少なくとも1通りでは異なる色になる（鳩ノ巣原理）



小課題 1 ($N \leq 131 = L + 1$)

- Anna (色を塗る人) は、1桁目と130桁目を同じ色に塗って、2桁目と131桁目を異なる色に塗る
- Bruno (場所を当てる人) は、受け取った色の並びの左端と右端が同じ色かを見て、同じなら1、異なるなら2と答えれば良い

小課題 2 ($N \leq 250 \sim 2L$)

- 隣り合う二つが等しいか異なるかに注目
- 値が同じことを0、異なることをXと書く
- Anna（色を塗る人）：隣あう二つの違いが図のように並ぶようにする

$X \cdots X \ 0 \ X \cdots X$



128 128

小課題 2 ($N \leq 250 \sim 2L$)

- まず0になる（色が等しい）二か所の色を決める
- Xになる（色が異なる）場所は、0に近い方から順に、三色のうち隣の色・禁止色のどちらでもない色を選んで行く



小課題 2 ($N \leq 250 \sim 2L$)

- Bruno (場所を当てる人) : 場所が $1, 2, \dots, 129$ のとき、0がある位置は $129, 128, \dots, 1$ になっているので、 $130 -$ 「0がある位置」で場所が分かる

$X \cdots X \ 0 \ X \cdots X$



128 128

小課題 3 ($N \leq 380 \sim 3L$)

- Anna : 隣あう二つの違いが図のように並ぶようにする
- ?はAnna制御できずよく分からなくなった値とする
- ?は0かXのどちらかとしてBrunoに伝えられる

$X \cdots X \ ? \ 0 \ X \cdots X \ ? \ 0 \ X \ ? \ 0 \ X \cdots X$

126 122 125

小課題 3 ($N \leq 380 \sim 3L$)

- Bruno : 0が二回以上出てくるので、その距離を見てどこの0か判定
- 値が0のとき、Annaが0と?のどちらで送ったか分からない
- 本当の0の次は必ずXで、?の次は必ず0なので、分かる

X.....X ? 0 X.....X ? 0 X ? 0 X.....X



126 122 125

小課題 4 ($N \leq 7000 \sim L^2/2$)

- 0の間の距離が $L-7, 3, L-8, 4, L-9, \dots, L/2$ になるようにする
- どの長さ L の区間にも0が2つ以上出てくるので、その距離を見るとどここの0か分かる

...X.....X ? 0 X.....X ? 0 X...X ? 0 X.....X...



The diagram shows a sequence of characters: "...X.....X ? 0 X.....X ? 0 X...X ? 0 X.....X...". Below this sequence, there are four orange curved brackets, each spanning a segment between two '0's. Under each bracket is a number in orange: 20, 101, 21, and 100. These numbers represent the distances between the 0's in the sequence.

20 101 21 100

小課題 5 ($N \leq 5 \times 10^5, L = 28 \sim 130$)

- $L = \log N \times \text{定数}$ くらいにしないといけない
- 2進数とか3進数を使う
- 大ブロックに分ける解法
 $L = 130 \rightarrow 60 \rightarrow 40 \rightarrow 33 \rightarrow 31 / 32$ (天才)
- 小ブロックに分ける解法 (de Bruijn 列)
 $L = 51(\text{乱択}) \rightarrow 34 \rightarrow 31 \rightarrow 29 \rightarrow 28$

大ブロックに分ける解法

$L = 130 \rightarrow 60 \rightarrow 40 \rightarrow 33 \rightarrow 31 / 32$ (天才)

$L = 130$ (40点)

- * を、Annaが自由に決めた0かXの値とする
- 図の長さ59のブロックを繰り返す
- **...* の部分でブロックの番号を2進数14桁で書く

? *..... ? * ? 0 X.....X

14×2

29

$L = 130$ (40点)

- 0の後にXが29個並ぶのはブロックの右端の部分だけ
→どこからブロックが始まるか分かる
→そこに書いてある2進数を見て、そのブロックの番号を求める
→どこを切り出したのか分かる

? * ? * ? 0 X X

14×2

29

$L = 60$ (50点)

- ブロックが完全に含まれている必要はない
- 困るのは、ブロックが途中で切れて、図のようになるとき
- 二進数が完全には読めない

? *... ? * ? 0 X.....X ? *... ? *

$L = 60$ (50点)

- ブロックの番号 x の下 k 桁と $x + 1$ の上 $14 - k$ 桁が分かる
- x の下 k 桁が $1 \cdots 1$ なら、 x の上 $14 - k$ 桁は、 $x + 1$ の上 $14 - k$ 桁 -1
- そうでない場合、 x の上 $14 - k$ 桁は $x + 1$ の上 $14 - k$ 桁と同じ
→ブロックの番号が分かる

? * ... ? * ? 0 X X ? * ... ? *

$L = 40$ (58点)

- 2進数ではなく 3 進数が使える
- R, G, Bを0, 1, 2とみなして、その差 (mod 3) を自由に決められる

? * ? * ? 0 X X

9×2

19

$L = 40$ (58点)

- * を0,1,2のAnnaが自由に決められる値とする
- 図の長さ39のブロックを繰り返す
- **...* の部分でブロックの番号を3進数9桁で書く

? *..... ? * ? 0 X.....X

9×2

19

$L = 33$ (80点)

- 前の解法は、右端以外の所で1が沢山連続しなければ良い
- $** \dots *$ の部分でブロックの番号を3進数9桁で書く

? * ? * ? 0 X X



9×2



19

$$L = 33 \quad (80\text{点})$$

- 図のように、間に3つごとに0を挟めばよい（9桁なので、ちょうど3つずつに分かれる）
- 右端で必要な $X \cdots X$ の長さが減る

$? * \cdots ? 0 ? * \cdots ? 0 ? * \cdots ? 0 X \cdots \cdots X$

$3 \times 2 \quad 3 \times 2 \quad 3 \times 2 \quad 8$

$$L = 31 \quad (88\text{点})$$

- 前の解法では、3進数9桁の間に0を2つ挟んで、*の部分でXが4連続しないようにした
- 3進数10桁であってXが4連続しないものを選ぶと、減る

$L = 32$ (天才) (84点)

- 3進数が書けるが、あえて1と2だけを使って2進数14桁を書く
- * を、Annaが自由に決められる1か2の値とする
- 右端が工夫されている

? * ? * ? X 0



14 × 2

$L = 32$ (天才) (84点)

- 0になりうるのは 0 と ? の場所
- 0が一つしかない場合：そこが右端の0
- 0が二つ以上ある場合：右端の0だけ、次の0との距離が奇数
→ 右端の0の位置が分かる

? * ? * ? X 0



14 × 2

小ブロックに分ける解法 (de Bruijn 列)

$L = 34 \rightarrow 31 \rightarrow 29 \rightarrow 28$

de Bruijn 列

- k 進数 N 桁のde Bruijn 列… k 進数 n 桁の文字列は k^N 通りあるが、それらがちょうど一回ずつ長さ N の文字列として現れるような列
- これは巡回させることができる
- 以下のグラフの一筆書きに対応する
- 頂点： k 進数 $N-1$ 桁の各文字列
- 辺： k 進数 N 桁の各文字列に対応し、その前 $N-1$ 桁から後ろ $N-1$ 桁の頂点へ有向辺

de Bruijn 列

- 有向グラフが一筆書きできる条件…連結&すべての頂点で入次数と出次数が等しい
- 前頁のグラフはその条件を満たす
→de Bruijn 列は存在

de Bruijn 列の拡張

- 逆に、グラフがその条件を保っていれば、de Bruijn 列のようなものが作れる
- k 進数 N 桁の文字列のうち、いくつかを取り除いても、それらがちょうど一回ずつ出てくるような列が作れることがある
- 例えば、「3進数11桁の文字列のうち、0が一回以上出てくるもの」についても、それらがちょうど一回ずつ出てくるような列が作れる（グラフが、連結で各頂点の入次数と出次数が等しいため）
- これを拡張 de Bruijn 列と呼ぶことにする

$L = 34$ (76点)

- ? * X を1ブロックとしてこれを繰り返す
- * の部分には、「3進数11桁のうち一回以上0が出てくるもの」
からなる拡張de Bruijn 列を書く

? * X

? * X

⋮

? * X

$L = 34$ (76点)

- 11ブロック分見て、mod 3ごとに、一回以上0が出てくるか見る
- 結果がxoxのとき、* になっている余りはoの所
- 結果がooxのとき、* になっている余りは右側のoの所
- Brunoは拡張de Bruijn列から整数を復元すれば良い

? * X

? * X

:

? * X

$L = 31$ (88点)

- $? * ? * X$ を1ブロックとしてこれを繰り返す
 - $*$ の部分二つには、
 - 「3進数6桁のうち一回以上0が出てくるもの (665個)」
 - 「そこから0...0を除いたもの (664個)」
 - かなる拡張de Bruijn 列をそれぞれ繰り返し書く
 - 6ブロック分見て、mod 5ごとに、一回以上0が出てくるか見る
 - 結果がNoになる余りは?とXの所だけ
- 前の $L=32$ 解法と同様に、次の場所との距離が奇数な所がX

$$L = 31 \quad (88\text{点})$$

- 周期が665, 664 のde Bruijn 列を合わせると、周期は $\text{LCM}(665, 664)$ になる
- Brunoは得た二つの整数から中国剰余定理を使って場所を復元
- $5 \times \text{LCM}(665, 664) \geq 500000$ で、通る

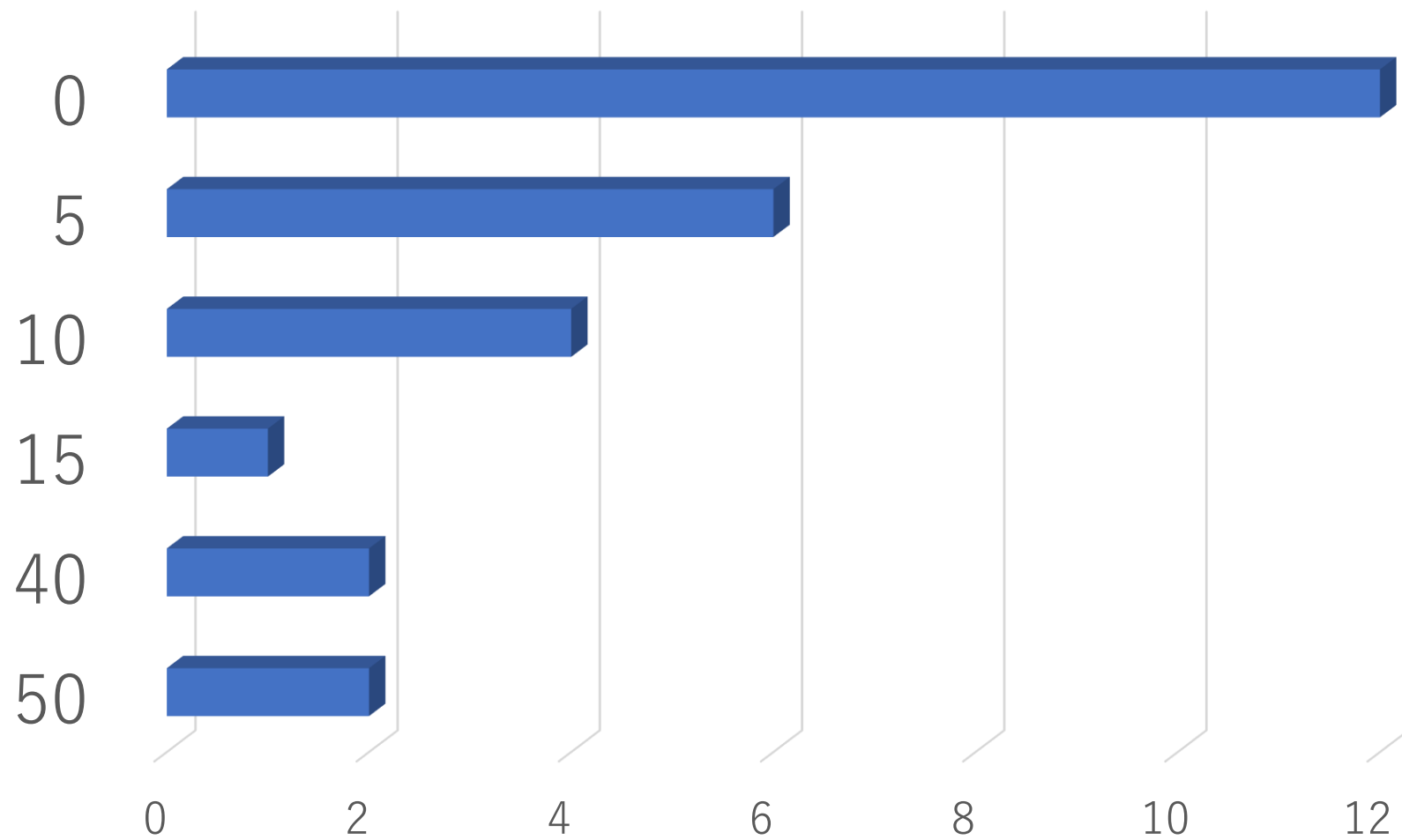
$L = 29$ (96点)

- $? * ? * ? * X$ を1ブロックとしてこれを繰り返す
- $*$ の部分三つには、
- 「3進数4桁のうち一回以上0が出てくるもの (65個)」
- 「そこから0000を除いたもの (64個)」
- 「そこから0101と1010を除いたもの (63個)」
- かなる拡張de Bruijn 列をそれぞれ繰り返し書く
- 余りの特定、復元 (中国剰余定理) も先ほどと同様
- $7 \times \text{LCM}(65, 64, 63) \geq 500000$ で、通る

$L = 28$ (100点)

- $? * ? * ? * ? * X$ を1ブロックとしてこれを繰り返す
- $*$ の部分四つには、
- 「3進数3桁のうち一回以上0が出てくるもの (19個) 」
- 「010,101を除いたもの (17個) 」
- 「010,101,000を除いたもの (16個) 」
- 「010,101,020,202を除いたもの (15個) 」
- かなる拡張de Bruijn 列をそれぞれ繰り返し書く
- $9 \times \text{LCM}(19, 17, 16, 15) \geq 500000$ で、通る

得点分布 (JOI)



得点分布 (JOIG)

10点 2人

0点 7人

カードゲーム 3

JOIG 2023/2024 春季トレーニング 競技2

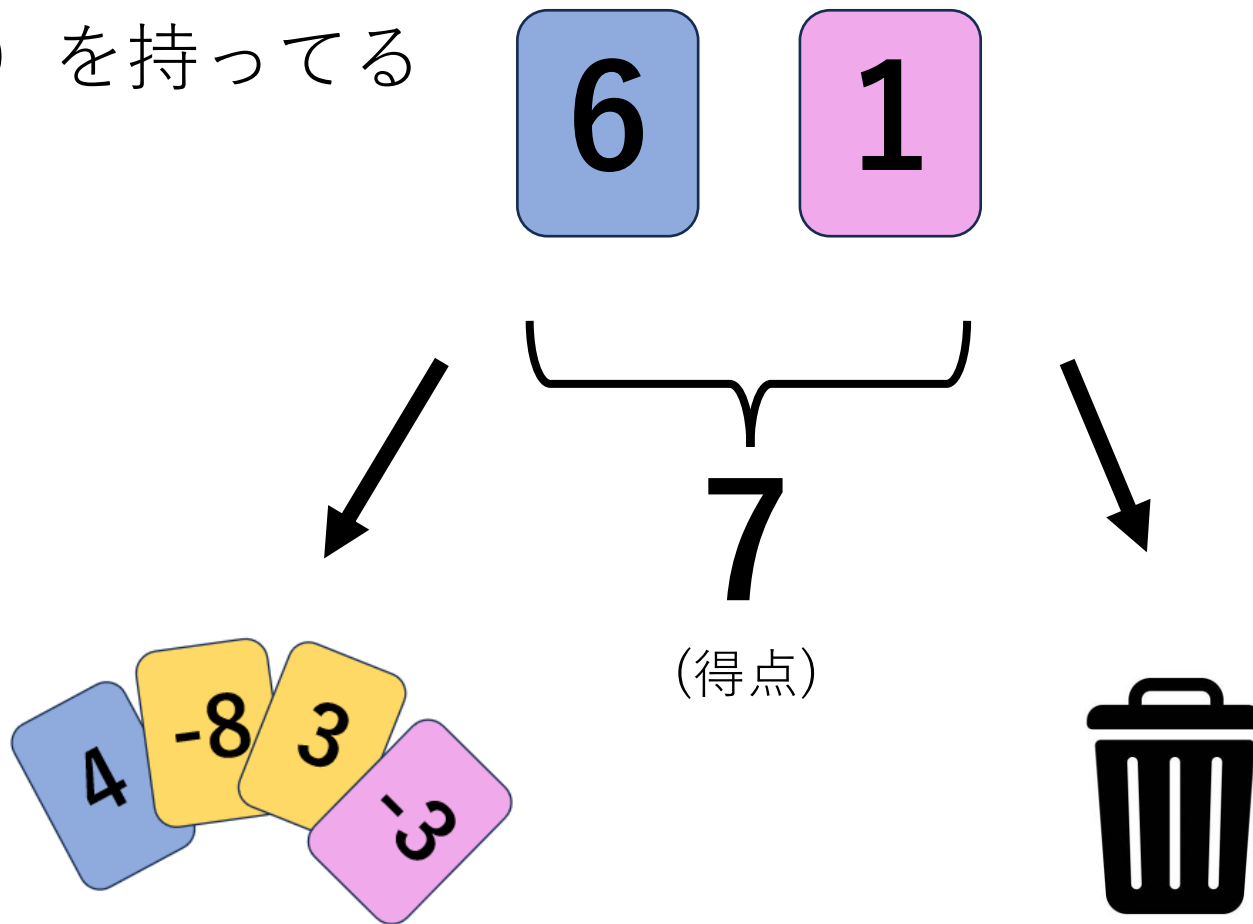
解説：大野 栞

問題概要

N 枚のカード（色、数字付き）を持ってる

操作

色違いを 2 枚選ぶ
2 枚の合計点を得る
片方→捨てる
片方→戻す



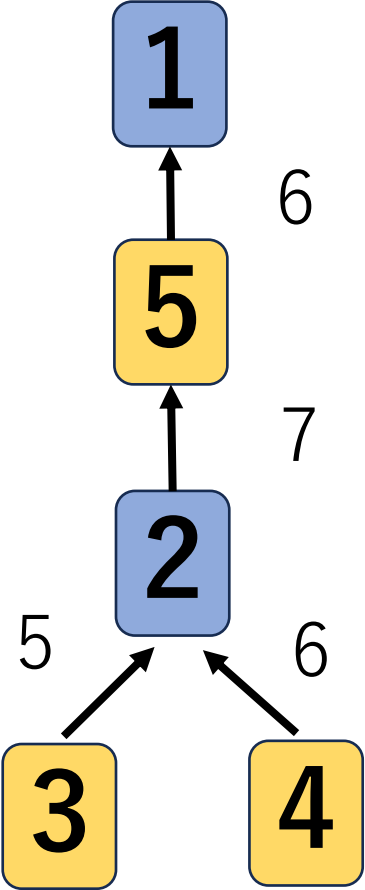
問題概要

制約：

- $2 \leq N \leq 50,000$
- $-10^9 \leq A_i \leq 10^9$
- $1 \leq C_i \leq N$

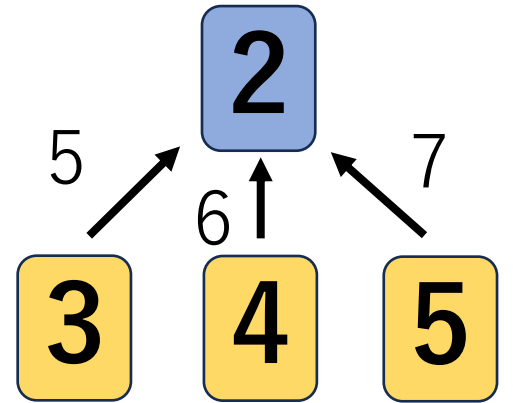
木で表される！！

操作	1	2	3	4
選んで捨てるカード	3	4	2	5
選んで捨てないカード	2	2	5	1
合計得点	5	6	7	6



小課題 1 $C_1 = 1, C_i = 2 (2 \leq i \leq N)$

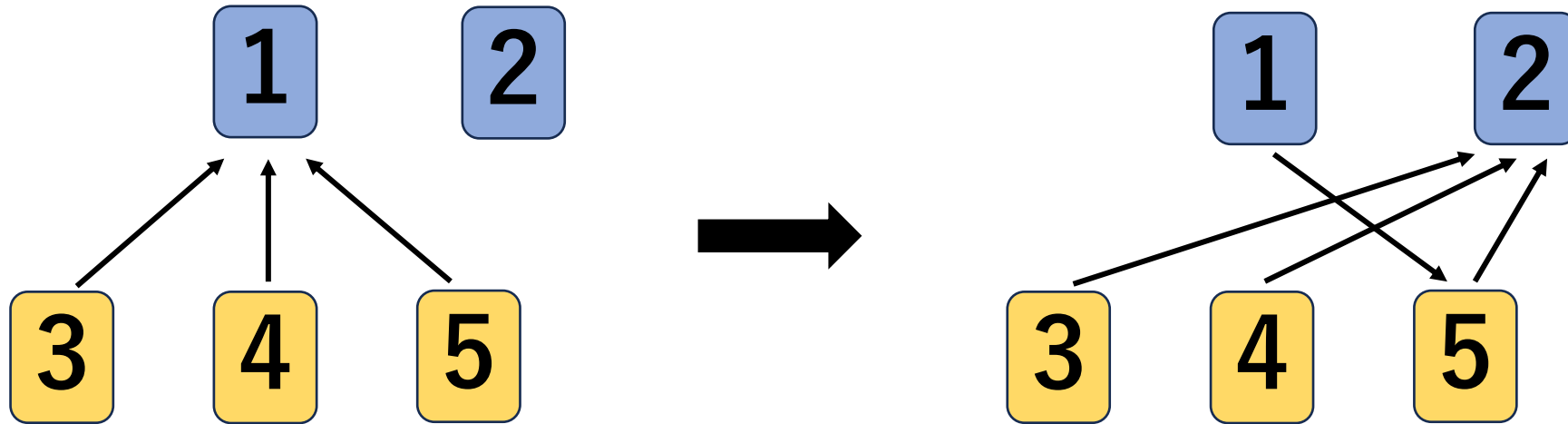
- カード 1 とカード i ($2 \leq i$) を選ぶ選び方しかない
- 最後までカード 1 を残せば、最大 $N-1$ 回操作可能。
順番は任意



カード i ($2 \leq i$) について、順番に

「 $A_1 + A_i > 0$ ならカード 1 とカード i を選び、カード 1 を捨てる」

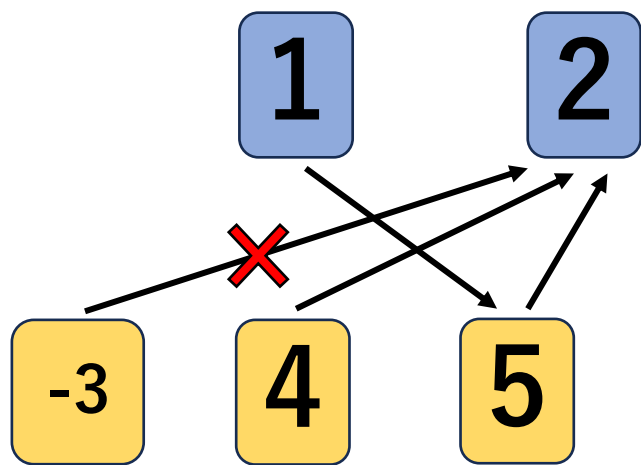
小課題2 $N \geq 3, C_1 = 1, C_2 = 1, C_i = 2 (3 \leq i \leq N), A_i \geq 0$



- カード 1 と 2 のうち A_i が小さい方を選び、カード j ($3 \leq j \leq N$) のうち A_j が最大のものも選んで、前者を捨てる
- カード 1 と 2 のうち A_i が大きい方を選び、カード j ($3 \leq j \leq N$) もそれぞれ選んで、後者を捨てていく

小課題3

$$N \geq 3, C_1 = 1, C_2 = 1, C_i = 2 \ (3 \leq i \leq N)$$



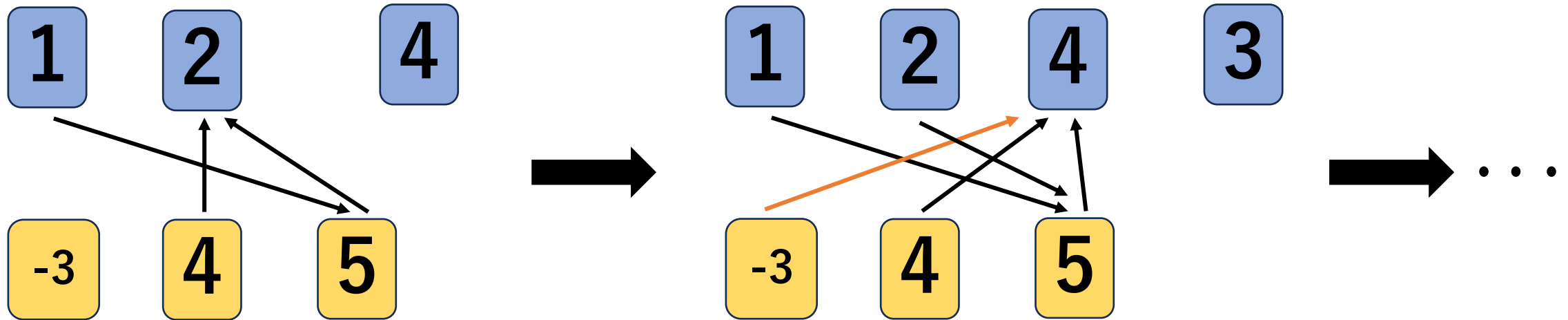
←任意のカード i を捨てるとき、 i と一緒に選べるカード j のうち A_j が最大のものを常に選んでいる

• $A_i + \max\{A_j \mid 3 \leq j \leq N, C_i \neq C_j\} \geq 0$
なら選んだ方が得

• $A_i + \max\{A_j \mid 3 \leq j \leq N, C_i \neq C_j\} < 0$
ならどのカード j ($3 \leq j \leq N, C_i \neq C_j$) を選んでも損

- カード 1 と 2 のうち A_i が小さい方を選び、カード j ($3 \leq j \leq N$) のうち A_j が最大のものを選び、後者を捨てる。ただし $A_i + A_j < 0$ なら操作しない
- カード 1 と 2 のうち A_i が大きい方を選び、カード j もそれぞれ選んで、後者を捨てていく。ただし $A_i + A_j < 0$ なら操作しない

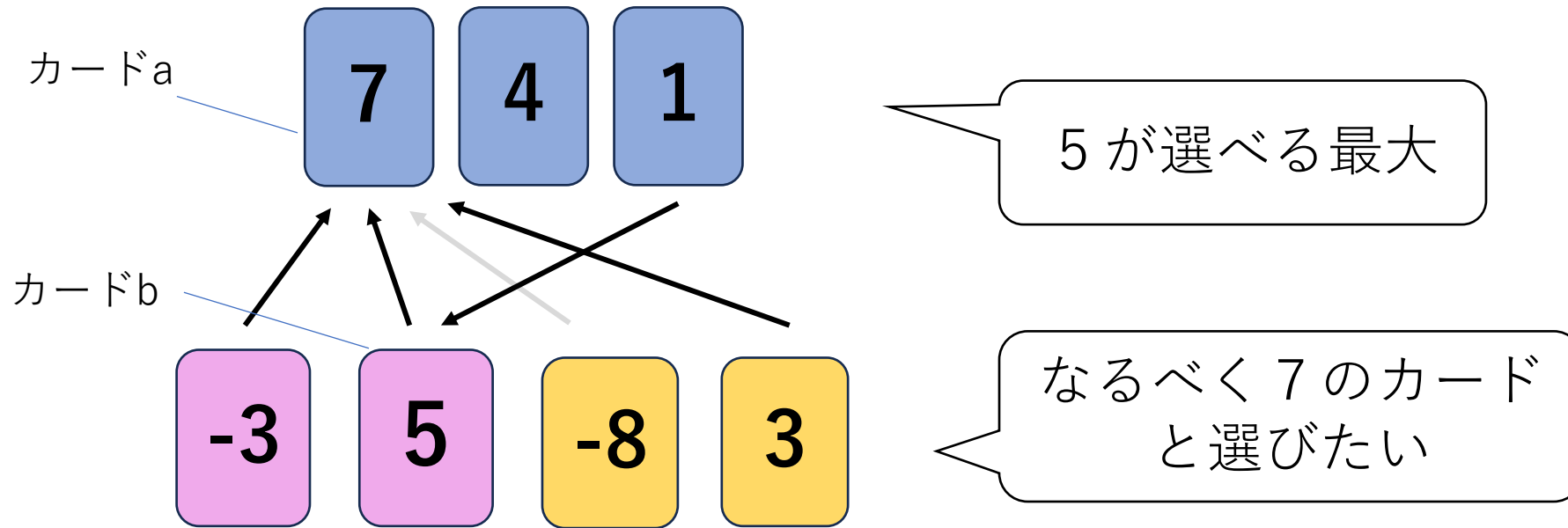
小課題4 $C_i \leq 2 \ (1 \leq i \leq N)$



- 色1のカードのうち A_i が最大でない i をそれぞれ選び、色2のカードのうち A_j が最大の j も選び、前者を捨てていく。ただし $A_i + A_j < 0$ なら操作しない
- 色1のカードのうち A_i が最大の i を選び、色2のカードのうち A_j が最大でない j もそれぞれ選び、後者を捨てていく。ただし $A_i + A_j < 0$ なら操作しない

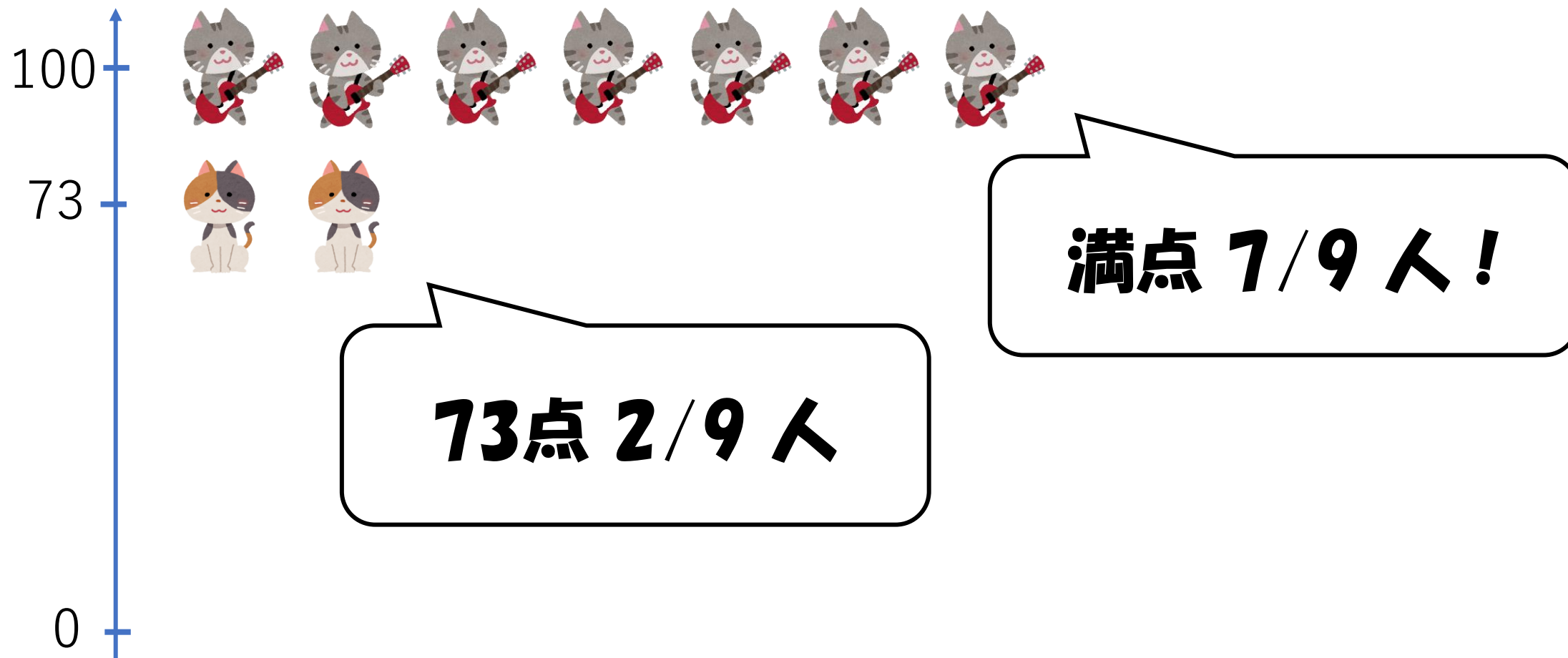
小課題5 追加の制約なし

方針はほぼそのまま！！



- 全カードのうち A_a が最大のカード a とし、 a とは色が異なるカードのうち A_b が最大のカード b とする。
- a と同じ色の a 以外のカード i をそれぞれ選び、 b を選び、前者を捨てていく。
ただし $A_i + A_b < 0$ なら操作しない
- b とは色が異なるカード j をそれぞれ選び、 a を選び、後者を捨てていく。
ただし $A_j + A_a < 0$ なら操作しない

得点分布



第4回 JOIG 春トレ Contest2

チョコレート

を増やせ解説



担当: Thistle(チューター)

目次



3-4 ページ

問題概要

10-11 ページ

小課題 3

5-7 ページ

基礎方針

12-16 ページ

満点解法

8 ページ

小課題 1

17 ページ

別実装

9 ページ

小課題 2

18 ページ

統計情報

問題概要



チケット $\times A_i \Rightarrow (\text{チョコレート} + \text{チケット}) \times B_i$

という交換規則が N 個ある。

最終的に K 個以上のチョコレートを手に入れるためには、

最初にチケットとチョコレートを最低何個ずつ購入する必要があるか？

$1 \leq K \leq M$ について求めよ。

問題概要



サンプル 1

規則 1 :  x 3 $\Rightarrow$ ( + ) x 1
規則 2 :  x 5 $\Rightarrow$ ( + ) x 2



基礎方針



考察



答えの代わりに、1 以上 M 以下の全ての x について

**x 個のチョコレートを買った時の
最終的に得られるチョコレートの最大数 y**

を求めることにする。

重要な考察 y 個未満を得る行動は考えないでよい。

⇒ 全ての x について $y = f(x)$ が一意に定まる。

基礎方針



考察



全ての x について $y = f(x)$ が一意に定まる。

⇒ **ちょうど** y 個のチョコレートを
得るときに必要な最小の x が求まる。

⇒ y 個**以上**のチョコレートを得るときに必要な
最小の x が求まる。これは求めたい値そのものの。

基礎方針

考察

買	得
1	1
2	3
3	3
4	5
5	6



丁度

得	答
1	1
2	-
3	2
4	-
5	4



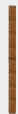
以上

得	答
1	1
2	2
3	2
4	4
5	4

小課題 1



制約



$M \leq 500, N=1$

解法



全ての購入数 x について、使えるだけ規則を使っていけば答えが得られる。

計算量は $O(M*M)$

小課題 2



制約

$M \leq 500, A_i - B_i = \text{const.}$

解法



一度規則を使った後チケットが減る枚数 $A_i - B_i$ が同じ
⇒ できるだけ **多いチョコ**を得られる規則を使うほうが得
⇒ 使えるチケット（現在のチケット枚数 $\geq A_i$ であるもの）
の中で最大の B_i を持つ規則を使う**貪欲法**を行う。
計算量は $O(M * M * N)$

小課題 3



制約

$M \leq 500, N \leq 500$

考察



小課題 1,2 みたいに貪欲法をしたいが、どうにも動かなそう。
こういった場合は、DPや賢い探索など他の解法を探さないといけない。
今回は「使用枚数が x 枚を超えないように $A_i - B_i$ 枚のチケットを
使っていき、 B_i を最大にする」という考察から、
ナップサック問題に見立てて解く。

小課題 3



解法



全ての x について、変則的なナップサック問題

容量 x のかばんに入る荷物の価値の合計の最大値は何か？

荷物 i の重さ = $A_i - B_i$, 荷物 i の価値 = B_i

ただし 同じ荷物は何度でも使える。

現在の重さ $+A_i \leq x$ である荷物しか使えない。

を解く。(価値の最大値 $+x$) が y になり、 $O(M^2N)$

満点解法



制約

$M \leq 100,000, N \leq 500$

考察



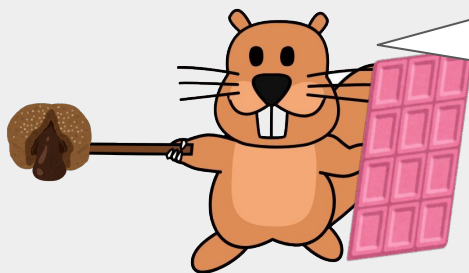
小課題 3 の解法は全ての M について行くと到底間に合わない。全ての M について同時にDPを行うことを考える。

とはいえどうやってDPすればよいのか？

満点解法

考察

DPには全探索の高速化という側面もあるので、全探索コードを書いてみると考察の参考になるかもしれません。
書いてみましょう。



再帰関数で書きたいビタ！

引数はチケットとチョコの数ビタ？

満点解法



考察



再帰関数の引数は、チケットとチョコレートの数？

再帰関数の戻り値は、

今の引数で表される状態から始めて最後まで進めたときの結果
(今回はチョコレートの数の最大値)

⇒ 実は、この値を求めるのに現在のチョコレートの数は必要ない！

⇒ 引数はチケットだけで十分！

満点解法



考察



実際に再帰関数として実装してみると、以下のようになる。

```
int rec(int x) {  
    int mx = 0;  
    rep(i, n) if (x >= a[i].a) {  
        chmax(mx, rec(x - a[i].a + a[i].b) + a[i].b);  
    }  
    return mx;  
}
```

チケットが x 枚の時の最大値 =
(チケットが $x - a_i + b_i$ 枚の時の最大値 $+ b_i$) の最大値

満点解法

解法

recは引数と同じなら同じ値を返すので、同じ引数のrecを2度呼び出さないようにしてみましょう（メモ化再帰）。

```
int dp[100005];
#define UNUSED -1
int rec(int x) {
    if (dp[x] != UNUSED) return dp[x];
    int mx = 0;
    rep(i, n) if (x >= a[i].a) {
        chmax(mx, rec(x - a[i].a + a[i].b) + a[i].b);
        if (mx > m) mx = m;
    }
    return dp[x] = mx;
}
```

状態数 M個

各rec $O(N)$

計 $O(NM)$ **AC!!**

別実装



解法



$dp[x]$ は x が小さい順に更新されていくので、

$dp[x]$ = x 枚のチケットがある状態から得られる最大数として

$$dp[x] = \max(dp[x - A_i + B_i] + B_i, dp[x])$$

としていく。

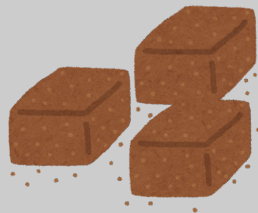
$dp[x] + x \geq y$ となる最小の x が $ans[y]$ となる。

としても正しく更新できる。計算量 $O(N * M)$ となり、AC!!

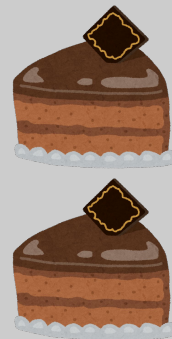
統計情報



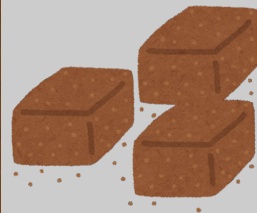
15点



30点



60点



100点

An underwater photograph showing a large, dense school of small, silvery fish swimming in clear blue water. A larger, dark-colored stingray is visible in the lower-left foreground, swimming towards the left. The background is filled with the movement of the fish, creating a sense of depth and activity.

Islands

Editorial : blackyuki



問題概要

問題概要

- N 頂点の木がある
- 木の情報は与えられていない
- 木の形を求めたい
- 「頂点 i から k 番目に近い頂点の番号は何か？」を質問できる
 - • • 同じ距離の頂点は番号の大小で決まる

問題の性質

インタラ
クティブ

木

インタラクティブにおける典型考察

- 色んな解法を試して上手くいくものを選ぶ
- 一つの解法に固執するのはもったいない
- 普通のBatch課題における典型アルゴリズムのシミュレーション
 - • • BFS, DFS, Dijkstra, など

木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

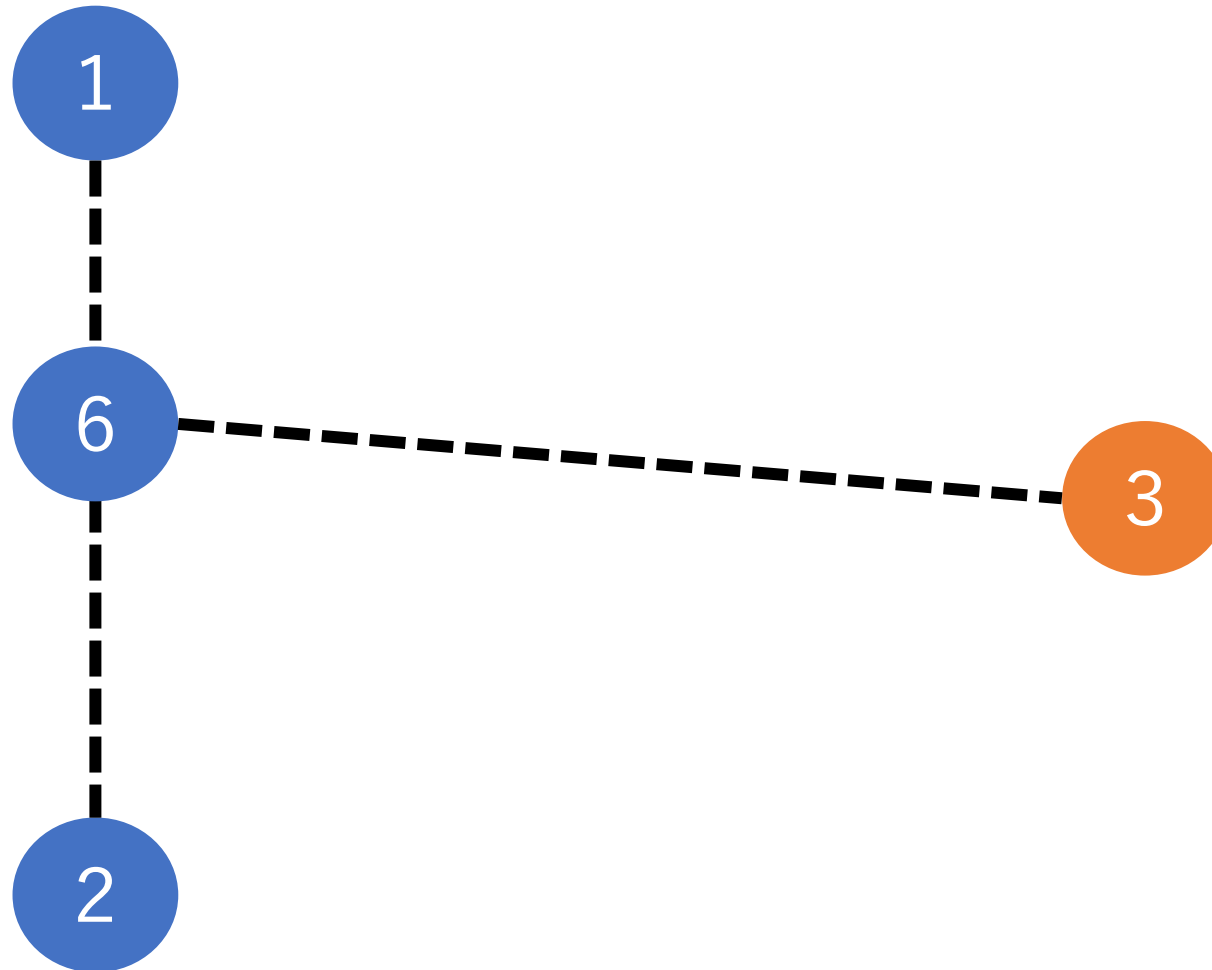
A. 頂点 $1, 2, \dots, N$ の順に追加していく

1

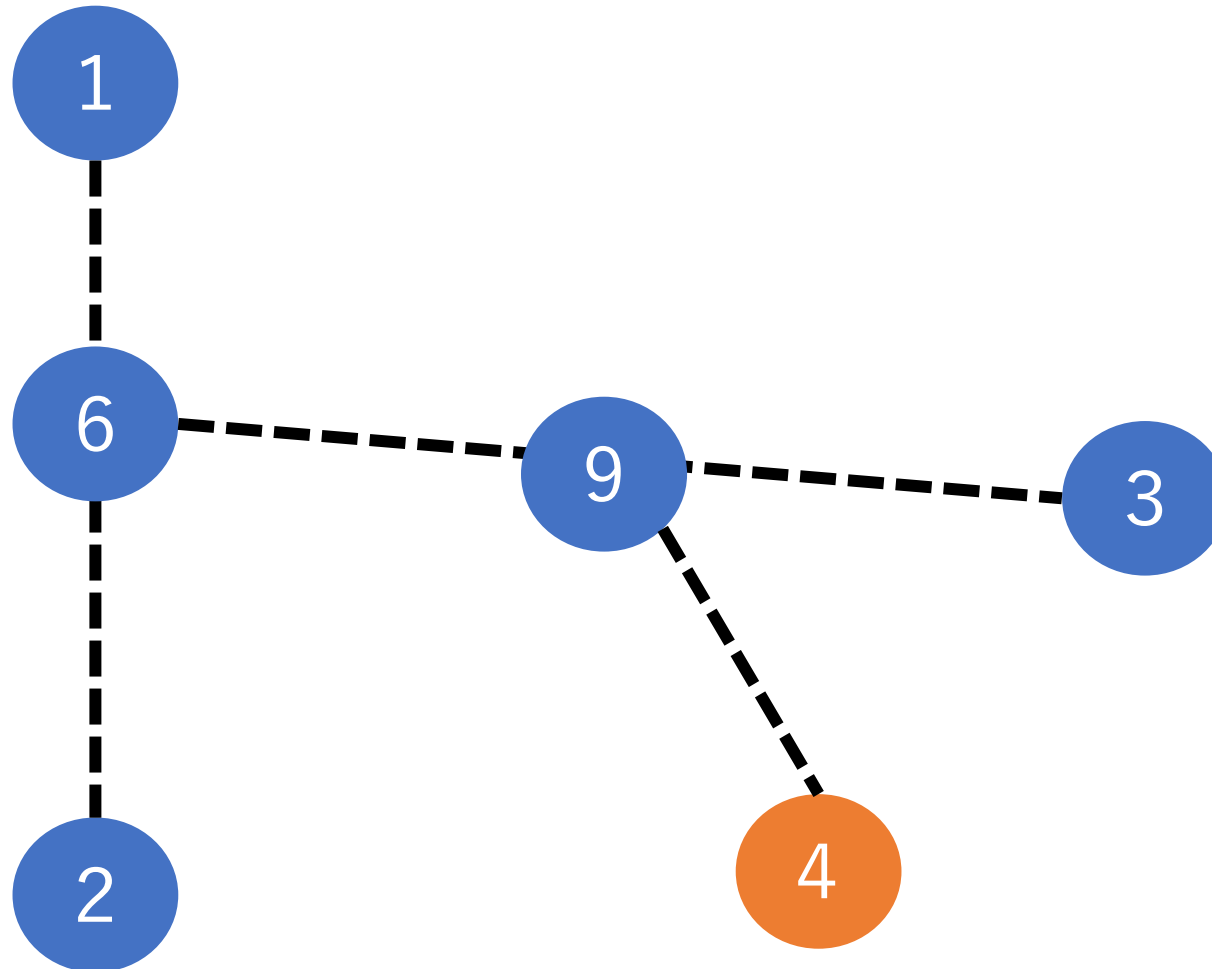
A. 頂点 $1, 2, \dots, N$ の順に追加していく



A. 頂点 $1, 2, \dots, N$ の順に追加していく



A. 頂点 $1, 2, \dots, N$ の順に追加していく



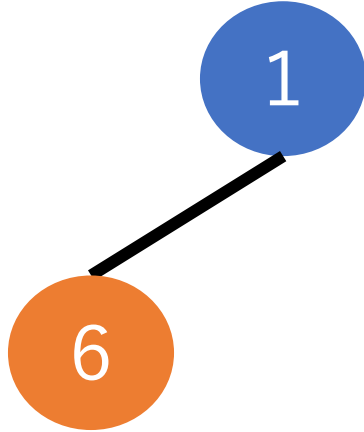
B. 頂点 1 から近い順に探索していく

BFS（幅優先探索）



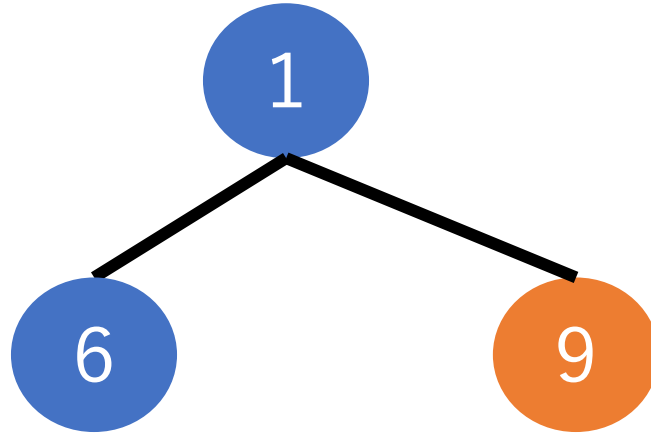
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



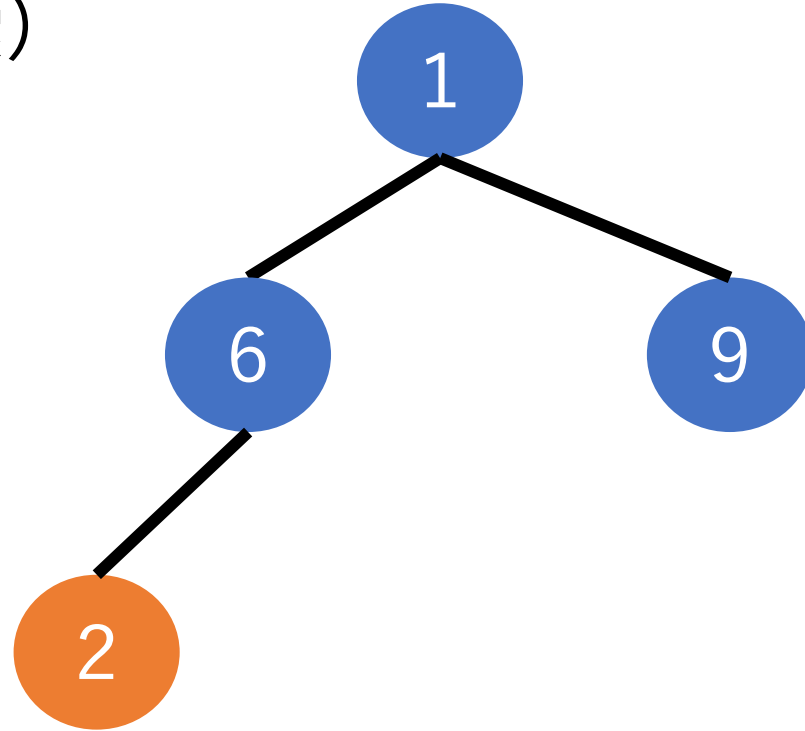
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



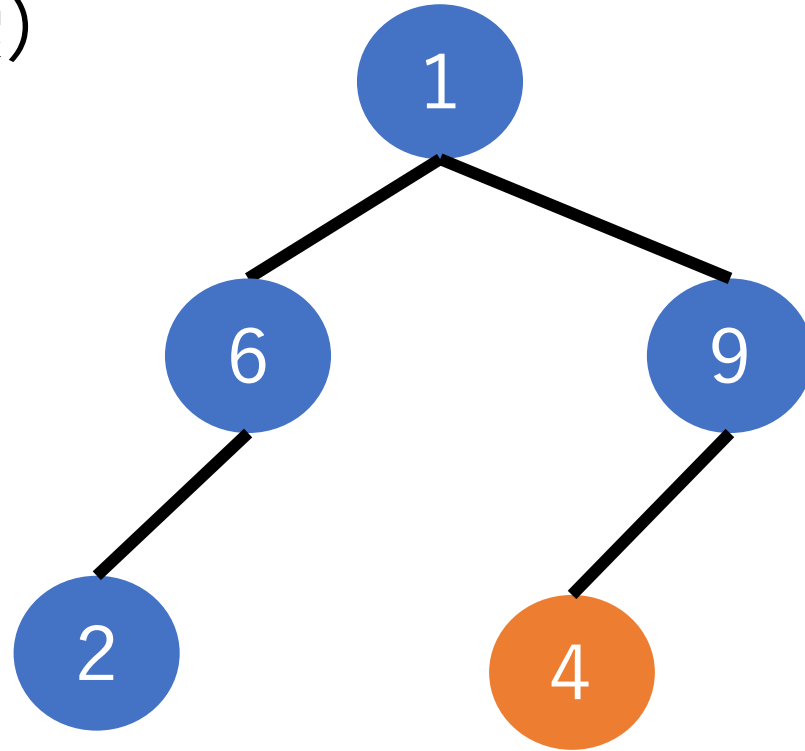
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



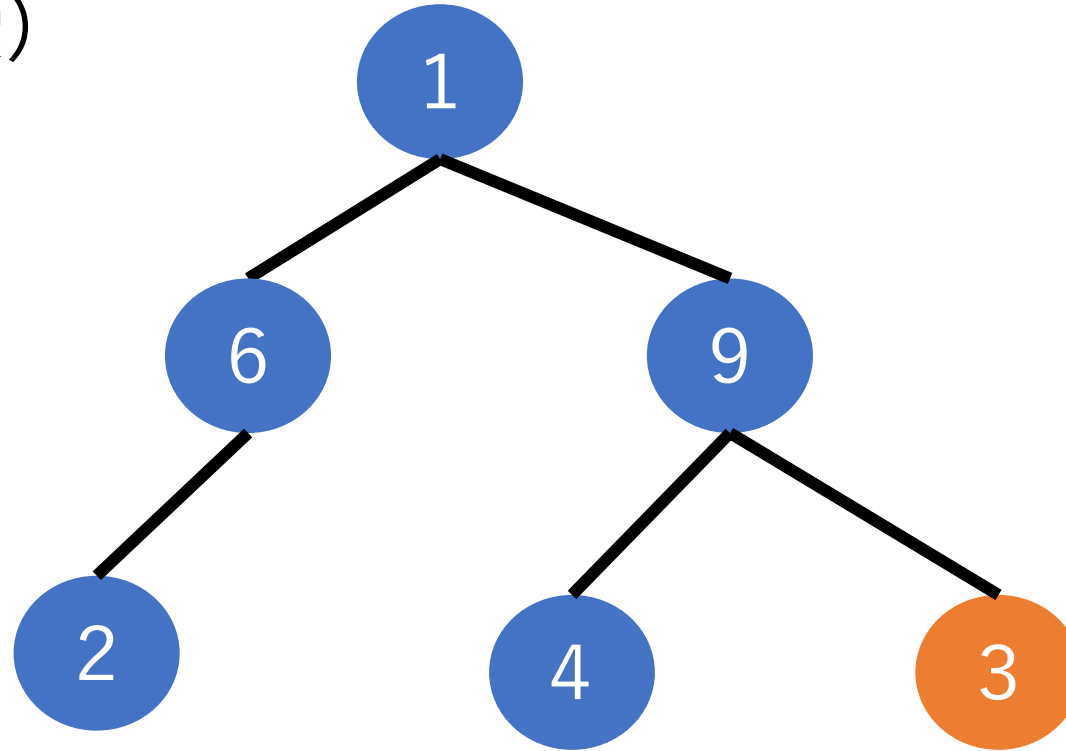
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



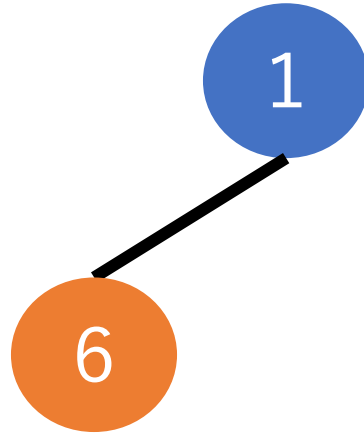
B. 頂点 1 から近い順に探索していく

BFS (幅優先探索)



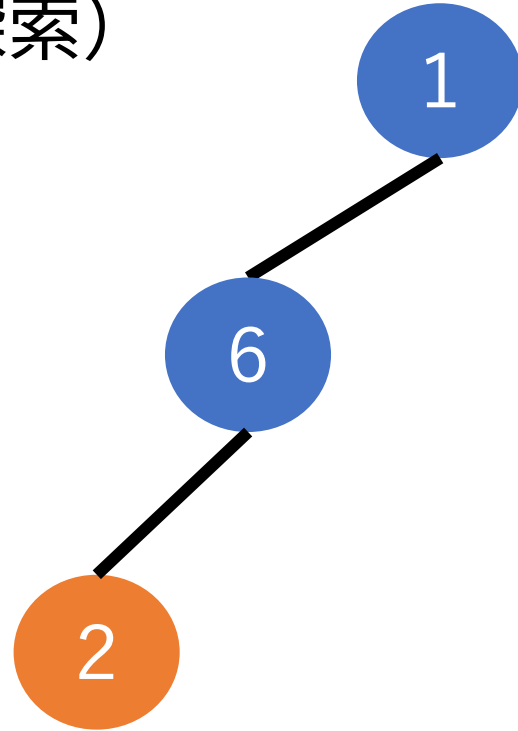
B. 頂点 1 から近い順に探索していく

DFS（深さ優先探索）



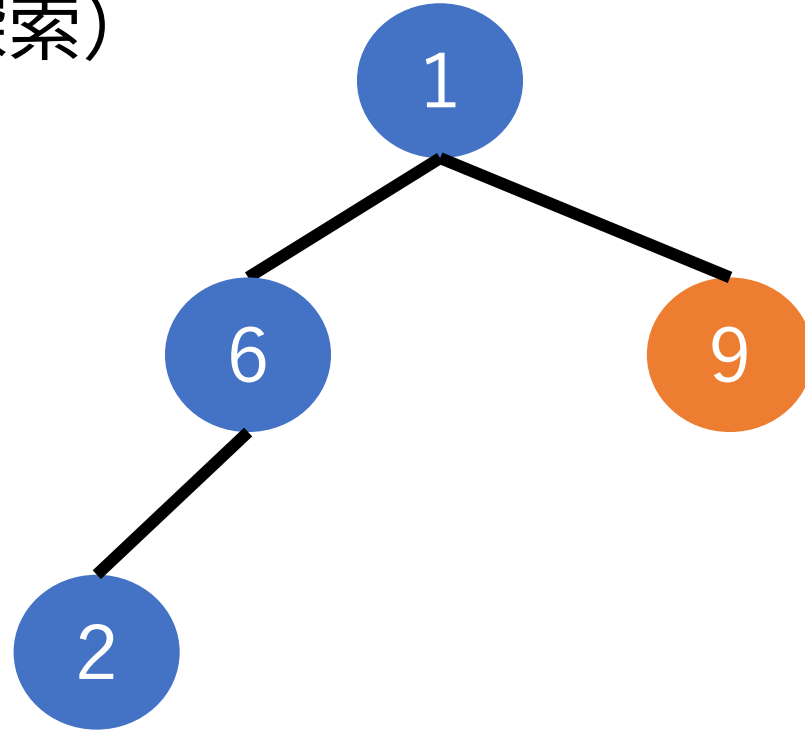
B. 頂点 1 から近い順に探索していく

DFS（深さ優先探索）



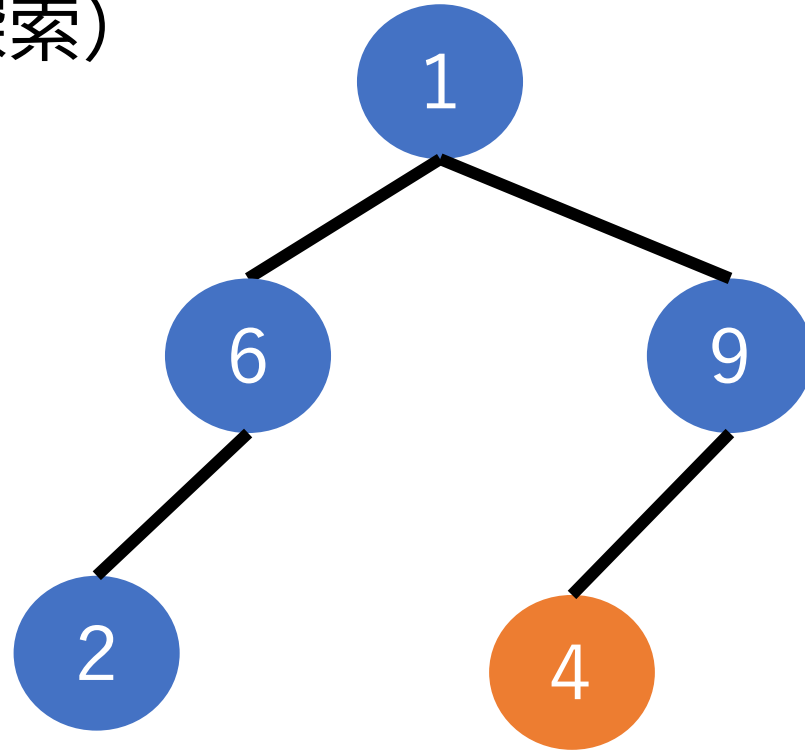
B. 頂点 1 から近い順に探索していく

DFS (深さ優先探索)



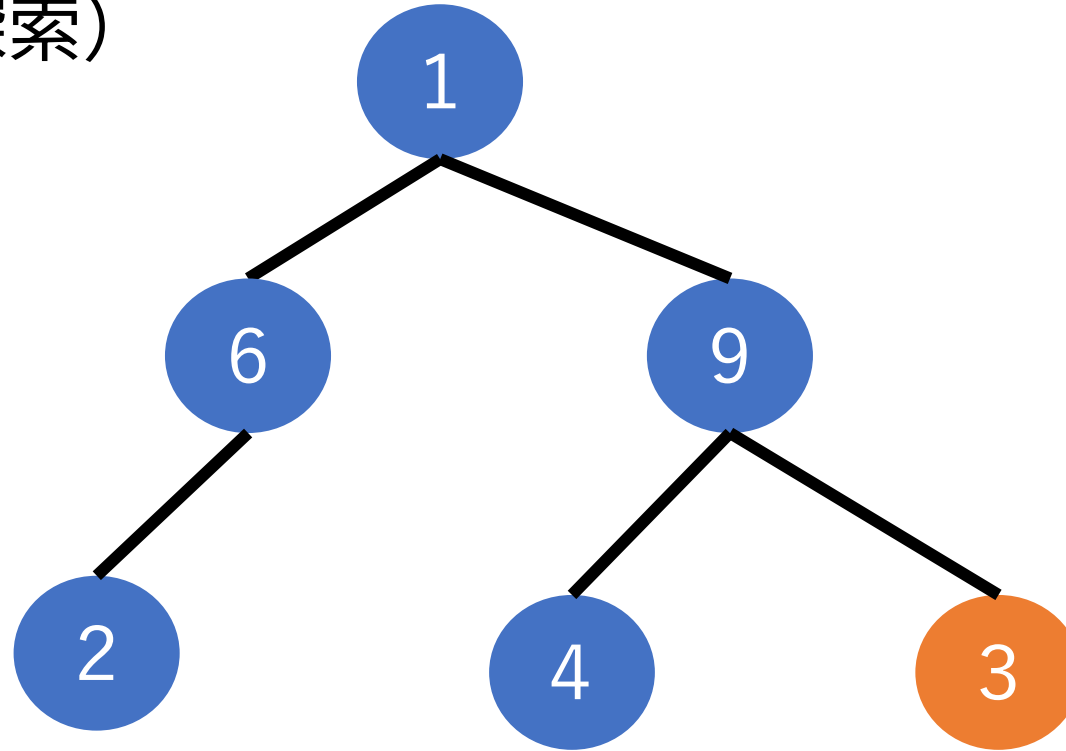
B. 頂点 1 から近い順に探索していく

DFS (深さ優先探索)



B. 頂点 1 から近い順に探索していく

DFS (深さ優先探索)

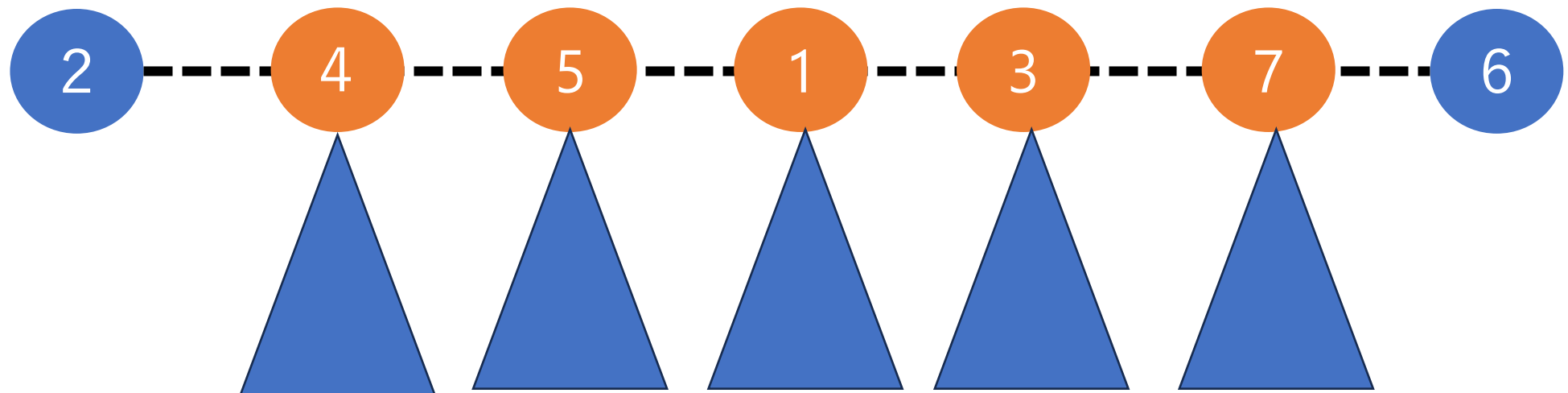


C. 葉から削っていく

まず何らかの方法で1つの葉とその接続先を見つける

→その頂点を取り除いて、頂点数が1つ少ない問題に帰着させる

D. 直径の利用

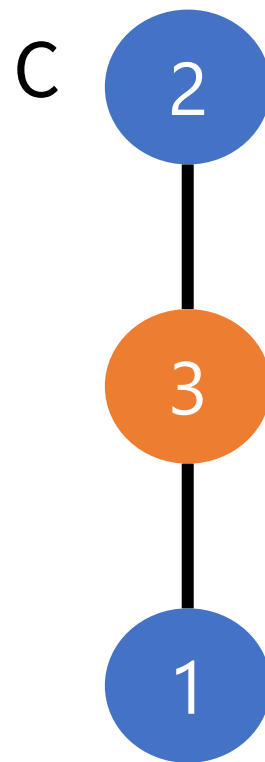
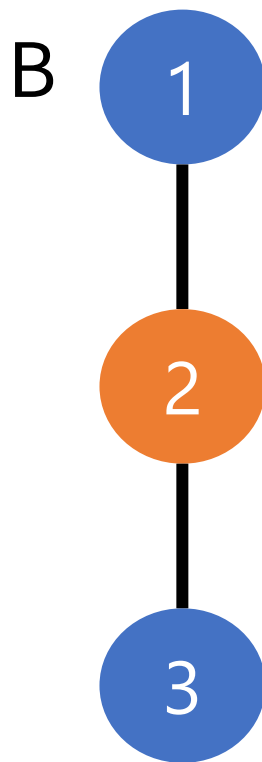
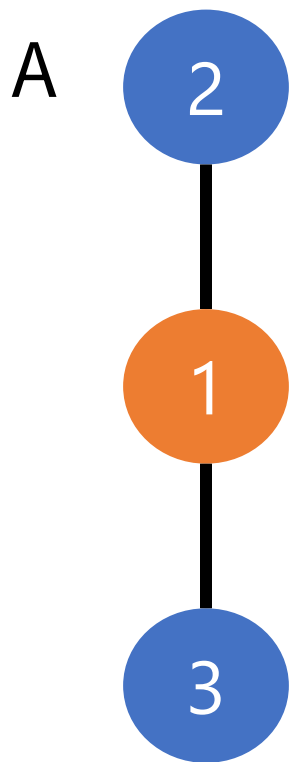


小課題 1

N=3

気づき

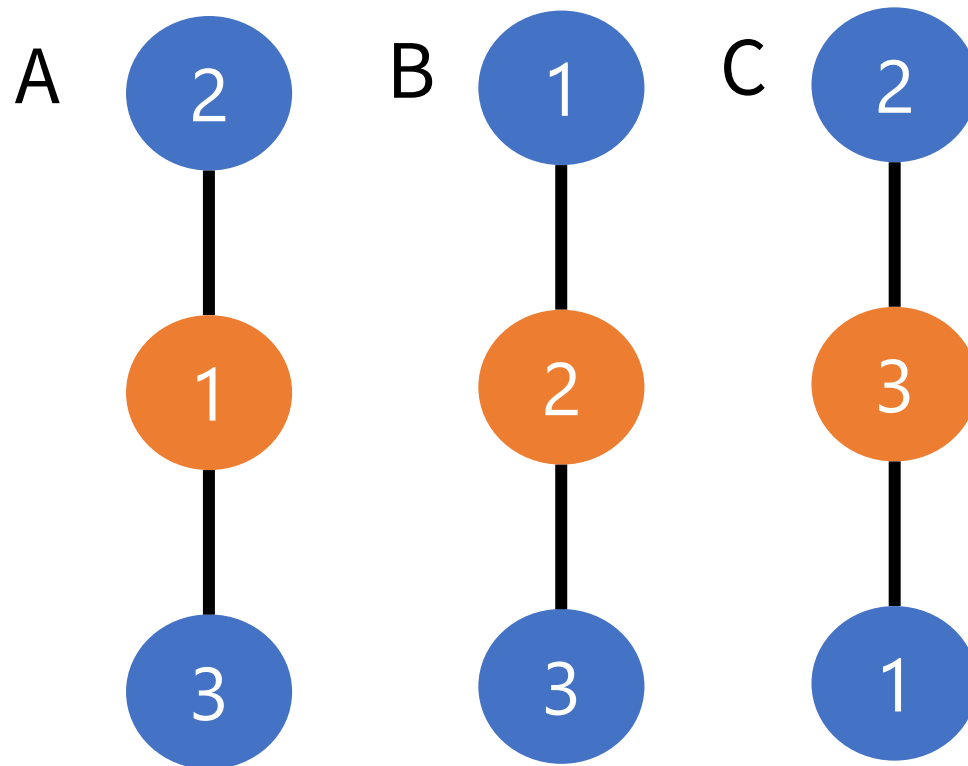
3頂点の木は種類が少なく、簡単に列挙できる



気づき

頂点 i から最も近い距離の頂点を聞く

	$i=1$	$i=2$	$i=3$
A	2	1	1
B	2	1	2
C	3	3	1



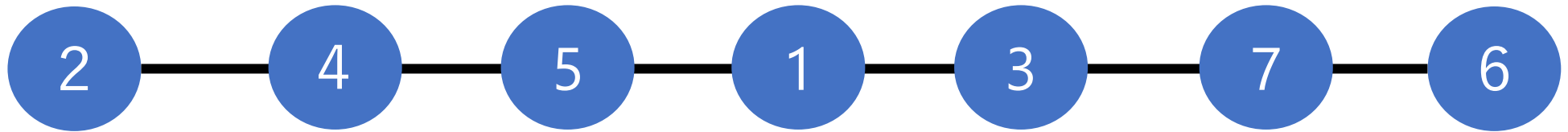
```
void solve(int N, int L) {  
    // N = 3, L = N^2  
    if (query(1, 1) == 3) {  
        answer(1, 3);  
        answer(2, 3);  
    }  
    else {  
        answer(1, 2);  
        answer(query(3, 1), 3);  
    }  
}
```

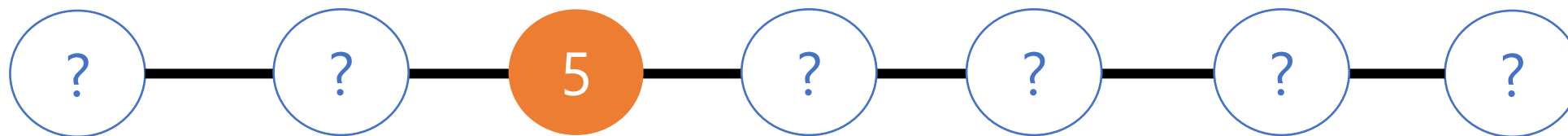
→小課題1 AC!

小課題2,3

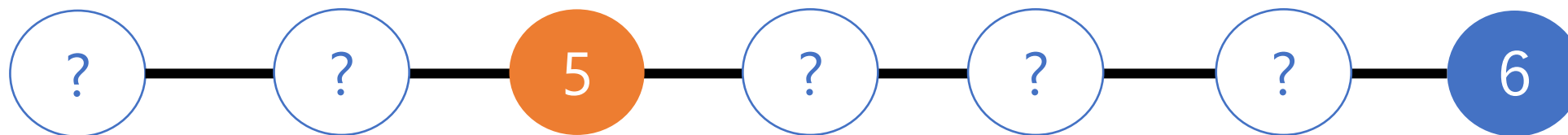
パスグラフ

パスグラフ

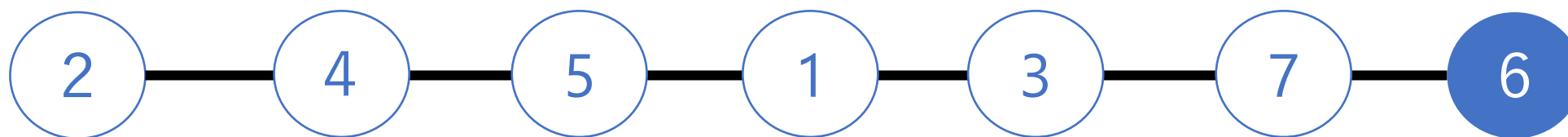




適当な頂点から最も遠い頂点を聞く



適当な頂点から最も遠い頂点を聞く



その頂点から近い頂点を順に聞く

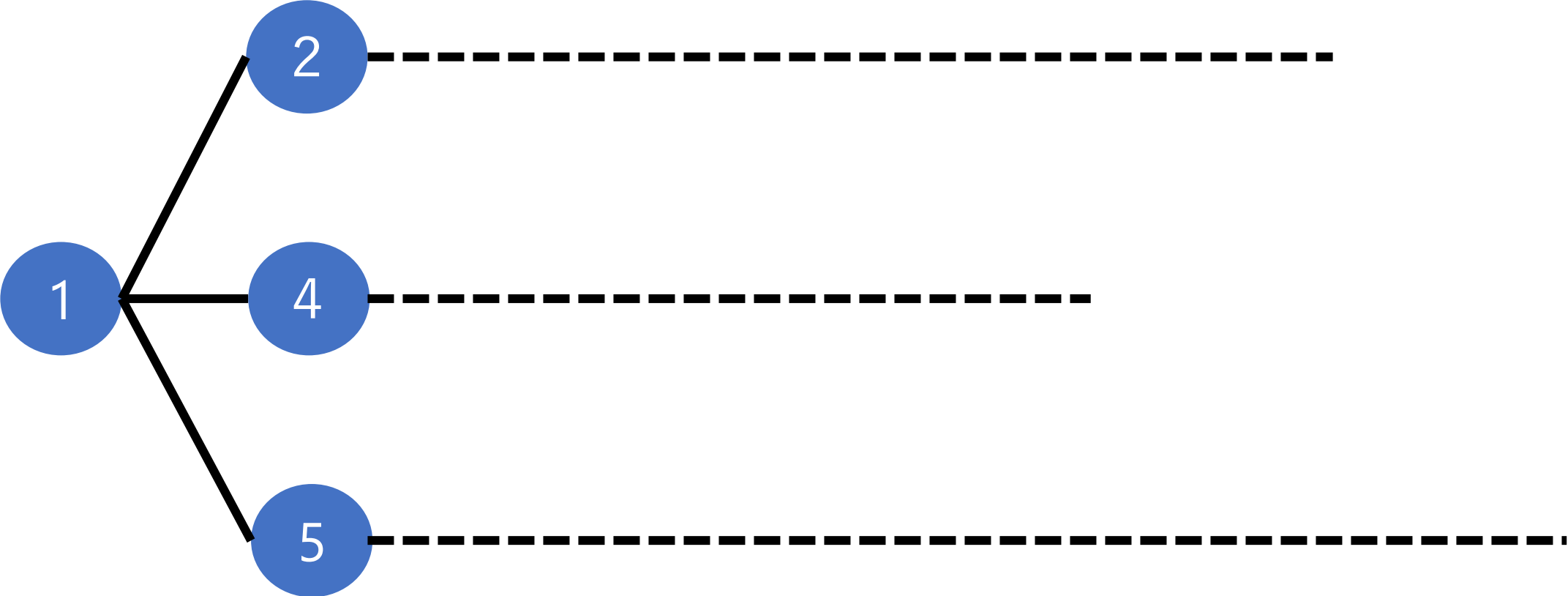
```
void solve(int N, int L){  
    // path, L = 2N  
    int a = query(1, N - 1), b;  
    b = a;  
    for(int i = 0; i < N - 1; i++){  
        int c = query(a, i + 1);  
        answer(min(b, c), max(b, c));  
        b = c;  
    }  
}
```

→小課題2,3 AC!

小課題4,5

特殊な形のグラフ

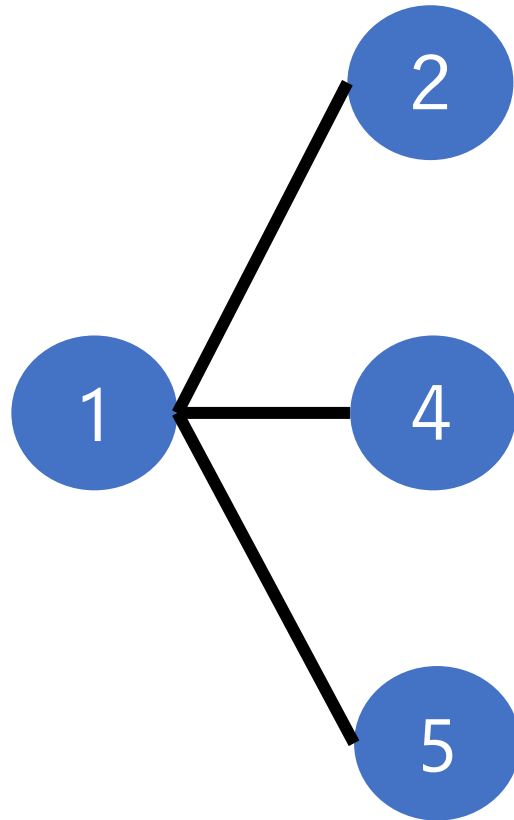
特殊な形のグラフ



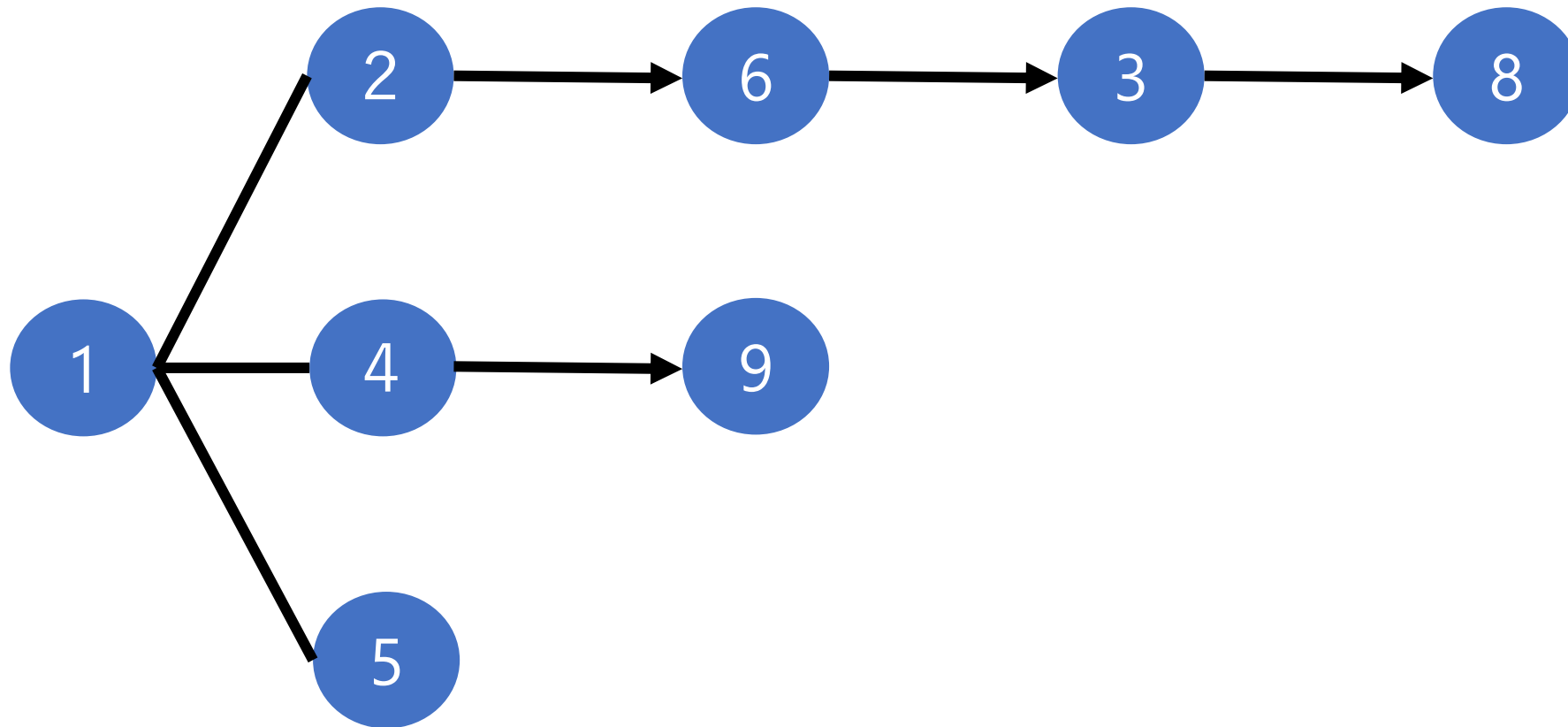
木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

最初に頂点1に接続する3頂点の番号を求める



DFSする（今いる頂点から近い2頂点を聞く）



```

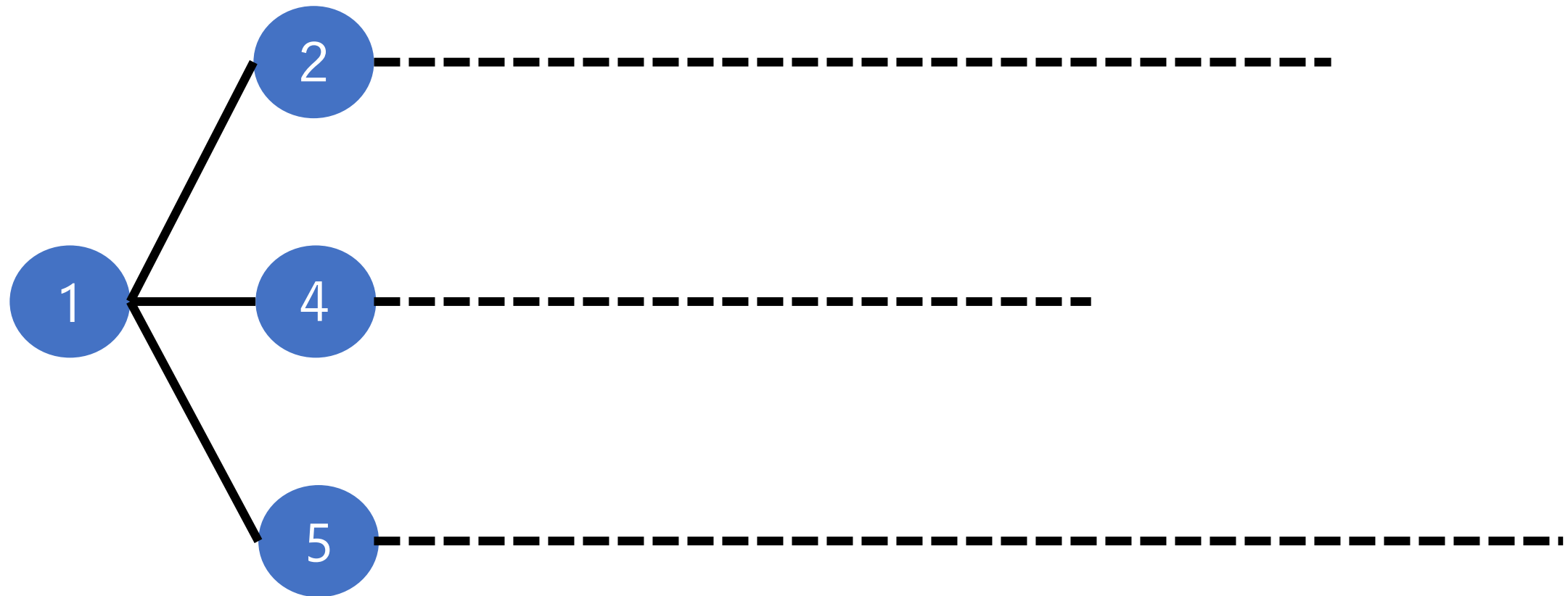
void solve(int N, int L){
    vector<int> ok(N + 1, 0);
    vector<int> v = {query(1, 1), query(1, 2), query(1, 3)};
    for(int i = 0; i < 3; i++){
        answer(1, v[i]);
    }
    ok[1] = 1;

    for(int i = 0; i < 3; i++){
        while(true){
            int b = query(v[i], 1);
            int c = query(v[i], 2);
            if(ok[b] == 0){
                ok[v[i]] = 1;
                answer(v[i], b);
                v[i] = b;
            }
            else if(ok[c] == 0){
                ok[v[i]] = 1;
                answer(v[i], c);
                v[i] = c;
            }
            else{
                break;
            }
        }
    }
}

```

→小課題4,5 AC!

(別解)



木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

頂点1から最も距離の遠い頂点を聞く

→その頂点から近い順に決めていく



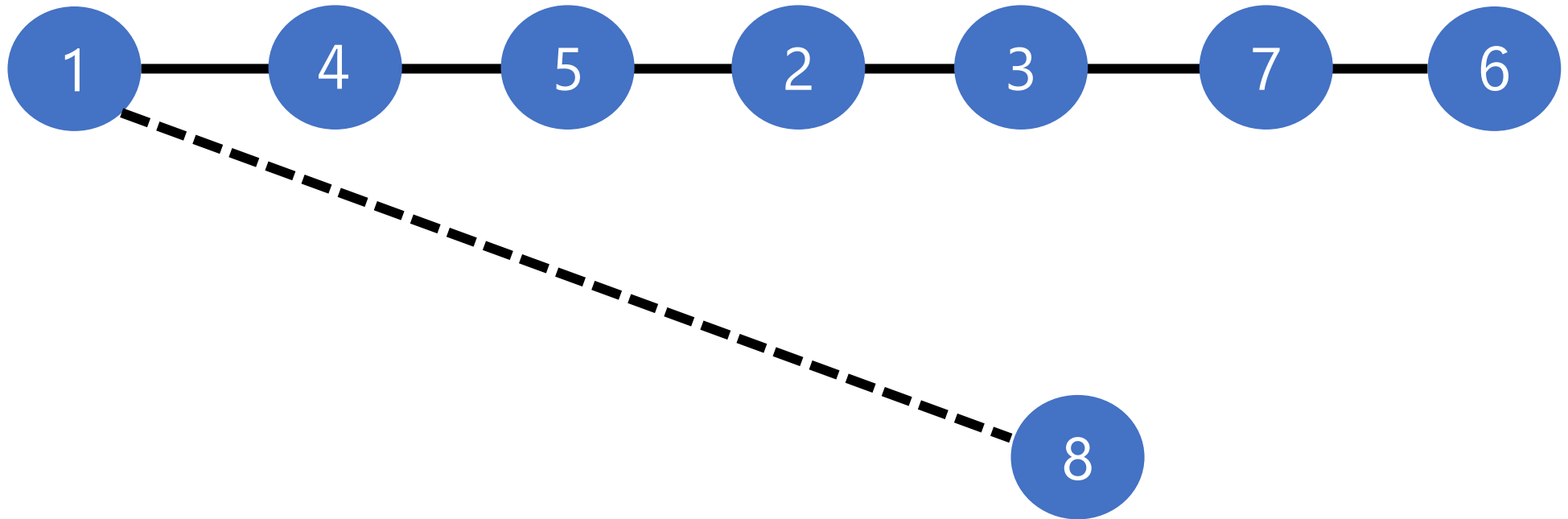
頂点1から最も距離の遠い頂点を聞く

→その頂点から近い順に決めていく

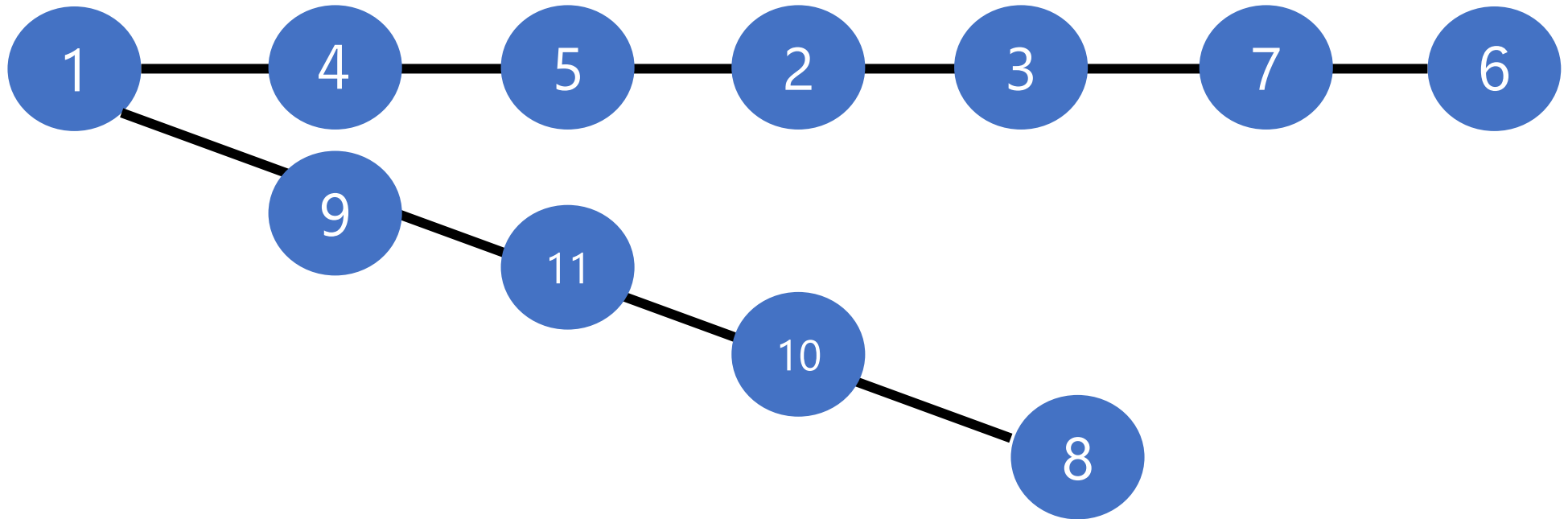
→頂点1が来たら打ち切り



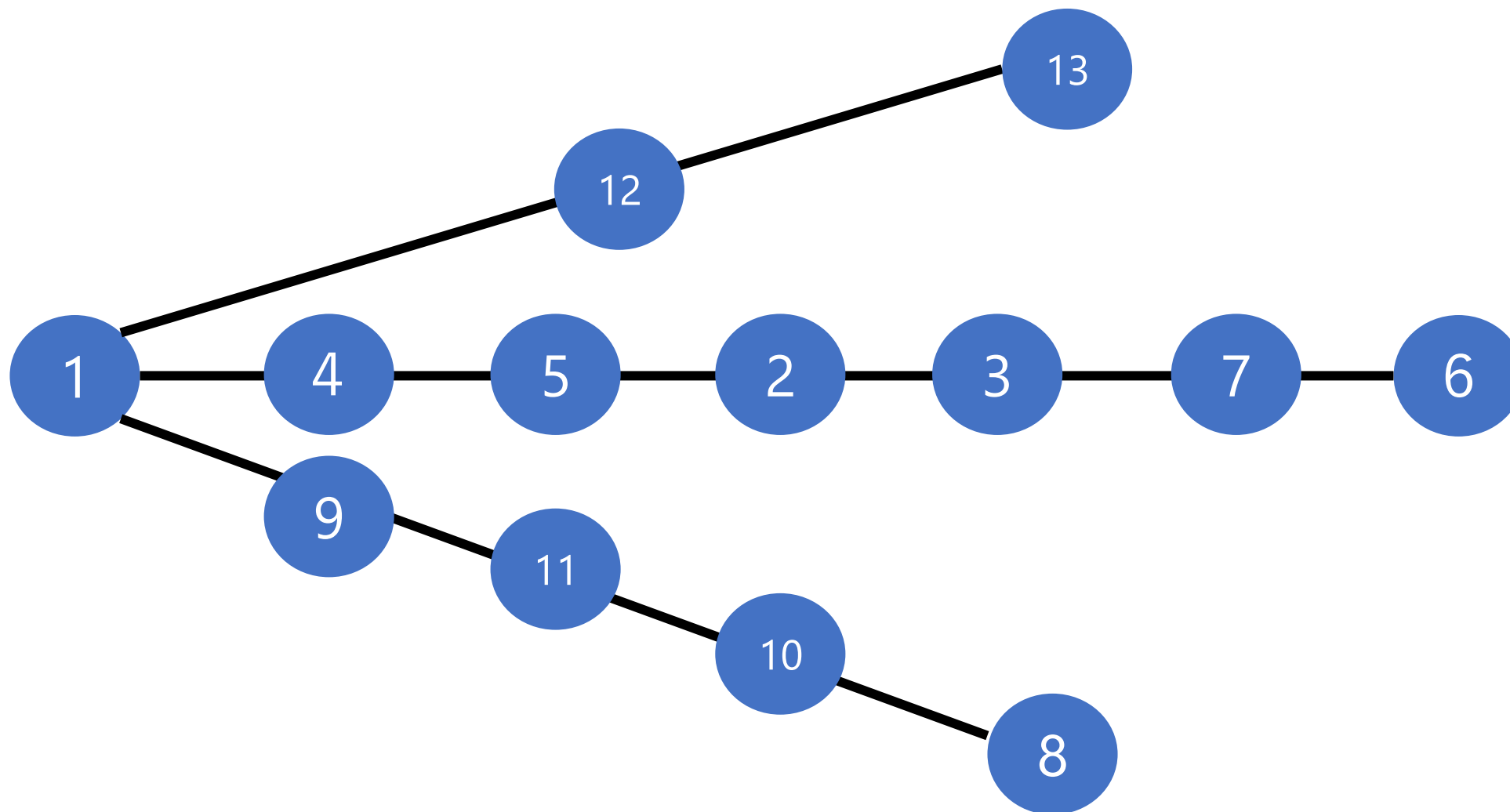
頂点6から最も遠い頂点を聞く



頂点8からも同様に聞いていく(頂点1で打ち切り)



残りの頂点は 1 からの近い順につなげる



小課題6

$$L=N^2$$

木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

頂点1から最も距離の遠い頂点を聞く

→その頂点は葉であり、その接続先も分かる

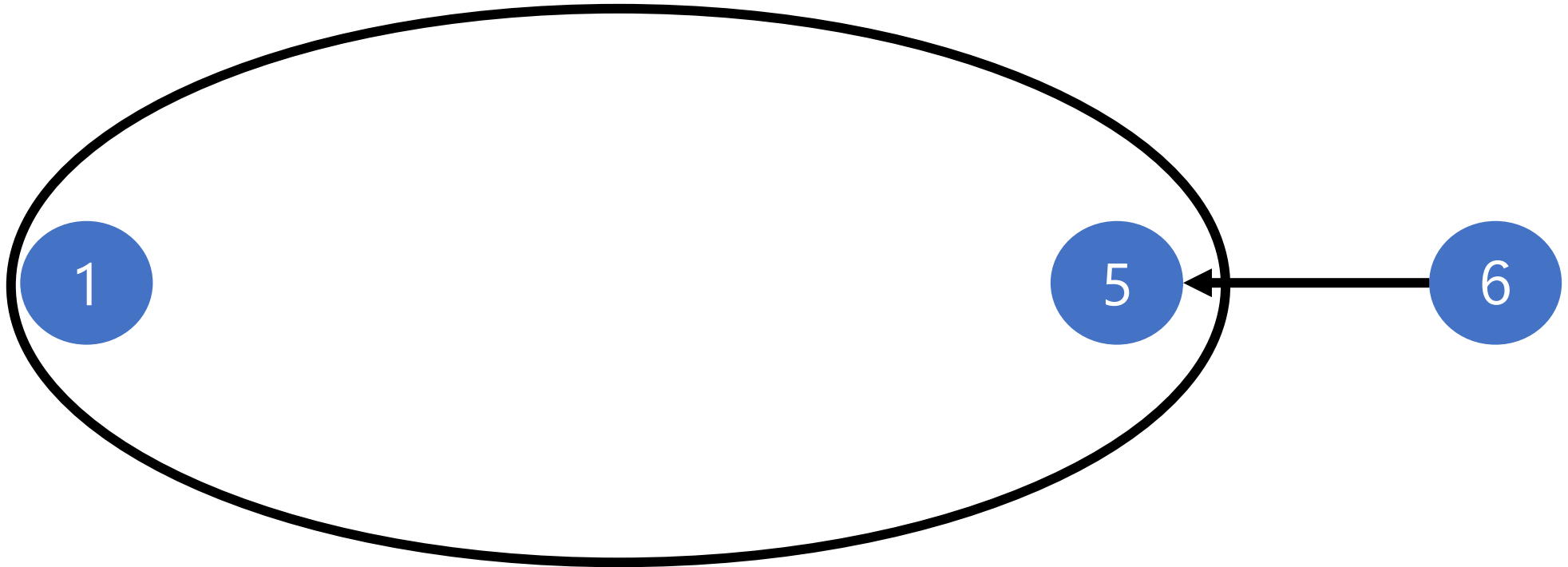
→その葉を無視して残りの木を再帰的に求めればよい



頂点1から最も距離の遠い頂点を聞く

→その頂点は葉であり、その接続先も分かる

→その葉を無視して残りの木を再帰的に求めればよい



```

vector<int> used(N, 0);
int a = Q[0][N - 1];

for(int i = 0; i < N - 1; i++){
    for(int j = 1; j < N; j++){
        int b = Q[a][j];
        if(!used[b]){
            used[a] = 1;
            answer(a + 1, b + 1);
            for(int k = N - 1; k >= 0; k--){
                int c = Q[b][k];
                if(!used[c]){
                    a = c;
                    break;
                }
            }
            break;
        }
    }
}

```

→小課題6 AC!

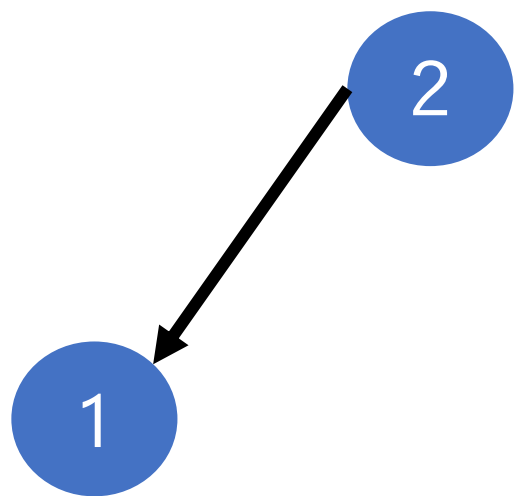
小課題7,8

$L=3N, 2N$

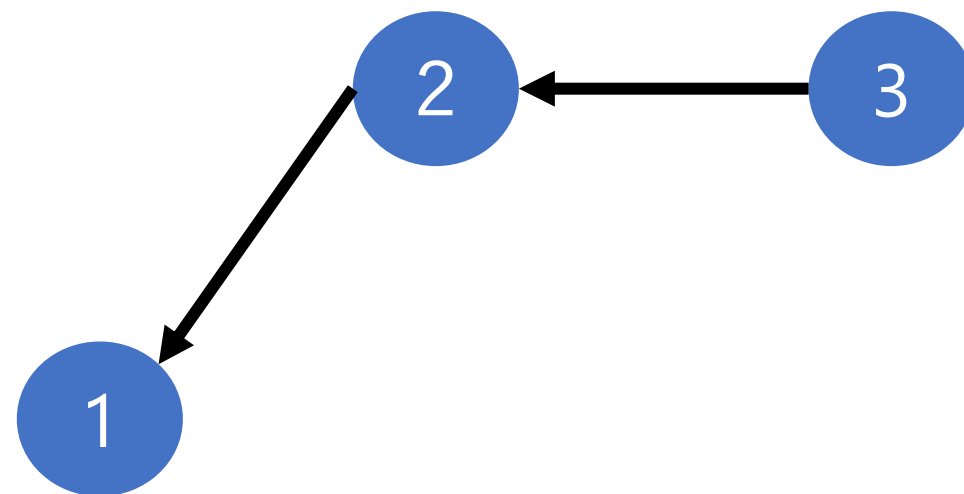
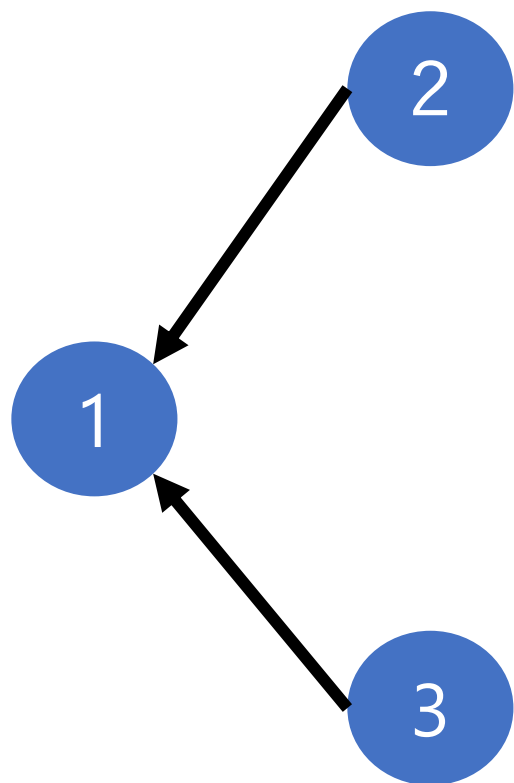
木を特定する問題における典型考察

- A. 頂点 $1, 2, \dots, N$ の順に追加していく
- B. 頂点 1 から近い順に探索していく
- C. 葉から削っていく
- D. 直径の利用

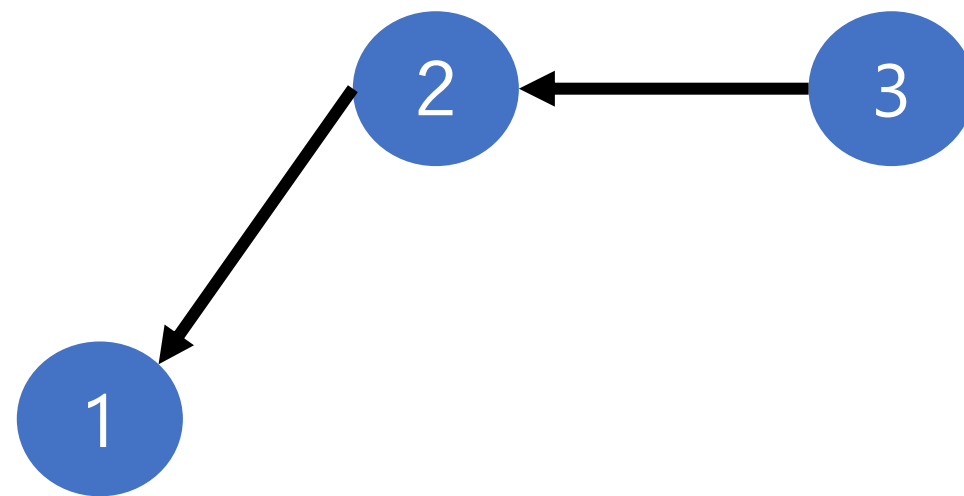
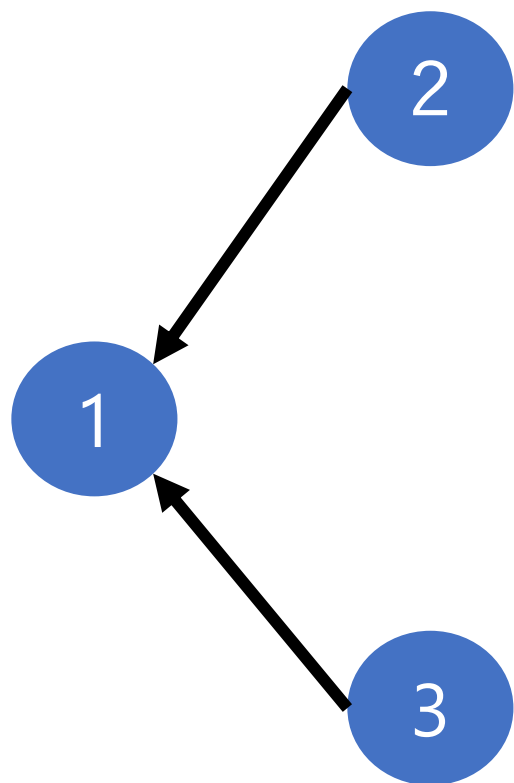
頂点1から最も近い頂点をつなげる



頂点1から次に近い頂点をつなげる



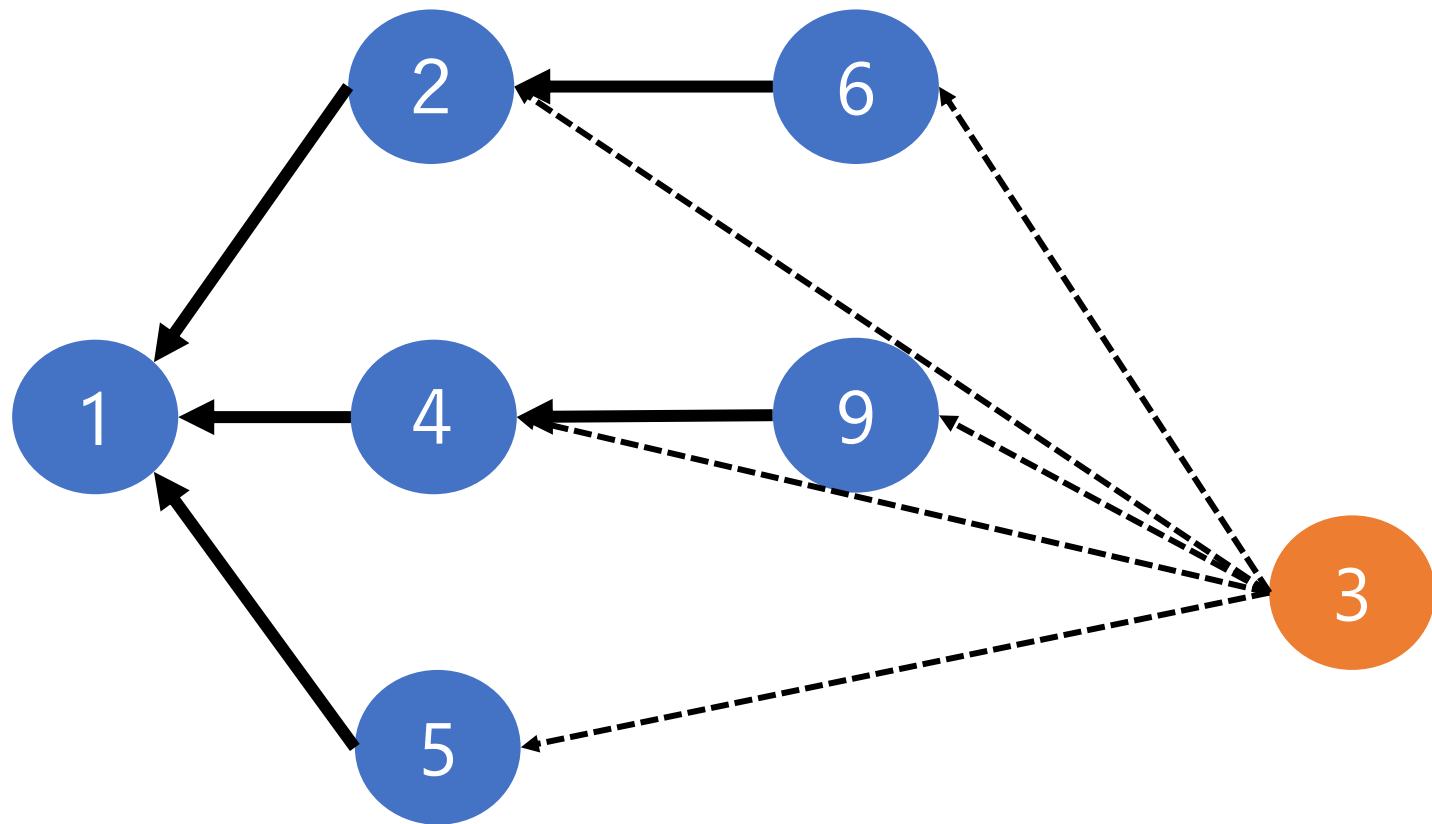
(どっちか分からない)



頂点3から近い頂点を順に聞いていき、
頂点1と頂点2のどちらが先に現れるか
で判定

まとめると・・・

- 1． 頂点1から近い順に頂点を追加していく
- 2． 追加したい頂点から近い頂点を順に質問していく
- 3． 既に見た頂点があればそれが親に確定する

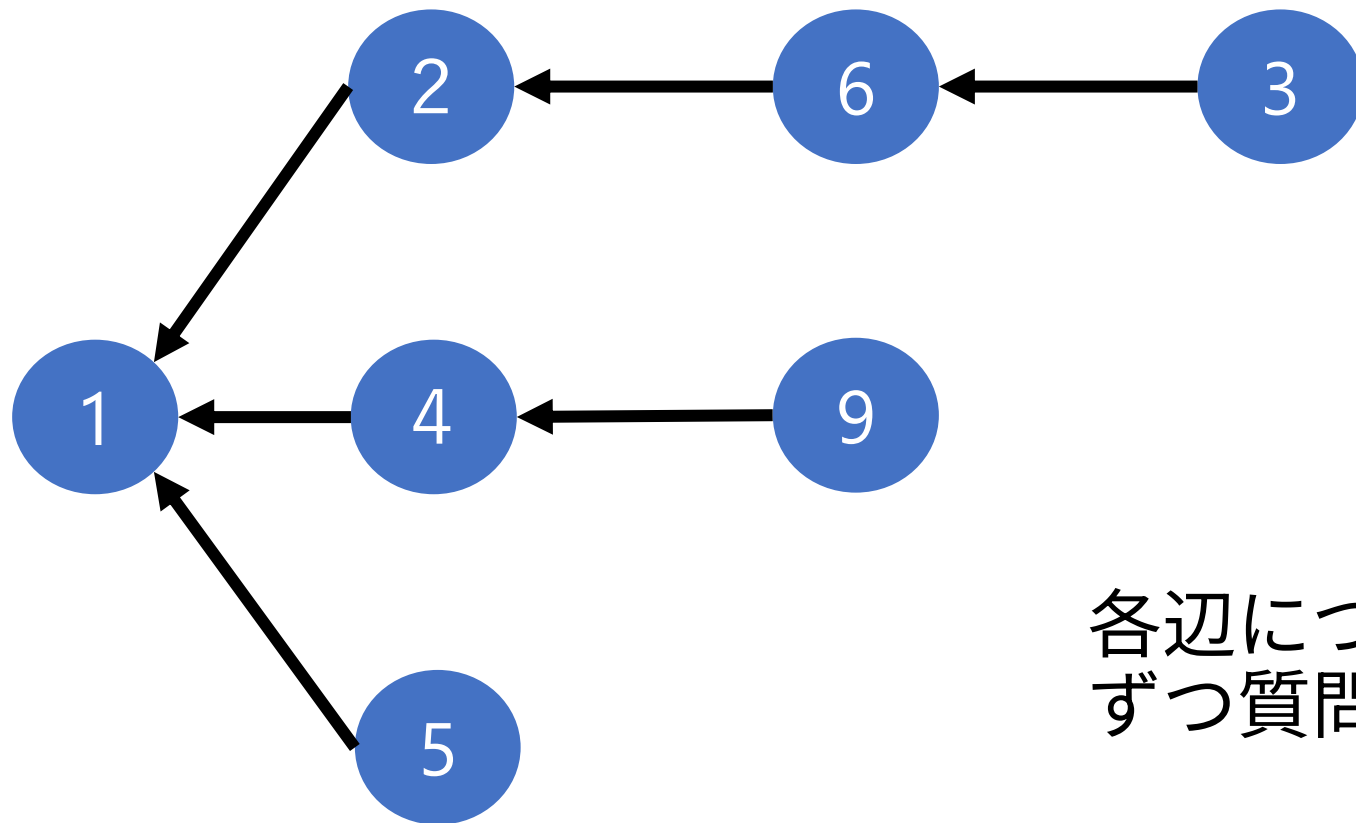


まとめると・・・

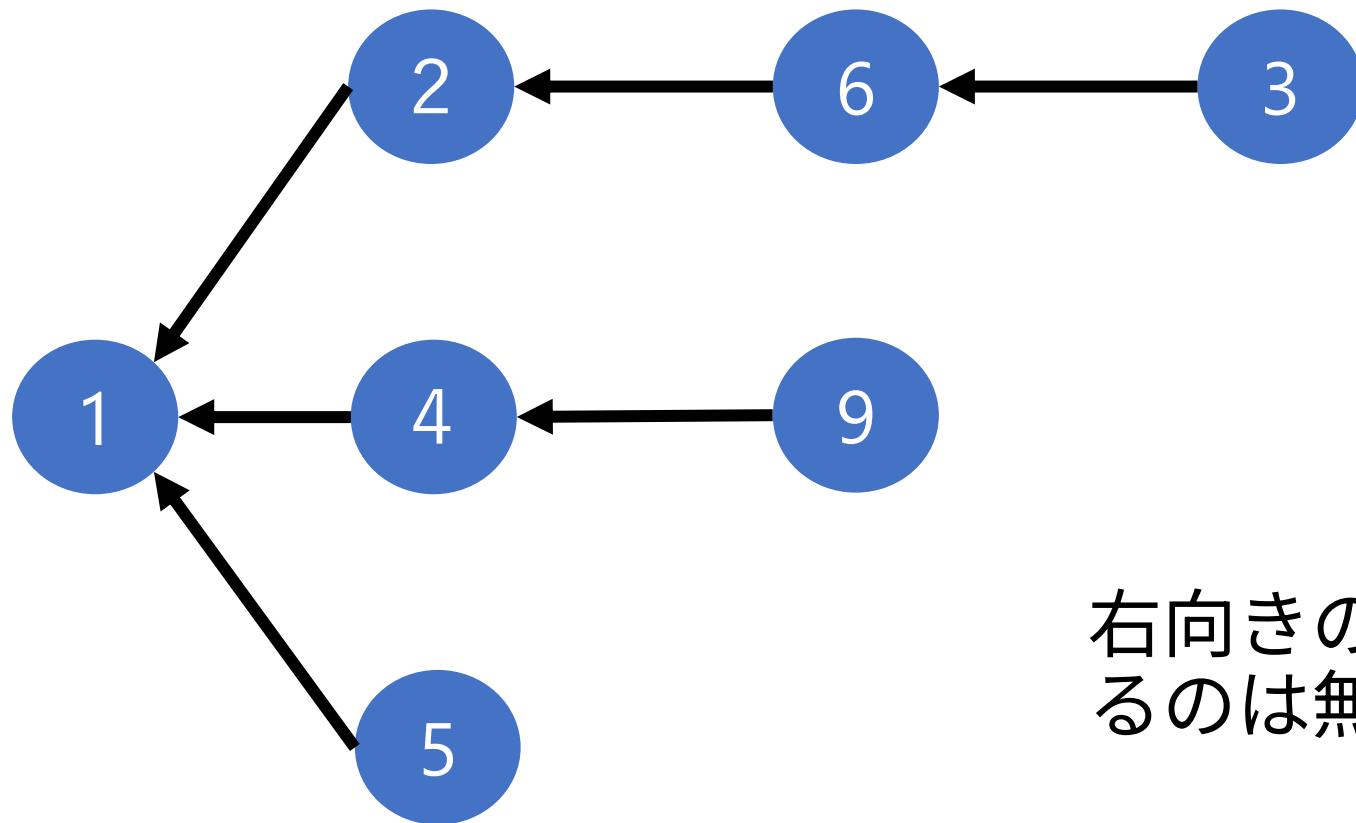
1. 頂点1から近い順に頂点を見ていく
2. 追加したい頂点から近い頂点を順に質問していく
3. 既に見た頂点があればそれが親に確定する

最初の N 回 + $2N$ 回 $\rightarrow$ 合計 $3N$ 回

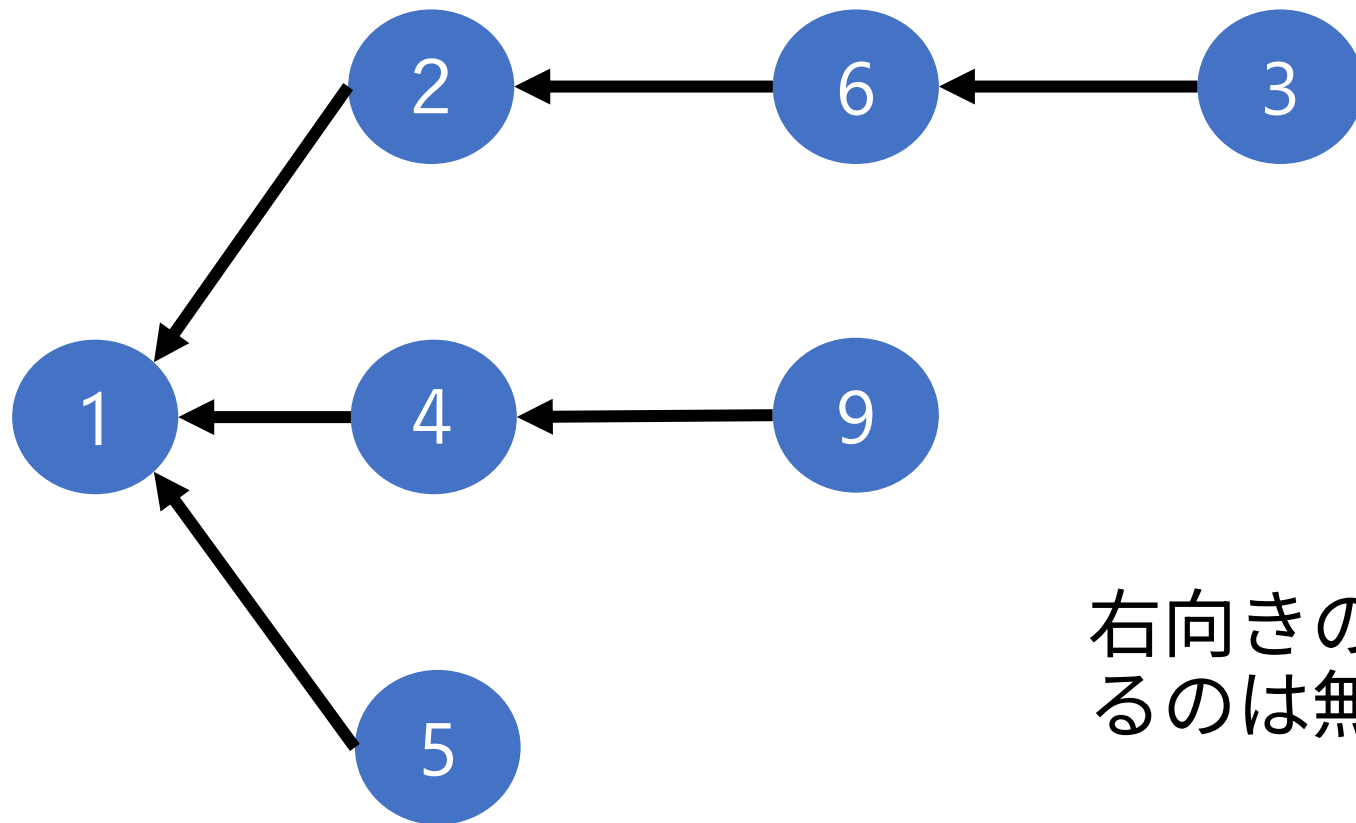
なぜか？



各辺について左向きと右向きの2回
ずつ質問することになる



右向きの辺を聞いた時の情報を捨てるのは無駄



右向きの辺を聞いた時の情報を捨てるのは無駄

→再利用すれば合計 $2N$ で満点が得られる

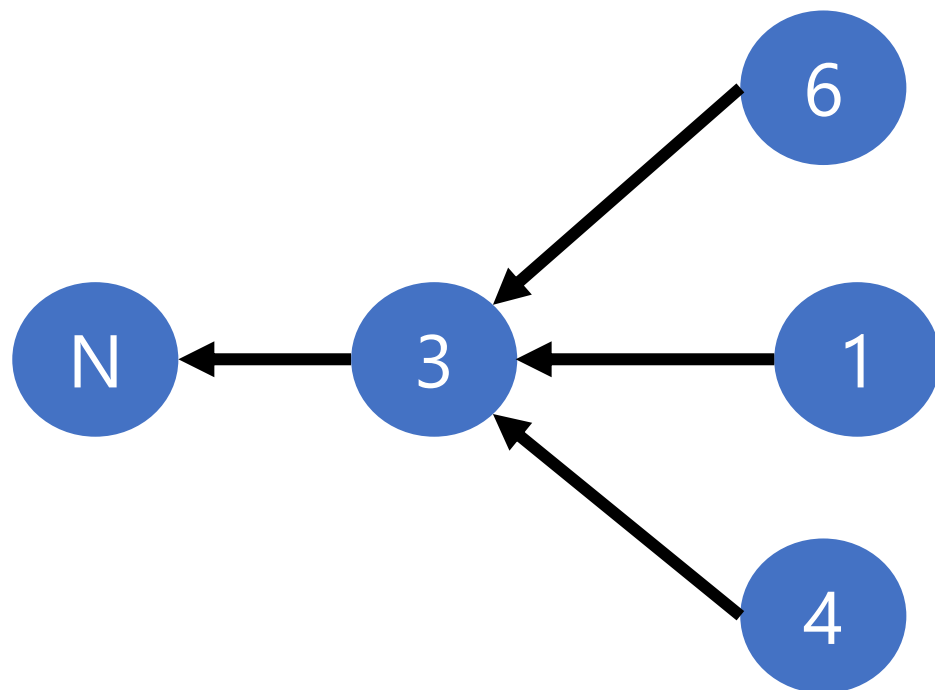
```
for(int i = 1; i < N; i++){
    int a = query(1, i) - 1;
    dist[a] = i;
    near[i] = a;
}

for(int t = 1; t < N; t++){
    int i = near[t];
    if(ok[i]){
        continue;
    }
    for(int j = 1; j < N; j++){
        int a = query(i + 1, j) - 1;
        if(dist[a] < dist[i]){
            answer(i + 1, a + 1);
            break;
        }
        else{
            answer(i + 1, a + 1);
            ok[a] = 1;
        }
    }
}
```

→100点!

頂点番号の大小をうまく利用する解法

・・・ 頂点 N の隣の頂点を聞く



→ $L=2.5N$ で満点は厳しい

得点分布

JOI

100点：5人

26点：7人

72点：9人

4点：4人

50点：3人

0点：0人

JOIG

66点：1人

20点：1人

52点：1人

4点：4人

41点：1人

0点：1人



JOI/JOIG 2023/2024 春季トレーニング 卓球 (Table Tennis) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- N 人の総当たり戦の結果を作る
- 条件「3 すくみの数が M 個」を満たしたいです
- 可能か不可能か判定、可能なら 1 つ作ってください
- 1 つの入力ファイルで Q 個解いてください

0

制約

- $1 \leq Q$
- $N \leq 5\,000$
- $M \leq {}_N C_3$
- N の総和は 5 000 以下

小課題番号	内容	配点 (JOI)	配点 (JOIG)
1 (JOIG のみ)	$M = 0$	0 点	5 点
2	$M \leq N - 2$	5 点	12 点
3 (JOIG のみ)	$M \leq (N-1)^2/4$	0 点	15 点
4	N の総和 ≤ 7	4 点	10 点
5	N の総和 ≤ 20	23 点	23 点
6	N の総和 ≤ 150	30 点	19 点
7	N の総和 ≤ 600	15 点	8 点
8	追加の制約なし	23 点	8 点

0

小課題

- 以降の小課題の番号は JOIG に準拠します

0 マルチテストケースの実装

- 1つの入力ファイルでテストケースが複数ある問題はたまにみえます(マルチテストケースとか言ったりする)
- 以下みたいに書くと楽です

```
void solve(){  
    // ここに処理  
}  
int main(){  
    int t; cin >> t;  
    while(t--){  
        solve()  
    }  
}
```

Subtask 1

$$M = 0$$

1

考察

- 3 すくみが 0 個
- 強い人がとても勝って、弱い人がとても負けてそう

1

考察

- つまり各人に強さを決めて、その強さの大小で勝敗を決めれば OK

Subtask 2

$$M \leq N - 2$$

2

考察

- 強さの設定はそのままにしたい
- 1 個逆転が起こったとする
- 強さが A 番目に小さい人と B 番目に小さい人の結果を逆転すると 3 すくみが $B - A - 1$ 個できる

2

B を探す

- $A = 1$ とすると $B - 2 = M$ になるような B を見つけたい
- $M \leq N - 2$ なのでそのような B は N 以下
- つまり必ず存在！

Subtask 3

$$M \leq (N-1)^2/4$$

3

考察

- $O(N^2)$ の式ということは N ごとに何か足しそう
- 係数が $1/4$ なので 2 人ずつ何かをしそう？
- 2 人以外と 2 人の間の関係を考える

3

考察

- 2 人を A, B として A は B に勝ったとする
- 後の人は全員 A に勝てた or 負けたとしよう
- B に対して同様
- 全員 A に勝てた & B に負けたとすると 3 すくみは $N - 2$ 個
- $N - 2$ 人の場合の問題になる

3

考察

- $(N - 1)^2/4 - (N - 2) \leq (N - 3)^2/4$ なので
- N を 2 小さくした小課題 3 の問題に帰着される！
- 実装上の注意として、 N が 6 以下ぐらいで場合分けが必要
- これをどうするかは次の小課題 4 で

Subtask 4

$$N \leq 7$$

4

考察

- 試合の数は $_NC_2 \leq 21$
- bit 全探索ができそう
- 3 すくみの数は愚直に数えて $O(N^3)$
- $O(2^{(N/2)} N^3)$ で解けました

4

埋め込み

- この場合ありうる (N, M) の数が少ない
- 全通り手で解いて答えをソースコード中に埋め込む
- 出力 $O(N^2)$ で OK !
- 10 ケースを超えるのを手で解くなら多分他の小課題に手をつけた方がいいと思います

Subtask 5

$$N \leq 20$$

5

3 すくみの数を数える

- 制約に $N \leq 5\,000$ とある
- 3 すくみの数を $O(N^3)$ より速くジャッジしてそう
- $O(N^2)$ で数える方法を考える
- 3 すくみでない数を数えられないか

5

3 すくみの数を数える

- A, B, C が 3 すくみでないときその 3 人の中の試合結果を考える
- A が B, C に勝って、B が C に勝ってる感じ
- つまり A が 2 勝している
- $N - 1$ 人のうち A が勝った人のうち 2 人選ぶと、A が 2 勝してるタイプの 3 すくみになる
- これで $O(N^2)$ で数えられる

5

問題の言い換え

- つまり人 i の勝利数 $d(i)$ を適切に決めて $d(i)C_2$ の総和を ${}_NC_3 - M$ にしたい
- まず $d(i)$ の条件を考える $d(1) \leq d(2) \leq \dots$ としよう
- 必要条件を考える
- $0 \leq d(i) \leq N - 1$
- $d(1)$ から $d(i)$ までの総和が ${}_iC_2$ 以上
- $d(1)$ から $d(N)$ までの総和が ${}_NC_2$

- 逆にこれが十分条件になっている(ランダウの定理)
- 証明: 略
- とにかくこれで DP をできそう
- DP[d(i)まで決めた][ここまでの d(i) の総和][ここまでの $\sum_{d(i)} C_2$ の総和]
- 計算量 $O(N^8)$ で d(i) を求められる

5

結果を作る

- $d(i)$ から試合結果を作りたい
- 人 N に着目 この人は $d(N)$ 勝っている
- 直感: 人 1 から人 $d(N)$ までには勝ってそう
- これでやると実はうまくいく
- 証明: ランダウの定理の正当性を認めるとできます

5

結果を作る

- 人 N は人 1 から人 $d(N)$ までには勝って人 $d(N) + 1$ から人 $N - 1$ までには負けたとする
- $d(d(N) + 1)$ から $d(N - 1)$ までを 1 減らす(人 N に勝ったので)
- $d(1)$ から $d(N - 1)$ を sort する(これで人の順番が変わるので最初どの人だったかを管理する)
- $O(N \log N)$ で人を 1 人減らせるので $O(N^2 \log N)$
- $N - 1$ 人の結果をもらった後 N 人の場合の番号に直すと $O(N^3)$

5

解法

- DP で $d(i)$ を求める $O(N^8)$
- 再帰的に試合結果を作る $O(N^2 \log N)$ or $O(N^3)$
- 全体で $O(N^8)$ で解けました

Subtask 6

$$N \leq 150$$

6

判定問題を考える

- $d(i)$ を作ることを速くしたい
- ところで Yes か No の判定は？
- $d(i)C_2$ の総和を ${}_NC_3 - M$ にできるか
- 最大値は $M = 0$ のとき これは小課題 1 で達成してる
- 最小値を求めよう

6

最小値を求める

- 一旦条件「 $d(1)$ から $d(i)$ までの総和が $_iC_2$ 以上」を無視して「 $0 \leq d(i) \leq N - 1$ 」「 $d(1)$ から $d(N)$ までの総和が $_NC_2$ 」だけ考える
- このとき $i < j$ について $d(i) + 2 \leq d(j)$ なら $d(i)$ に 1 を足して $d(j)$ に 1 を引くと $_{d(i)}C_2$ の総和が小さくなる
- $(_{d(j)}C_2 + _{d(i)}C_2) - (_{d(j)-1}C_2 + _{d(i)+1}C_2) = d(j) - d(i) - 1 \geq 1$
- そのような i, j がない状態が最小
- つまり $d(1) + 2 > d(N)$

6

最小値を求める

- このような $d(i)$ は N の偶奇で異なる
- N が偶数のとき $d(i)$ は $N / 2$ と $N / 2 - 1$ が $N / 2$ 個ずつ
- N が奇数のとき $d(i)$ は全部 $(N - 1) / 2$
- またこのとき「 $d(1)$ から $d(i)$ までの総和が ${}_iC_2$ 以上」が成り立っている
- ${}_{d(i)}C_2$ の総和の最小値が求まりました

6

間はどうか

- $d(i)C_2$ の総和が最小となる実際の試合結果は人 i が $i+1$ から $i+d(i)$ までに勝って、残り負けてるとすれば実現可能
- $d(i)C_2$ の総和の最小値と最大値の間にしたいときどうしよう
- $(d(j)C_2 + d(i)C_2) - (d(j)-1C_2 + d(i)+1C_2) = d(j) - d(i) - 1$ を思い出すと、 $d(i) = d(j) (= x)$ ならこの 2 人の勝利数を $x+1$ と $x-1$ にすると 1 増やせる
- これを ${}_NC_3 - M - (d(i)C_2 \text{ の総和の最小値})$ 回すればいい

6 間はどうか

- 実はこれが可能
- i と j の間の試合結果を入れ替えてみる
- i が j に勝ってたのを入れ替えると i は $x - 1$ 勝、 j は $x + 1$ 勝になる 元々 i が j に負けてたら i は $x + 1$ 勝、 j は $x - 1$ 勝に
- いずれにせよ $d(i) = x$ となる 2 人を $x + 1$ と $x - 1$ にできる

6

解法

- 1 回あたりの操作は i, j を見つけるのに $O(N)$
- 操作回数は $O(N^3)$
- なので $O(N^4)$ でできる
- さっきの方法で直接試合結果を作ってもいいし小課題 5 で述べた方法で d を作ってから試合結果を作ってもいい
- どれでも $O(N^4)$ とかで解けます

Subtask 7

$$N \leq 600$$

7

高速化

- d を変えるところを高速化しよう
- 案 1: 1 回あたりの操作を $O(N)$ より速くしたい
- x 勝った人の番号とかを管理して、 x 勝った人が 2 人以上いるような x の一覧を set とかで管理すれば $O(\log N)$ などでも OK
- もうちょっと頑張ると $O(1)$ とかにもできます

7

高速化

- 案 2: 操作回数を $O(N^3)$ より小さくしたい
- $d(N) = N - 1$ とする このとき人 N がいなくとも他の d は変わらない
- ${}_NC_3 - M - {}_{N-1}C_2$ が $N - 1$ のときの ${}_{d(i)}C_2$ の総和の範囲なら N が小さい問題になる
- 逆にこれができないとき操作回数は $O(N^2)$ になってる
- 案 1 と案 2 のどちらを採用しても $O(N^3)$ とか $O(N^3 \log N)$ で解けます

Subtask 8

$N \leq 5\,000$

- 案 1 と案 2 の両方を採用すれば $O(N^2)$ とか $O(N^2 \log N)$ で解けます
- d から試合結果を作るのを $O(N^3)$ だと難しいかも
- 実は $O(N^2)$ で作るアルゴリズムがあったりします

Step 9

補足

9

違う視点で見る

- 乱択や山登りができそう
- 実際書いてみると $N \leq 150$ ぐらいまでは行けたりする
- 何も思いつかなかったら乱択や山登りを書いてみるのもあり



Step 10

得点分布

10 得点分布(JOI)

- 100点 : ?人
- 77点 : ?人
- 62点 : ?人
- 27点 : ?人
- 9点 : ?人
- 5点 : ?人
- 4点 : ?人

10 得点分布(JOI)

- 100点 : 1人
- 77点: 1人
- 62点: 3人
- 27点: 1人
- 9点: 11人
- 5点: 3人
- 4点: 1人

10 得点分布(JOIG)

- 42点 : ?人
- 17点 : ?人
- 15点 : ?人
- 5点 : ?人

10 得点分布(JOIG)

- 42点 : 1人
- 17点: 2人
- 15点: 1人
- 5点: 4人